C4 and ADRs of the production backend of FinApp1.0.

Author: Thomas Costa

Main body text word count: 40,738

Date (dd/mm/yyyy): 12/09/2026

Text Font: Times New Roman. Text Size: 11. Text Interline: 1.5

Language: British English

This work is protected by copyright and other intellectual property rights,

which are retained by the author.

Table of Contents

Introduction....1

1. C4 System Context....2

1.1. Diagram....2

1.2. ADR: monitoring....3

1.3. ADR: Stripe....3

1.4. ADR: Email Notification Service....3

1.5. ADR: security systems and in-container/in-component security solutions....4

2. C4 containers context....8

2.1. FinApp1.0. System's containers Diagram....8

2.2 FinApp1.0. System's container ADRs....9

2.2.1 ADR: Data and rules storage solution....9

2.2.2. ADR: structuring as a modular monolith....11

2.2.3. ADR: AI Agent and AI models used....12

2.2.4. ADR: using a RESTful API....14

2.2.5. ADR: using FastAPI:....15

2.2.6. ADR: where does Jinja2 run?....16

3. C4 components context....17

3.1. FinApp1.0 System - FastAPI container....17

3.1.1. Diagram....18

3.1.2. ADR: DDD bounded contexts as some of the app container's components....18

3.1.3. ADR: database caching....19

3.1.4. ADR: Rendering and frontend solution....20

3.2. AI Agent....22

3.1.1. Diagram....22

3.2.2. ADR: AI models choice....22

4. C4 codes context....24

4.1. ADR: concurrency strategy....24

4.2. ADR: Clean Architecture and DDD division and annotation of the code....26

4.3. ADR: function or object? 30
4.4. ADR: design patterns and SOLID Principles: 33
4.5. ADR: Use uv as dependency manager 35
4.6. ADR: DB queries code 36
5. Deployment: example of the US deployment. 38
5.1. Diagram 39
5.2. ADR: OS 39
5.3. ADR: using Oracle-python DB driver: 40
5.4. ADR: AI-FastAPI communications and merger? 40
5.5. ADR: AI models' management and storage 41
5.6. ADR: by-FastAPI/in-FastAPI TLS decryptor: 42
5.7. ADR: personal data encryption 43
5.8. ADR: user input format validation solution 43
5.9. ADR: Endpoint caching? 44
5.10. ADR: system failure management 45
5.11. ADR: load balancer for HTTP requests, DNS, and eventual failover system? 46
5.12. ADR: Hosting and virtualisation decisions 47
Options: 47
Decisions: 48
Rationale: 48
Solutions retained based on legal and industry requirements: 49
Solutions retained based on backup plan and app's statelessness 50
Containerisation solution chosen: 51
Capacity estimates 51
Expected User traffic info (scenario-based): 51
Duration of a usage cycle 51
Times of usage and impact on capacity calculation: 51
HTTP requests per second per million inhabitants: 52
Concurrent users, and new usage cycles per day per million inhabitants: 52
Needed capacity of the DB, its backup space and a standby db: 53
Disk type: 53

Optimisations: 53
Nature of queries, number of writes and number of ‘user write’ commits: 55
Peak datafile reads: 56
Peak data file random writes (physical writes): 57
Peak redo write size: 59
Online redo log files, their size, and redo archive space size 59
DB Physical disk storage space size: 61
DB Peak Storage bandwidth: 65
DB Peak memory: 65
DB CPU: 66
Needed capacity for the AI Agent: 68
AI agent Disk storage needs: 68
AI agent’s VRAM: 70
AI agent’s CPU and GPU usage 71
Needed capacity of the FastAPI container: 74
Number of FastAPI instances per deployment: 75
Disk storage: 75
FastAPI container’s CPU 75
FastAPI container’s RAM: 76
Needed capacity for the Docker overhead: 76
Needed capacity of the machine and OS runtime 77
Solutions retained based on capacity requirements: 77
Solutions retained based on the level of control I need 78
Solutions retained based on processing speed and performance 79
Solutions retained based on the level of isolation I want, and containerisation 81
Solutions retained based on price: 82
Consequences: 94
5.13. ADR: Cloud provider 94
5.14. ADR: Best fastAPI - AI machines message broker 96
Bibliography 97
Appendix 1: probabilistic tree of write and read loops happening during 1 loop of the main use case 136
Appendix 2: samples of the AWR Report of a DB of my Oracle contact: 137
Appendix 3: US revenue, profit, and pricing model 141

Glossary

acronym	term	definition
ADR	Architectural Decision Record	Structured written trace of a decision taken regarding the architecture of software.
ACL	Anti-corruption layer	Translation layer between a bounded context and an external service, ensuring both can communicate even when their language/terms vary [84].
	App context	The pool of data of an app.
BC	Bounded Context	A Domain-Driven Design concept designating a specific subtask within the general task that a computer program achieves.
CA	Clean Architecture	An architectural guideline commonly used in software development.
	Cohesion	Cohesion is often cited with adjectives which describe how it is achieved specifically, but it basically always means the same thing: “it is the degree to which all elements directed towards performing a single task are contained in the same component” [215].
CORS	Cross-Origin Resource Sharing	“CORS determines which external domains are allowed to access your API. By restricting this to only your trusted frontend application, you prevent other websites from making requests” [107].
	Coupling	Coupling knows specific declinations and has one unified meaning: it is the action of making multiple elements work co-dependently, or one depending on the other two elements which cannot work without one another are coupled [215].
CRSF	Cross-Site Request Forgery	“A malicious website tricks an authenticated user into unknowingly submitting a request to your application. This is a classic attack vector known as Cross-Site Request Forgery (CSRF). If your authentication relies solely on browser cookies, your app could be vulnerable to actions like changing a user's email or submitting data without their consent” [107].
CSR	Client-side rendering	the client generates the HTML content. Opposed to server-side rendering [220].

acronym	term	definition
	Customer	A customer is the legal person receiving my service, via one or multiple users, against payment.
DDD	Domain-driven design	A framework used to conceptualise the core code layer elements of a computer program [213].
DI	Dependency injection	“a programming technique that makes a class independent of its dependencies. It achieves that by decoupling the usage of an object from its creation. This helps follow SOLID’s dependency inversion and single responsibility principles” [82].
DIP	Dependency Inversion Principle	“A software design approach that prioritises the business domain at the core of development, coined by Eric Evans” [245].
JWT	JSON Web Token	“JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object” [12].
OLTP	online transaction processing	A type of data management system optimised for programs which update a lot of data and/or insert a lot of new data constantly [258, 259].
	Read loop	Read loop: e.g., “for each …, read…”, but sometimes there is no “for each”, just “read..”, usually at the beginning of the use case loop.
	rule	A rationale a program applies upon a context (not the application itself, just the rationale). For example, “if cacti, water once per month” is a rule for watering plants. That rule can be stored as code or as a relationship in a table.
	Serving/in serving mode	Designates an element of a program when it is actively in charge of delivering the service to the user as the single or co-deliverer (e.g. a db performing db tasks co-delivers a service with the fastapi app and other services). This is opposed to a dormant, or in-dormant mode element, or no-serving element, which can be part of the architecture, can do some work of its own (e.g. a db mirror instance which also does some writes) when dormant or no active work, but which does not directly provide the service to the users (in this example, the primary instance of the db would be serving, while the mirror copy

acronym	term	definition
		would be non-serving).
SSR	Server-side rendering	Content is rendered before being sent to the browser.
SSO	Single sign-on	“Single Sign-On (SSO) streamlines user authentication by allowing access to multiple applications through one set of credentials” [260].
	Target	a table’s column in which a certain user answer must be stored
	Sub-workflow	In the context of my program, this designates the process that a bounded context runs.
TLS	Transport Layer security	an encryption security used so that signals going over a network are not readable.
	Sub-workflow	the next smaller logical unit of a workflow. A sub-workflow is never just a given rule, because, as evoked in the rule definition, the latter is a rationale, while a sub-workflow is a step or multiple execution steps. Therefore, for example, a (sub-)workflow to water plants can execute watering based on watering rules such as “if cacti, water once per month”, but not the other way around.
	Usage cycle	A usage cycle is a cycle of customer activity during which that customer (via its unique or multiple users) uses all the features of the app that he/she must use to receive the service 1x in its integrality. The usage cycle contains the completion of 1 user project, users working on this user project logging in/out, and setting user rights for working/consulting this user project.
	Use case	This is a Clean Architecture concept, designating workflow-orchestration as-code rule specific to the application [103].
	Use case loop	Use cases can contain code loops, that id dub use case loop.
	User	An individual who interacts with my program’s website to receive the online service it offers. The individual who manages the customer profile and rights of users who are part of the customer is also a user.
	user project	A logical Metadata container for contexts and inference runs applied

acronym	term	definition
		upon users' interactions to provide financial case information. The completion of a user project is one part, and the main part of the usage cycle.
	User session	Authenticated (logged-in) user activity. Visitors who only view the homepage or signup page but do not access any personal space are not part of a user session.
XSS	Cross-Site Scripting	A form of attack where "an attacker injects malicious scripts into your application through user input" [107].

Introduction

I discuss here the production stage system workflow and system components of FinApp1.0, aside from the code itself (which I keep private), as well as why I chose them for this program. FinApp1.0 is an online paid multi-user financial service, applicable to worldwide financial cases. Users receive the service via the web on computers. I use Python and SQL as backend languages.

I present the architecture using a C4 template, as C4 helps best picture the roles and relationships of a software system's elements at each granularity level [20]. I dedicate one chapter to each C4 layer. I explain my architectural/code choices in Architectural Decisions Records (ADRs) [261] that I place after the related C4 diagram.

I built this project as if I were the CEO of the company commercialising that service: the constraints are costs, performance, scalability, and code reusability.

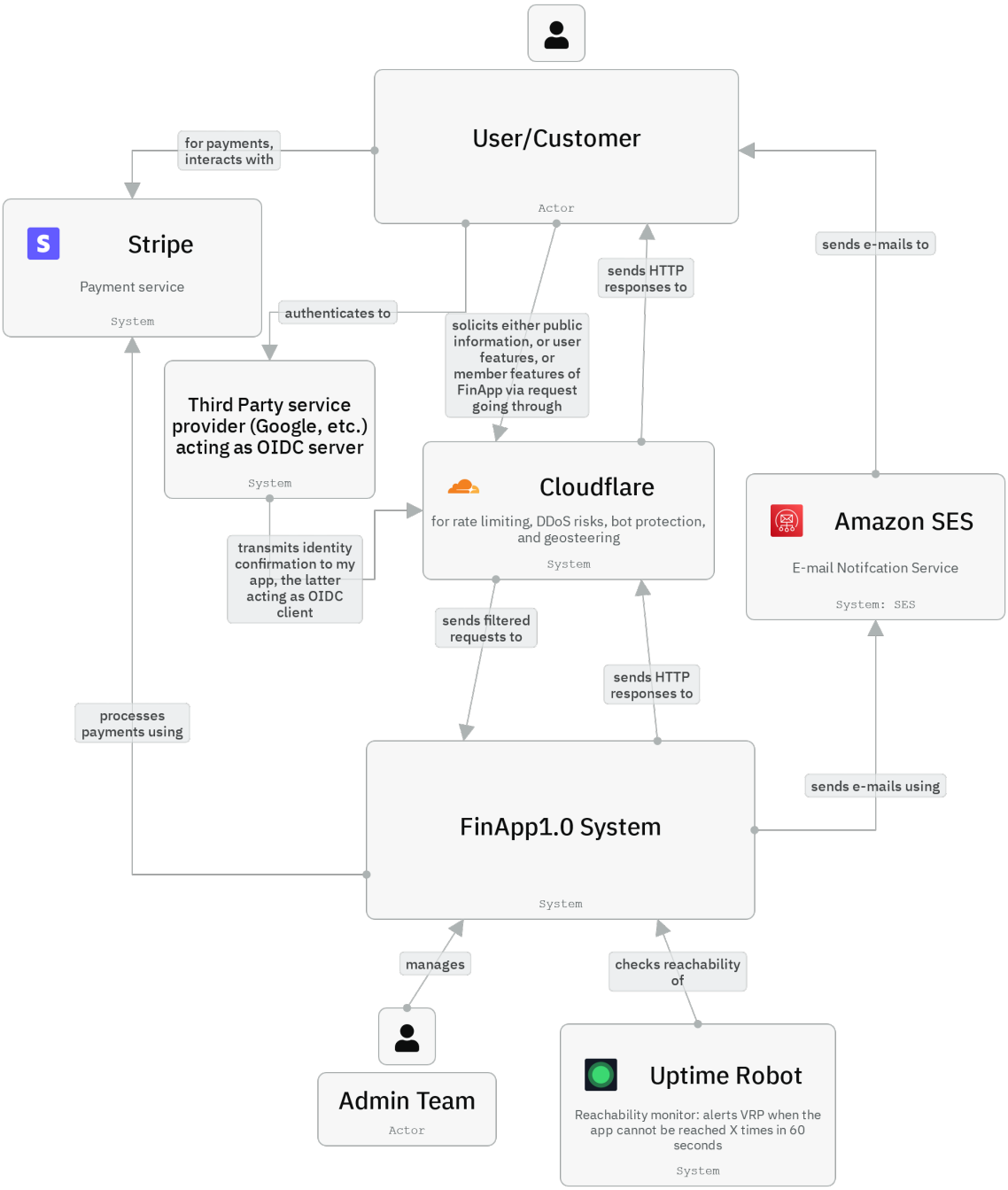
For reasons I discuss later, each country of clientele requires its own local deployment. I focus on the US deployment. This specific deployment influences choices beyond the deployment.

I am currently improving the code, and the program is not yet running. So I don't know the eventual capacity requirements of most processes. However, these requirements determine the ideal deployment solution. I still want to train in building the deployment, so where I lack an exact metric from my program's activity, I use/derive one from third-party projects and some arbitrary values.

Note that sections of this paper have not been completed in their presentation order, but iteratively until all was coherent and accounted for. And some frontend is discussed when relevant to backend choices.

1. C4 System Context

1.1. Diagram[1]



[1] Diagrams (besides the deployment diagram) are drawn with IcePanel.

1.2. ADR: monitoring

Date (dd/mm/yyyy): 20/07/2026

Status: accepted.

Context: to be alerted of performance issues and system/component failures, I need monitoring.

Options: external or self-coded monitors.

Decisions and Rationale:

Oracle has its integrated performance monitor called AWR, and internal alert systems. So I do not need a specific supplementary monitor for my Oracle DB.

To measure the CPU performance of other components, I will use tools such as UserBenchmark, GeekBench, or CineBench [162, 262].

To check website unreachability and to warn my KeepAlived for IP switches (see deployment ADR), I will use UptimeRobot, which performs regular checks at set intervals and alerts my VRRP when the app cannot be reached X times within 60 seconds [248].

1.3. ADR: Stripe

Date (dd/mm/yyyy): 25/05/26

Status: Accepted

Context: my program needs a service fee payment system.

Options: crafting them or using any online provider of such service.

Decision: I choose Stripe.

Rationale: It is a common robust solution that does not require any installation, since it is an external service.

Consequences: the app must call/redirect to this service to proceed a payment.

1.4. ADR: Email Notification Service

Date (dd/mm/yyyy): 25/08/26

Status: Accepted

Context: my program needs a tool or component to send emails to customers/users for personal info, password setting/resetting, confirmations, exceptional security warnings, company newsletters, etc.

Options: crafting them or using any online provider of such service.

Decision: I choose Amazon SES.

Rationale: it is a common secure system ideal for apps sending emails more or less sensitive, which is my case [263].

Consequences: the FinApp1.0 container sends email requests to it.

1.5. ADR: security systems and in-container/in-component security solutions

Date (dd/mm/yyyy): 20/07/2026

Status: accepted

Context: Software intended for commercial use requires security. I must determine whether I need any specific security systems, or if all security features are part of a given system/container/component.

Here are the main security concerns:

- Unauthorised requests : I want no individual to be able to benefit from the service without being authorised to explicitly. This is mostly ensured by using authentication/authorisation mehcanisms. Moreover, “modern cybersecurity standards require passwords to be stored using strong hashing algorithms. The primary reason is to limit damage if (or when) a breach happens. A hashed password database, in theory, forces an attacker to identify each password hash rather than simply reading passwords in plain text. Thus, hashing is essential for compliance and security best practices.” The solution must be strong enough against usage of rainbow tables and others [121].
- SQL injections: risks exist whether the queries are in ORM or raw SQL (although in my case only in raw SQL, since it is coded this way) [178].
- XSS attacks: Each usage cycle (which includes activities such as filling up the user registration page, completing a user project, paying, editing user info, user rights , editing user project name, extracting user project summary), comprises a per customer process. i say "per customer", because the service is engineered so that someone can supervise the work of other users completing the user projects for him. there is no blog-like or comment-like shared space on the website. De facto, a user has no input cell it can fill, that another untrusted user will be able to see (thus also unable to edit or hover over, or click on). That eliminate cross users forms of XSS, but not all XSS, since a user can access what he/she entered via the q&a history ribbon, and because change of display (in uor case, from the input cell, to the ribbon) does not neutralise the XSS risk.

To neutralise the remaining threat, what matters is that the browser interprets the user's value as data, and never as code/HTML [264].

- Rate limiting: “Rate limiting is essential for any public API. Without it, a single client can consume all your resources, affecting other users” [43].

Options:

- For password hashing: Bcrypt, scrypt, Argon2id, Argon2i, Argon2 [117].
- For blocking SQL injection in raw SQL: adding raw SQL blockers.
- For authentication and authorisation: JWTs or Session Cookies, SSL, SSO (single sign-on) with external providers like OAuth2; request-level authorisation with request middleware (while keeping authorisation logic in the core logic code), specific protocols.
- for additional data security: secrets management systems (e.g. Amazon Secrets Manager, Microsoft Key Vault, Google Secret Manager), WAFs (e.g. Cloudflare, Amazon WAF), HMAC, secure random strings
- Against XSS attacks : WAFs (e.g. Cloudflare, Amazon WAF), auto escape mechanisms [107].
- To achieve rate limiting : either SlowAPI with in-memory backend [101, 135], or using Redis [265], or Flint [109], or pre-written pure Python code algorithms [43], using the FastAPI-RateLimiter module [96]. Rate Limiters can sit either in the App, at the edge or in an API gateway, but each can implement the most optimal limiting algorithms known, namely Token Bucket and Leaky Bucket [252].

Often, tools or libraries can implement solutions to address multiple of these concerns.

Decisions:

- for authentication: OAuth2.0 authentication (with authentication made by an external service provider like Google or other, with the identity user uses to access to that third-party service) with in-app OIDC.
- In-DB secrets management.
- For Password hashing: I use Argon2id.
- To block SQL injection: I use SQL injection blockers within the raw SQL queries.
- Against XSS attacks: I use the Jinja2 auto-escape mechanism.
- For rate limiting, DDoS risks, bot protection, and geosteering: Cloudflare.

Rationale:

As we will see later, my app is developed as a FastAPI project.

Because FastAPI “does not have built-in protection against CSRF in the way a traditional web framework like Django does”, I use a JWT provider and a Cross-Origin Resource Sharing (CORS) policy to protect against CSRF [107]. For loggin in and prodiving such a system, I can either take care

of it in-app, or delegate jwt creation and identity/authorisation creation and management. Creating an authorisation policy is optimal in my case, since I want to fix exactly what a user can do depending on who that user is (do they manage user projects and who can access them, their payments, can they modify the projects, or just read them, etc.). OAuth2.0 is a standard protocol to do so and attaches authorisations to a JWT, whether my app creates that JWT or an external provider does, and OAuth's separation of the Authorisation and the API side entices users send requests with the correct permissions without the program having to check at each HTTP request the user's rights [2, 64, 73]. For security purposes, another service creating the JWT is a great option, because, in the context of OAuth2.0, it means users can log in to my app using identity credentials they created for using a third-party service (typically Google credentials or Microsoft credentials, but it can also include institutional identity systems, and OAuth is compatible with many authentication systems), without sharing login data with my app, which completely delegates identity management and security out of my app, making log in very safe and allowing me to cut on costs (since I do not run the identity management and security implementations; these are just attached to the identity of the user wit these providers like Google etc., and I just run FastAPI as an OIDC client), but my users still have an app user account which I store in my DB, so I can link activity and payments to them on the Db side [2, 64, 73].

For secrets management, my Oracle DB, whether deployed in the cloud or not, has sufficient features [45, 238].

For password hashing: Argon2id is the ideal solution, with parameters that take 250 to 500 ms on verification servers, with OWASP minimum configuration settings, and moving up from there if hardware allows. But it only works if the platform has a well-maintained Argon2 library (e.g. argon2-cffi for Python), and the system is not subject to a compliance regime that mandates something else [117]. FastAPI supports argon2-cffi [266]. My system's compliance regime will not mandate something else. Thus, I can use Argon2id.

For SQL injection prevention: because my queries are in raw SQL (see ADR: Queries code, in code context chapter), the SQL injection will be prevented via SQL text tuning [107, 108].

For XSS attacks: Jinja2's (the rendering solution I choose, as explained in the rendering ADR later) auto-escape mechanism is sufficiently addressing my specific risks evoked above, as it enforce sthat whatever the user sends, will be rendered as data and never as html. This is a kill-all-risks solution, unlike WAFs. Moreover it does not require to add more architectural components to the c4 [107, 264, 267].

For rate limiting and DDOs risks and bot protection and geosteering: I want to set the limit of requests per specific computer because many concurrent users might share an IP (typically colleagues in an office) [252]. Only those rate limiters sitting in an API gateway or in the App can provide per user

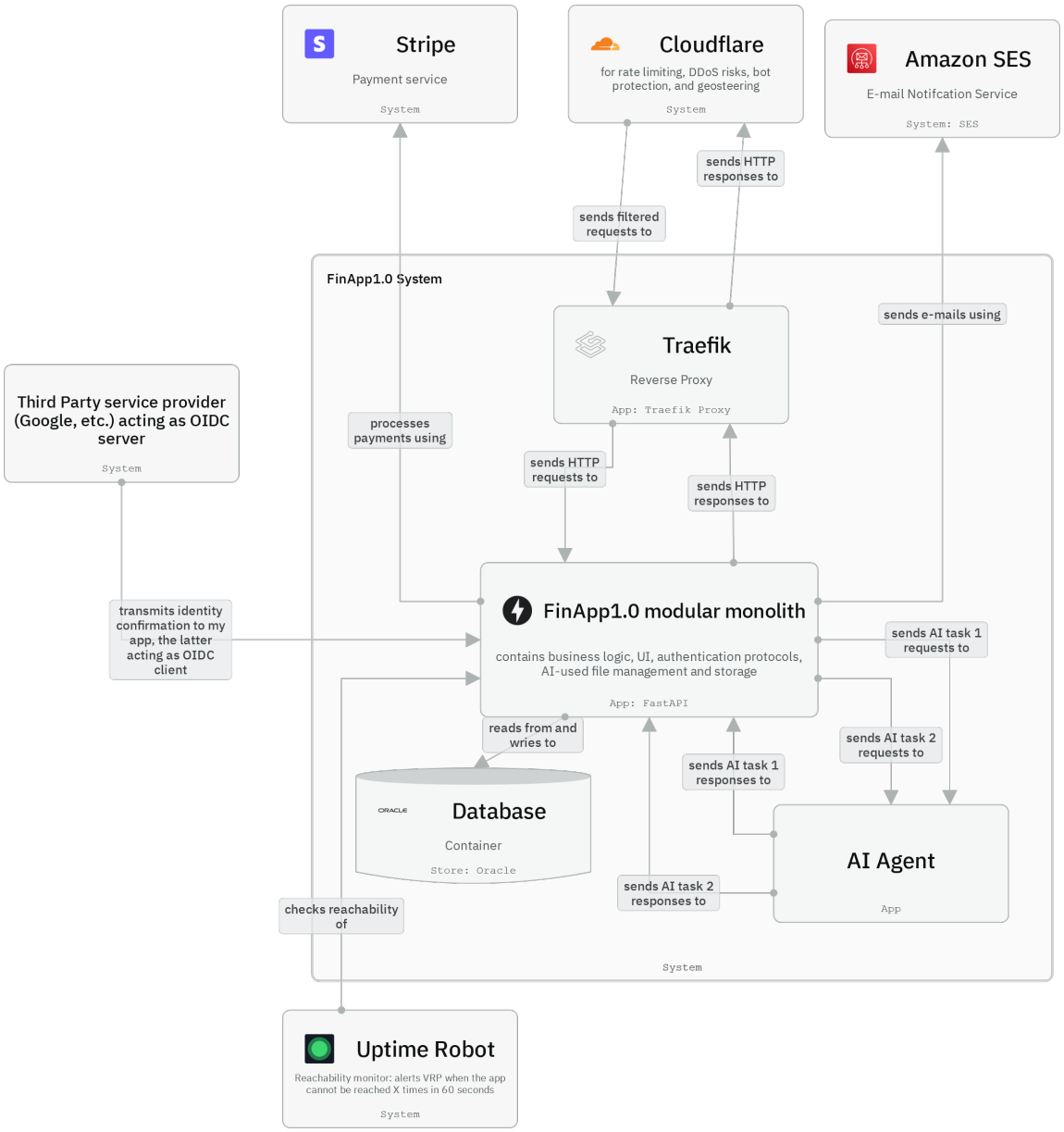
limiting [252]. SlowAPI with an in-FastAPI memory cache only provides per-uvicorn-worker rate limiting [268]. This is unfit to my deployment example, because for this deployment example I expect to have dozens/hundreds of workers, and since most HTTP requests get a response in less than a second, limiting to one or a couple requests per uvicorn workers/min still means that one individual can send thousands of requests per minute, and this would be way above the normal request traffic per user. So my program requires a centralised external cache which allows to count requests across workers, not per workers. Putting the rate limiter in an API Gateway is often recommended at production [121]. This is pretty logical because this sits between user requests and the FastAPI, ensuring that FastAPI does not get flooded and tehn realise internally that it's flooded in requests. I know that i also need a specific tool which is in charge of enforcing that user ips from a given country go only to the program instance deployed in that country. And I know I also need to protect the app form bots and DDos attacks. Cloudflare provides all 4 features exactly as I need the to be parametered [247, 249, 269–271]. Another advantage of Cloudflare is that it works with any deployment, either on-cloud or else [244]. Also note that Cloudlfare decrypts and re-rencrypts TLS encrypted signals, which is convenient because this is the encryption method common browsers used and I want users to be able to access my service via common browsers [139].

Consequences: I must configure Oracle DB secrets management, and the OIDC and OAuth2.0 part I manage myself are simply the protocol files within my FastAPI C4 component. Also, I need to find an in-app-container TLS decryptor (since Cloudflare re-encrypts the signals before sending them to fastAPI), and I must add Cloudflare in my C4 diagrams. Since Cloudflare is an online service that I purchase, there is no capacity to provision for it.

2. C4 containers context

This section only details the containers of the FinApp1.0 system, because it is the only system with multiple containers.

2.1. FinApp1.0. System's containers Diagram



2.2 FinApp1.0. System's container ADRs

2.2.1 ADR: Data and rules storage solution

Date (dd/mm/yyyy): 10/09/2025

Status: Accepted.

Context:

I must decide how my program stores user data, user project data, user project execution data and business domain rules and application-level rules (the two latter include static and dynamic rules).

Conceptualising the domain showed that the data the program uses is structured, represents relationships between domain objects (e.g., "this sanction applies to this person under these conditions"), and can be stored in tables in a highly logical and query-efficient way, with only a minority in JSON. However, the JSON data are critical, and there will be a high volume of reads and writes of JSON data. The JSON columns also store structured data, each in a very specific way I conceptualised.

The DB must process requests in the order of their arrival, ideally processing multiple requests concurrently (given the workload volume discussed in the deployment ADRs).

The program revolves around users registering, paying service fees, and answering questions, with an AI agent providing failure messages or small amounts of data per call, storing information provided by the user or the AI agent, building a response based on the read information, the user's answer, or small aggregations (at most a few hundred lines), and then returning that response (either stopping the user's workflow if an error is raised, or not stopping if no error is raised) before the user's next interaction. This type of program activity is called OLTP (online transaction processing) [258, 259].

I must retain thousands of 1kb rows worth of data per user project, for thousands of users and user projects over durations from a day to months or years (see deployment ADRs). And I also must store tens of thousands of legal, financial, and math rules I have to create myself (because not existing anywhere yet). These rules must be held for as long as the service exists, and other services later on might use them too.

Because user project-specific rows are continuously altered, the DB must ensure consistency by accounting for these changes in real time. And the data pool must be secured, as it contains valuable domain knowledge, confidential legal cases, and user information.

I can code in SQL.

So users can close and reopen their user projects at any point in the project's completion, my program must store the data required to resume the project in a secure, user-friendly and program-friendly way (data about the financial cases at hand, math data, user data, user project workflow data).

Options: Stored as code, stored in a database (with multiple possible providers, e.g. postgres, mongoDB, dynamoDB etc.), held in live memory.

Decision: Static business domain rules for the user project workflow, and user data, both stored in an ORACLE database. App/orchestration level rules, as well as the dynamic business domain rules for the user project workflow, are stored in code.

Rationale:

I must retain thousands of 1kb rows of data per user project, for thousands of users and user projects, over durations ranging from a day to months or years (see deployment ADRs). Holding all this data in memory would be more costly than storing it in a database, and it carries risks of losing in-memory data if a failure occurs [69]. And my large amount of static rules is a great fit for database storage, as the db provides a single source of truth, which eases audit, tracking, versioning, and rule search.

‘Relational databases are the best for storing structured data, i.e., data organised in a way which highlights relationships between entities and/or objects [201]. ‘Relational databases are recommended for accounting/financial applications, as well as for managing user or user project data [151, 193]. “Relational databases use SQL to store structured data” [193]. The structured nature of the data and the use of SQL make for easier querying than in NoSQL databases [144]. ‘One can scale a relational database vertically’ and horizontally (“**by partitioning the data or replicating the database among more database servers, and using caching and load balancing”** [272]) to fit international-scale business needs [259].

SQL DBs such as PostgreSQL, Oracle, SQL Server, and MySQL are the most well-known and commonly used DBs for commercial projects. I want to use one of these to learn about a DB I can find on other projects in my career, and also because they have a strong community (and some have also good customer assistance), and reliable performance and features.

“With the right extensions, tools, and configurations, open-source databases like PostgreSQL and MySQL can meet enterprise security standards”, and they each have a strong community, which helps with finding solutions to given problems [259].

Also, I want a DB which has data validation features, because this is key to avoid permanent storage errors [46]. Additional features of interest include the DELETE ON CASCADE command [114].

PostgreSQL runs faster and supports more data types and features (which is beneficial to meet unexpected needs derived from scaling or adding features to the app) and does JSON reads and writes better, and scales horizontally more easily than SQL Server, and all for free [206]. PostgreSQL also

supports concurrent table queries, unlike SQL Server, which makes it better suited to high-traffic scenarios [206]. PostgreSQL can run on most widely used OSes, while SQL Server is more closely tied to Windows [259]. But I use Linux machines (see Section 5.2.).

Oracle includes batch processing tasks integration features [153]. PostgreSQL too [40].

PostgreSQL and Oracle rather match in terms of performance, but security features might be a little more extensive on Oracle, and “for highly regulated industries, such as finance and healthcare, Oracle is the preferred database management system. It ensures compliance with industry-specific requirements, such as SOX, PCI DSS, and HIPAA" [225].

Oracle enables data distribution for horizontal scaling [136]. This is great for transitioning from a small to a larger workload as the service gets more popular or when I use the DB’s tables for new services (which is totally possible and likely given the nature of my tables).

Finally Oracle provides its own Linux OS versions, which enable better DB maintenance and DB performance while also allowing to install other elements on the same machine (which I need to do as mentioned in my deployment ADRs) [216]. Note that putting it all on the same machine might slow down the DB if the other elements happen to face spikes of RAM need or other needs beyond what is available, but in my case the max workload is predictable (see deployment ADRs) and the required capacity to support is calculated here, so same-machine cohabitation is perfectly fine.

Consequences:

- This storage decision entices that the application is stateless [69].
- conceptualising optimal tables which can store all needed rules and user data optimally, is key.
- Communication between the DB and the python app requires a communication intermediary [242]. In my deployment, I must discuss which intermediary I use and where it lives.

2.2.2. ADR: structuring as a modular monolith

Date (dd/mm/yyyy): 03/03/2026

Context: The software’s structure to meet business needs and optimise scalability.

Options: One can structure the app either as a monolith, a modular monolith, or microservices.

Decision: I structure the app as a modular monolith.

Rationale:

I might use some bounded contexts for other digital services, either as such or with minor adjustments. Whether or not I do so, I want my current service to perform great anyway. In a modular monolith, one breaks code into 1 module per function (in DDD terms, we can say one module per bounded context). Modular monoliths perform great at large scales and under high load if correctly

designed. They execute faster, notably because everything is in one place, so they have way less network-call-induced latency than with microservices. They are also 2-3x cheaper to maintain. Each module is separable from the others because each has its own interface. And thus, I can 1) troubleshoot one function without affecting the entire app, 2) paste with minor adjustments some modules in other projects' deployments or 3) easily switch to a microservices deployment (if other projects require similar functions, or if scaling shows few modules require more CPU and ram than the others signficiantly enough so that scaling the entire monolith would be cost inefficient, or if the dev team grows to 50+ members and coordination becomes complicated), because the modules correspond already to each microservice I would deploy [66, 98, 116].

An OS's process scheduler can dynamically allocate incoming processes to the free cores/ram space of one machine. But from the moment that machine reaches full capacity and one needs to scale horizontally, even if only one entry point drives most of the traffic, one must replicate the entire app when scaling, because the inter-machine dispatching technology (often a load balancer) can only spread loads per app instance, not per bounded contexts/entrypoint [98, 171]. Up to a certain point, this replication constraint is not an argument for switching from a monolith to microservices. But if the costs of deploying and maintaining replications exceed the microservices deployment costs for the same capacity, then and only then can microservices become an interesting option. But this is a rare situation, because, on average, monoliths are 2-3x cheaper to maintain, so microservices are more profitable only at a scale of solar proportions. In practice, even apps driving billions of daily interactions often favour a monolith because it costs them 50%+ less and offers performance benefits [171].

Consequences: unlike for a microservices app which shows as multiple separately deployable smaller apps at the C4 container level, a modular monolith shows as a single block at this c4 level [21].

2.2.3. ADR: AI Agent and AI models used

Date (dd/mm/yyyy): July 22 2026

Status: Accepted.

Context: my system needs software that reads documents to find target information and returns it to the app if found. It also needs a way to communicate with that software.

Options: There are two ways to benefit from such software. Building it myself, or using a third-party provider's [242]. Then I can integrate it into the given app or separate it. If I separate it, my core code can communicate with that software via either an Application Programming Interface (API) or a Model Context Protocol (MCP) [242].

Decision: I use Ai models within an AI agent.

Rationale:

The software here communicates only with the core domain code, not with users. The software has 3 jobs: understanding what the core code wants the software to look for and where, reading/interpreting document info, and returning a value in the right format and unit (s), or returning nothing.

For the first job, the core code sends a standardised request: “this table cell needs that info, from that target document”. The code sends additional information to the info fetching software: the formatting convention used (e.g., the financial record model from an accounting organisation) and the measurement unit that the core code expects (e.g. if the app needs dollars as the unit of monetary amounts, and the info is found in euros, then it must be converted into dollars). The target document is always a document attached by the user to the user project, in an imported format. This means the document is not accessed via a URL on the internet, but always from an app-specific pool. The document might contain information written in text format or embedded in images.

Each user project contains multiple documents, each providing different information. I can easily set up application code that allows users to upload documents and ensures they are categorised. With that, I need a ‘document-info research’ pairing policy that enforces the rule "for this use case/use case subpart, call the document info fetching software so it parses that/these documents".

For the second job, the software faces the issue that many terms can be used to designate a given thing, and must also determine in what form it must find the info (derived from some info or in explicit formulation). Due to the “*if software does not find the info in explicit terms, ask user*” fallback, the software does not need to know very rare terminologies, nor to derive a given value (if the target info is a variable's value) from other variable values found in the documents. But the software must still be aware of common formulations in each possible language (since it must analyse documents in any country’s language, regardless of the app’s customer's country). Secondly, document writers can use different measurement units for a given numerical value. The software must find the match, regardless of the measurement unit, whether the measurement is written in Arabic numerals or in words, and whether the unit is written in full or in abbreviated form.

Every request to the software asks it to extract and return specific information from the designated document(s). There are two options to do that:

- Either the software must read the document(s) and extract very specific information at every request.
- Or, upon loading the document, it extracts everything it might need, then for every request for a given info, the software parses everything it pre-extracted to find and send the relevant info.

In my case, I know that in all subworkflows of the core code app needing info fetching, the app repeatedly (from 10 to hundreds of times) needs info from the designated documents. An argument for

the first option is that, in the main subworkflow of my service (80-90% of FinApp1.0's activity), the program does not yet know what information it will have to fetch in the next step of that subworkflow. But there isn't really a way for that workflow to "wisely eliminate" the items to be extracted from documents as the subworkflow advances. Using a rule such as "when the core code app is asked to find info more than X times, fetch everything you might need out of the documents" is not ideal, because it would just multiply the number of full-document parses, which would make the software's activity more expensive and increase latencies more than a single full document parse. On the other side, I know just how many times it can ask to fetch info from documents if extractions are separate, and I know that the core code app usually needs 60% or more of a document. So I opt for option 2: a one-time extraction (herein referred to as "Task 1"), and then reads from the extracts (each read is an instance of what I dub "Task 2").

I want Task 1 to extract the text, word-for-word, and store it as structured JSON. And I want each Task 2 to perform a semantic search. The word-for-word extraction in Task 1 avoids unnecessary semantic inference for info the core code app might not request. The JSON format makes Task 2 cheaper than an info search on the original PDF or PNG, because structured JSON sections allow Task 2 to analyse only a small section of the JSON file(s). But word-for-word extraction would amout in enormous JSON which would necessitate creating an enormous context. If not too expensive, I want to not retain full sentences, spacing, and other formatting from the document, simply storing info as {infoname:infovalue}. Now, this could still cause information loss if the information the core code is searching for is uncertain. But in our case, if the info isn't explicitly found in the documents, the core code app still asks the user for it. So I could keep my 'task 1-then flush docs' strategy. Last, I propose that task 1 merges all documents of one user project as a single JSON file, so that task 2 looks within a single file, thus avoiding JSON file selection logic, and simply having JSON file's section selection logic.

Many input documents or parts of them will be images. I need an LLM-VLM to perform task 1 because VLMs can extract text from images, whereas the other text-extraction technology I found, namely legacy OCR, cannot [25, 243, 273]. To do task 2, an LLM is the only technology I found capable of retrieving info from a JSON file, no matter how that info might be formulated [118].

Consequences: I must determine which models to use.

2.2.4. ADR: using a RESTful API

Date (dd/mm/yyyy): 10/04/2026

Status: Accepted.

Context: A domain code which communicates to HTTP users needs a clear interface to communicate so [242].

Options: any type of web Application programming interface (API) [274].

Decision: I choose a RESTful API.

Rationale:

I can use a RESTful API with stateless apps, and my app is stateless. Also, that type of API separates client and server side clearly, enticing each can be modified independently of the other at any time [232, 275, 276]. And a RESTful API pairs greatly with the usual medium-to-large-production-stage-quality cybersecurity solutions, namely, JSON Web Tokens (JWTs), Session Cookies, a TLS (transport Layer Security) or a SSL, SSO (single sign-on) with external providers like OAuth2, request-level authorisation with request middleware (while keeping authorisation logic in the core logic code), a secure random string, HMAC, secrets management systems (e.g. Amazon Secrets Manager, Microsoft Key Vault, Google Secret Manager), WAFs (e.g. Cloudflare, Amazon WAF), can ensure the security of sensitive (notably for authentication) data transiting in the stateless app, and API keys enforced within an API gateway .(e.g. Amazon API gateway, Google Apigee, Cloudflare API Shield) [107, 190, 277–279].

Consequences: I must determine how I deploy it.

2.2.5. ADR: using FastAPI:

Date (dd/mm/yyyy): 10/05/2026

Status: Accepted.

Context: I want to use a web deployment framework, as it is software that structures and simplifies the deployment and code writing of a web app [157].

Options: any Python web framework working for RESTful APIs, like Django, Flask, etc.

Decision: I use FastAPI as my web dev framework.

Rationale:

FastAPI can work with all the elements I yet chose: the RESTful API for web-app communication, the Python-OracleDB driver, and vLLM.

Also, FastAPI allows me to code an async RESTful web API benefiting from the ASGI server Uvicorn, which I chose (see ADR concurrency in the C4 level of the paper), but also to benefit from all the routing, serialisation, Pydantic data validation, OpenAPI spec automatic writing, and dependency injection “Depends” features coming along with the Starlette-based FastAPI implementation (features that I need, and that Uvicorn alone does not provide) [211, 280]. This ensures that I already benefit from these features on top of uvicorn, in their most performant form,

without having to manage or write them. Overall, this web app framework proved faster than Python competitors like Flask and Django [192].

Also, FastAPI allows me to use both asyncio (which I selected: see ADR concurrency in the C4 level of the paper) and synchronous handling in Python within user-project sequentiality, to use the async/io libraries as well as threads/multiprocesses for CPU-bound tasks, and to use type hints to facilitate code writing [167, 210, 281, 282].

FastAPI is faster than other Python frameworks for my program, whether or not those frameworks use asyncio. Also, FastAPI automatically generates the OpenAPI spec from the code, which is helpful because the spec is a key file for understanding the app's client-server communication. The Depends and Pydantic Models one can use in FastAPI simplify dependency injection and validation, respectively. And the documentation is exhaustive and detailed [72, 166].

Consequences: I must determine how the other key architectural elements, supposed to interact with the core code, interact with FastAPI, and how FastAPI receives requests.

2.2.6. ADR: where does Jinja2 run?

Date (dd/mm/yyyy): 02/05/2026

Status: Accepted.

Context: In the components context chapter, I explained I chose Jinja2 rendering. I need to determine where it lives (what runs it).

Options: In the FastAPI app, or in a separate frontend app [21, 119].

Decision: The frontend lives in FastAPI.

Rationale: This frontend can physically live within the FastAPI container in terms of deployment, thus including in the C4 deployment diagram [283, 284]. The nature of the frontend (almost only static HTML) entices this frontend should also be put within the FastAPI c4 container rather than as a separate c4 container [21].

Consequences: at C2 level, one will not see the frontend as a distinct container, since it is a part of the FastAPI container, and each component must own all the responsibilities it includes [21].

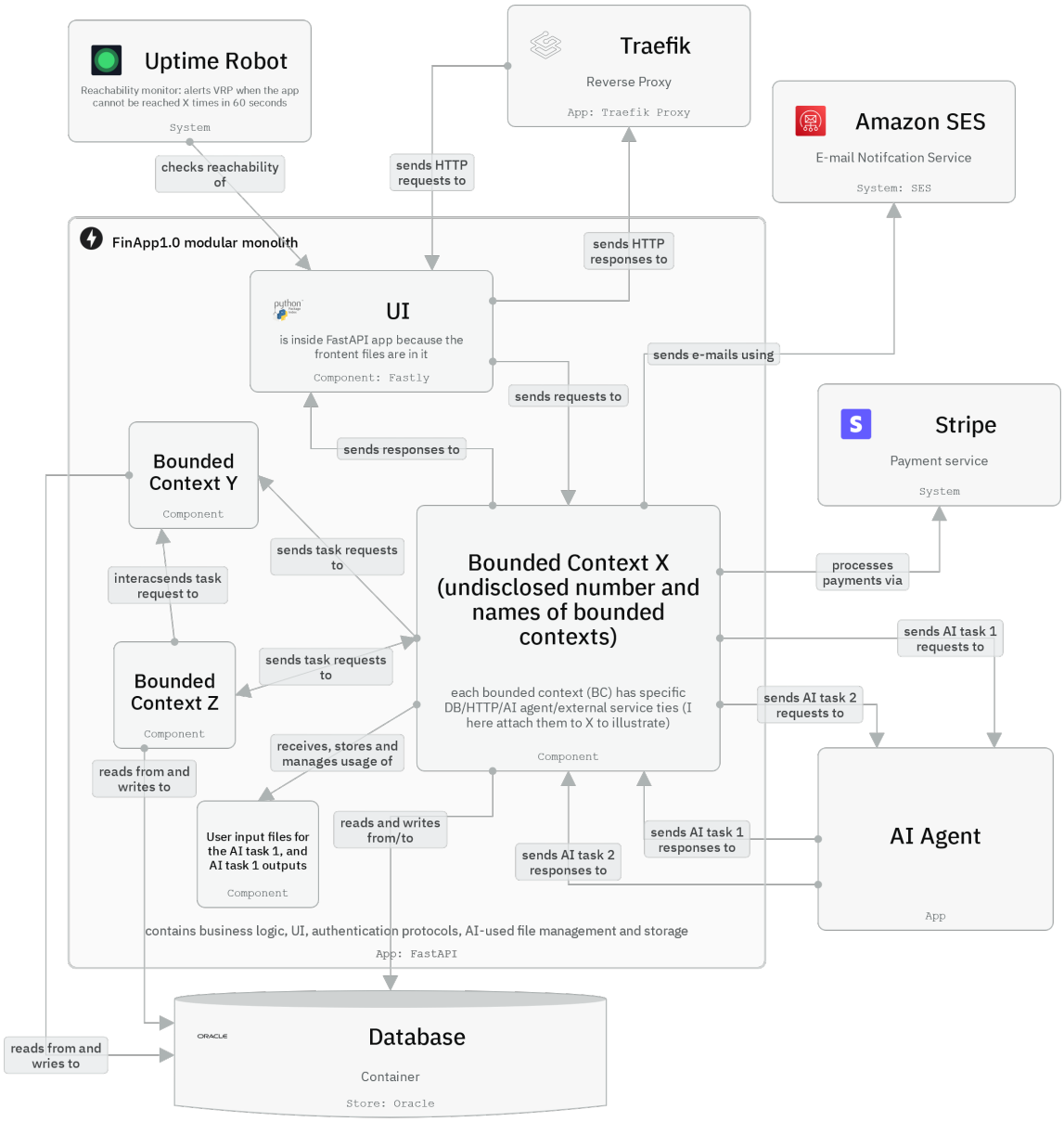
3. C4 components context

Here I only detail the fastAPI app component, because the inner details of the other elements besides the DB are not particularly relevant, and because the inner details of the DB are either domain-sensitive (which I do not disclose in this paper) or deployment/standard Oracle structure-specific (e.g. the SGA cache space etc.) which I discuss in the deployment section.

3.1. FinApp1.0 System - FastAPI container

FastAPI gathers a lot of files (dockerfile, toml file, uv.lock, test.DB) which fall outside the spectrum of C4, because they are not what c4 represents at this level, i.e., fragments of runtime units [22].

3.1.1. Diagram



3.1.2. ADR: DDD bounded contexts as some of the app container's components

Date (dd/mm/yyyy): 10/06/2026.

Status: Accepted.

Context: I must determine whether my FastAPI app container has multiple c4 components and, if so, which ones.

Decision: I divide the FastAPI app container into components, which are its bounded contexts (whose names I will keep to myself because I do not share domain details in this paper).

Rationale:

The software developer's main design goal is to obtain the lowest coupling and highest cohesion [215]. "Coupling refers to the degree of interdependence between software modules. […] Cohesion refers to the degree to which elements within a module work together to fulfil a single, well-defined purpose" [215]. Obtaining the lowest coupling and highest cohesion means that a change in one part of the code must entail the fewest changes to other parts of the code, that each part of the code performs a single task, and that there is no redundant code. Achieving this concretely happens in code, but conceptually starts at a higher level, when determining the general tasks of the FastAPI app, because each must perform a clear role.

DDD is a framework for conceptualising the invariant core code of a software [30]. It particularly fits complex finance software projects [213, 221, 285]. In DDD, bounded contexts are the general tasks that a software runs to deliver a service [30]. Bounded contexts correspond to the modules of a modular monolith (in C4 language, they correspond to the components of an app container) [202]. My app is a modular monolith that provides a complex financial service, and I want to facilitate the migration of modules to other projects/microservice setups.

3.1.3. ADR: database caching

Date(dd/mm/yyyy): 03/06/2026

Context: I want to minimise I/O and DB-related latencies beyond DB caching solutions [286].

Options: adding a remote cache or a local cache [286].

Decision: I will use only the Oracle DB buffer caching only.

Rationale:

A modular monolith suits a local DB cache better. Local caching improves performance, is easy to set up, and does not introduce additional network calls, infrastructure, or costs, unlike a remote cache. It is also more secure than a remote cache. In-memory caching in the app can include adding an LRU map or configuring SQLite as a local cache layer on the host. The local cache, however, struggles to deal with dynamic data environments [35].

But how does the app handle dynamic data, and will it prevent the use of the local cache? And which data is subject to caching?

Create/edit the name/deletes of a user project, or import/edit/delete of supporting documents, do not follow one another; they occur in isolation within any usage cycle. Moreover, they are highly sensitive

operations: undertaking any of these entails significant cost considerations for the service buyer and/or the progress of their user projects. Thus, permissions data must be the most up-to-date data. And for it is not consulted over long periods/often, these data do not need to be in cache. What needs to be in cache is the data that a given user project workflow-specific use case needs (data about what I call the sw instances, sw inputs, and sw outputs, as well as currently consulted in-DB legal-financial-math rules), because this information is consulted over long periods, constantly. For these, cache invalidation is easy. Supporting documents are not in cache. Changing the supporting documents kills info-fetching progress, which directly affects cached info. The user project is pending for the duration the program kills this progress.

For rather simple queries, whether calling on new data or on the same data already optimally fetched by indexing and in-DB caching, the in-app cache is overkill, because checking the cache, deserialising data, and managing cache keys would actually be slower than just hitting the database directly [100]. But many of my queries are grouped and analysing the same data pool: a use case analyses a certain data pool, and depending on the result, it usually inserts or updates a row/rows in a table (other than form that data pool) and provides to these/this row(s) values that can be found in the data pool: this makes close caching very attractive.

However, the data pulled is always sensitive: it contains either user info, billing info, or financial/legal info specific to a business discussed in the user project. This is too sensitive to keep in an in-app cache [100].

3.1.4. ADR: Rendering and frontend solution

Date (dd/mm/yyyy): 01/05/2026

Status: accepted.

Context: I must determine my rendering solution, then pick an HTTP request traffic system and security features compatible with its security and functionality.

During the questionnaire part of the usage cycle, questions are answered one after another. Answer confirmation redirects the user to the next question/question group.

For user creation and payment, the display follows the usual standard for these subservices: a single form-like window with all required information, and a single confirmation.

During the info fetching part of the user project's completion, which takes the form of a questionnaire from the user's point of view,[2] the user sees a "q&a answer history ribbon", in which each info fetched during the questionnaire, either by the AI agent or user-provided, is displayed in chronological order

2 It is not strictly fetching answers from the user's answers, in so that the ai agent contributes to finding info as well, hence me aying "a questionnaire from the user's pov".

of fetching. This ribbon enables the user to find and edit information already provided about the financial case. I do not require an expand/collapse behaviour for this ribbon. The list of info fetched from questionnaires tends to grow long, and reloading the entire ribbon every time might be time- and energy-intensive. Therefore, the rendering of the ribbon's contents would benefit from a dynamic or dynamic-like solution that loads only the new info fetched since the last HTTP request after each HTTP response, while all previous info stays on display.

Decision:

I use Pure SSR with Jinja2 to render the entire app, except for the financial case info ribbon and the question placeholders (which will rely on a Jinja partial template, e.g. _ribbon_item.html and HTMX).

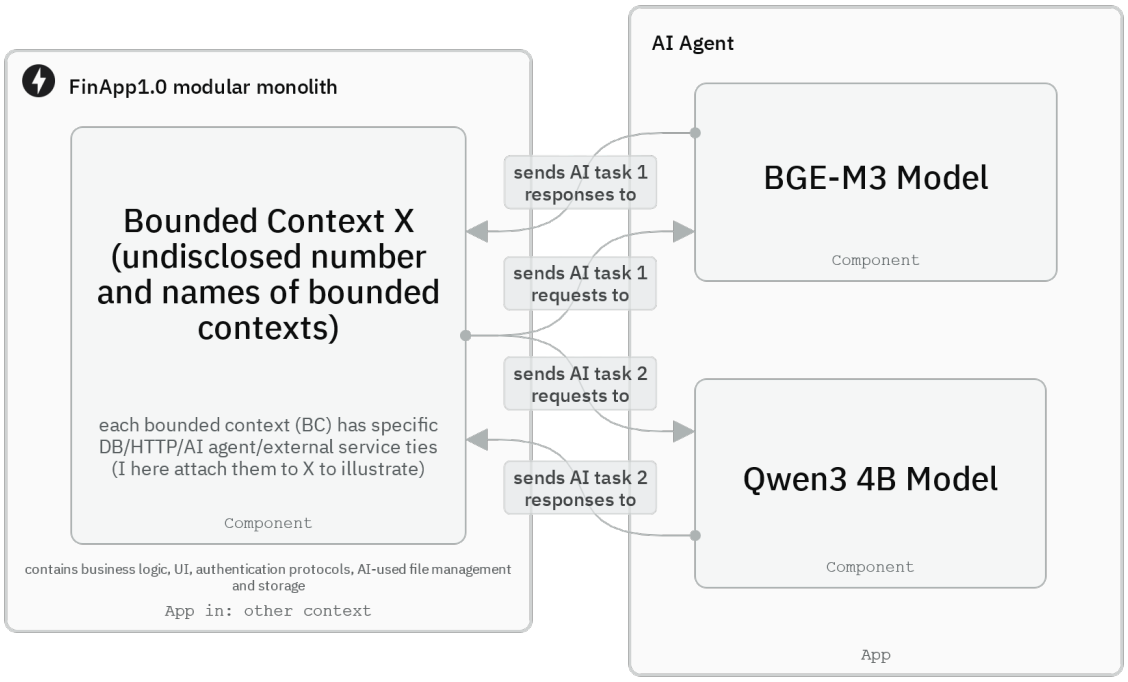
Rationale:

- Pure SSR with Jinja2 works great for static rendering [287].
- Jinja protects from XSS attacks via its auto-escape system [107].
- The advantage of HTMX, is that it allows, without using .js frontend, for questionnaire frontend to be a questionnaire page, with the only thing that changes on this page is the question and the history ribbon, which are added to the frontend page as HTMX fragments. Notably, for the Q&A answer-history ribbon, hx-swap="beforeend" appends the newly obtained information while preserving previously displayed entries [288, 289].

Consequences: I must decide where this frontend resides/what runs it.

3.2. AI Agent

3.1.1. Diagram



3.2.2. ADR: AI models choice

Date (dd/mm/yyyy): 22//07/2026

Status: Accepted.

Context: In the previous chapter, I determined I need AI models to perform two tasks. But which LLM and LLM-VLM model to choose?

Decision: I use BAAI's BGE-M3, and Qwen's Qwen3 4B.

AI models are made of one or more parameters (more or less like the model's tools) and a context (the pool of info that the tools can work on at once, we could say it is its 'reality') [47, 250]. RAG is a form of answer-building logic that relies on data external to the model's context [59]. RAG is useful when one cannot fit all the data into the context window required for correct inference, either because the model's context window is too small or due to other obstacles to inserting that data into the context. This is what the main LLMs on the market (form Anthropic or OPENAI for example) use to find info form online sources in real time. There is an argument to use RAG for task 2, because the models' parameters and context might not contain all needed info about the expected potential formulations of a target info (storing all this represents millions of words or more because of the large

variety of info the model might search in my case due to the nature of the service). However, despite the eventual overall big size (see US deployment ADRs), there is no context size limits and I can load all needed info for a reasonable price by sending task 2 requests with the needed formulation info (from a table queried pre-AI request which indicates common formulations of an info in a given jurisdiction) and an order for task 2 to load only a small JSON section based on date index and JSON file sections names (e.g. "{'financial records';'13 August 2023'}") that the core code specifies. So I do not need RAG. Thus, and knowing RAG needs multiple models [104], I do not need multiple models for each task. But I need semantic search, which is a form of logic that allows the AI model to understand the context of the search, rather than just searching for exact words (because, despite my formulations DB, some formulations might be unknown in the context) [242]. Thus, I need an embedding model for task 2, because it is the type of model that performs semantic search [290].

I eliminate coding my ai agent myself, since many models exist online or will exist later for document extraction for my types of documents, and bearing all this effort myself would be unnecessarily bothersome since, as we will see a bit later, some existing and performant models can be used for free on a self-deployment. Therefore, I focus on third-party AI models that I can deploy co-locally to my core code/app instance.

For task 1, the best llm for that seems to be Qwen3 or above, and there seems to be no specific performance gains (rather the opposite) above 4 billion parameters [104]. Qwen3 has a context of 256k tokens, up to 1M tokens per request, which roughly equals to the equivalent of loading thousands of pages, which largely exceeds what each request will need (at most to load a page or few pages worth of JSON section at 5,000-8,000 tokens a page, plus the request itself with the DB formulations info). OS I choose Qwen3 4B for task 1.

The most performant and open-source embedding model is BGE [104, 240]. Its maximum sequence length (how many tokens it can handle at once) and other aspects can vary, and choosing the right config depends on request load and size [240]. As discussed in the deployment section, I choose BAII/bge-M3 for task 2.

Consequences: I must determine the adequate deployment solution for these models.

4. C4 codes context

I am not showing the code here, but I would like to discuss some general choices and observations about it.

4.1. ADR: concurrency strategy

Date (dd/mm/yyyy): 10/05/2026 (with an additional source added in August, to mention worker anomaly and re-creation features of Uvicorn)

Status: Accepted.

Context:

Together with the supporting hardware, how the app handles the flow of requests (herein referred to as the “processing strategy”) determines the app's performance and scalability [163, 203]. My program revolves around two ‘scales of execution’: the “user” workflow (the stream of requests-execution-responses for a specific user), which is sequential (because the prior HTTP response determines the possible next HTTP responses), and the “across-users” workflow (the stream of requests-execution-responses for the entire app) whose processes can run simultaneously (because the prior HTTP response of one user does not determine the possible next HTTP responses of other users). I need to find the processing strategy that best implements these two workflows.

Options:

Parallelism is the practice of executing tasks simultaneously, and concurrency is a complementary approach that improves the scheduling and execution of tasks on a given processor thread [91].

Background tasks, Celery, and Asyncio are 3 ways to achieve effective concurrency of I/O-bound tasks in Python (concurrency programming) while ensuring each user’s workflow remains sequential [39, 93, 222].

Using multiple workers or threads on a single CPU core, or multiple CPU cores, achieves parallelism [91]. But note that Python threading alone, without this config defined multi-threads or the multiple cores, does not produce real parallelism, only pseudo parallelism, due to the GIL, and the real Python solution is creating multiple CPython processes (for example, in FastAPI, a process is the unit of execution a Uvicorn/Gunicorn worker executes) instead of trying to create Python threads within a CPython process [99].

Decisions: I use asyncio to ensure in serving moder-workflow’s sequentiality and concurrency across user workflows. I enforce true parallelism for speeding the across-users workflow, by using multiple Uvicorn workers.

Rationale:

My program revolves mostly around single-table lightly joined queries on a small-to-moderate row count per call (one user_project or one sw_instance's worth of instances/inputs/outputs). Such tasks' latency is mostly bound to the performance of the i/o [36]. That I/O latency dominance means these tasks are 'I/O-bound' [121]. Only concurrency reduces the I/O latency [67, 291, 292].

Asyncio handles high concurrency better, provides much better cancellation and lighter task switching, and comes with fewer concerns about race conditions, but it is only compatible with a few Python web frameworks, and external libraries only support it in their asyncio-compliant versions, if any [39, 93, 218, 222]. But 'asyncio is simply not beneficial in terms of execution time when the code must be blocked by legacy libraries or because the DB driver used is synchronous' [293, 294]. Asyncio alone is best for i/o-bound tasks [113]. If asyncio can be used, I will exclude Background tasks, for they do not provide a retry system in the event of task/system failure/reset [111].

Asyncio is compatible with parallelism [231]. Secondly, the DB driver I need supports asynchronous programming, notably with asyncio [295]. Also, Oracle DBs are compatible with Celery, requiring no more effort than other databases used with Celery, in case I ever change my asynchronous handling from an asyncio-based system to a Celery-based one [296]. Lastly, I have asyncio versions of Python libraries my program needs (this only regards i/0-bound tasks; CPU-bound tasks are discussed after):

- Oracle DB-Python communications (storing and retrieving info, whether the user answers the questionnaire, or new user projects, or user info): such communications require setting up a connection between Python and the Oracle DB, using either Python-OracleDB in thin mode or in thick mode (the latter requiring using Oracle Instant Client libraries) [297]. I found the "aioodbc" library for connecting to my Oracle DB from Python, using Oracle Instant Client ODBC as a substitute for the usual "pyodbc" library [230, 236, 297–299]. When not using Oracle Instant Client (i.e., when using the python-OracleDB driver in thin mode instead of thick mode), it is also possible to use asyncio [295].
- Client/service-like communications: Since my program is currently at the prototyping stage and only receives inbound HTTP requests, and queries only via drivers rather than making outbound HTTP calls to external services or internal microservices, I do not need to give the program the ability to act as an HTTP client for outbound calls. Therefore, I don't yet need an asyncio library that provides that ability (e.g. aioHTTP), but I know that such libraries exist if I may add such a feature to my program at some point [300].
- Authorisation and permissions checks (e.g. user wants to create a new user project, system must check if user is authorised to do so):
- unique/sample of data assignment (e.g. when the program executes a calculation, either an algo or a formula, 1st it must assign the right values to each variable in formulas, or the right data

sample to an algorithm) (noteworthy if that falls under data retrieval, please ignore this bullet point)

- mailing: aiosmtplib is a library implementation of smtplib, i.e. the Python library for sending emails, compatible with asyncio [23, 50].

Therefore, asyncio is compatible with my sequentiality requirements and the 95-99% I/O bound of all my programs' tasks, with the other tools of the program, and is a good fit if I ever add parallelism.

Now, let's talk about whether to use parallelism.

Beyond the Pythonic world, program threads (different from the cpu threads that is often pre-conigured on cores, as you can see many double threaded cores online; these hardware threads show to the ap as separate CPUs) handle multiple operations part of the same process and exploit the same data, while processes are better for handling a heavy task/a task that is not part of a group of tasks that exploit the same data. However, in Python, due to the GIL, threads do not provide true parallelism and can introduce latency because only one thread executes at a time. The only true parallelism in Python is to create several workers. In my case, it is valuable because my app receives so many requests at once (see deployment chapter) that it would execute them much faster via multiple workers [91, 99].

Uvicorn is the fastest WSGI/ASGI solution for an I/O-bound Python app (deduction I base on TechEmpower querying benchmarks). It also has anomaly detection and worker recreation features, and it is fine that it does not have workflow re-dispatching like Celery does, because the user can just re-send the request, it is not expected to really make the user experience bad, since it should not happen often (and if it does I troubleshoot) [90].

Consequences: I must find a config that allows me to use asyncio and uvicorn.

4.2. ADR: Clean Architecture and DDD division and annotation of the code

Date (dd/mm/yyyy): 08/10/2025 (and some source additions later in March 2026)

Status: Accepted.

Context: I want my code to be organised as logically and clearly as possible, so that the general role of each part of the code is clear just by looking at a single or a few easy-to-find lines/words (whether folder names or others).

Options: using code tags, specific code folder names, writing Python comments, using specific colourring/highlighting on the code text.

Decision: I present and organise the contents of each DDD bounded context using Clean Architecture (CA) as the main sections and DDD for the subsections within the CA domain layer, as well as in-object-name and in-function-name DDD and CA tags.

Rationale:

Code text colouring and highlighting is always used, but is not a sufficient solution, because my number of bounded contexts already exceeds or nears the number of easily distinguishable colours, and there are other dimensions to account for (is this code element a parameter, an object name, what is its workflow role beyond its Python syntax nature, etc.). Also, I want to organise and annotate the code so that any developer can easily understand it and use their own colour-coding system, since multiple developers/software engineers will work on it.

In-element-name tags and folder names are straightforward and reliable. When you look in any folder, you can see from the tags in the name of any element which file it is stored in, its workflow role, and its bounded context of belonging.

Like DDD, CA best helps developers achieve low coupling and high cohesion (which all programs aim for) and modularity (which I need for separate reusable and project-specific code) in large, complex financial programs (which perfectly matches FinApp1.0's description) [68, 174]. CA complements DDD, as it is a pattern for conceptualising and organising the code and other software elements that the core invariant code needs for app deployment, in a way that pursues low coupling and high cohesion (which I define later) by following the SOLID software design Principles [56, 174, 233]. CA also supports isolation testing of the different code objects [141]. Moreover, DDD and CA naturally fit together since the "core code layer" dubbed *ENTREPRISE BUSINESS RULES* or *Entities* in CA corresponds to DDD's Tactical design elements (entities, value objects, aggregates, domain events, DDD repositories (aka concrete repo's interfaces), services, factories), i.e. the elements created in DDD after establishing the main task and subtasks (domain and subdomains) of the software and the general code departments addressing these tasks (bounded contexts) [30, 71]. Then, CA's Application Business Rules and interface adapters and frameworks & drivers help us use these ENTREPRISE BUSINESS RULES in a deployment context. Because my FastAPI application c4 container contains components which are its bounded contexts, it only makes sense that the code of each bounded context gathers all the CA code which is specific to that given bounded context, and to logically section it in the 3 code CA layers, namely the domain layer code, the Application Business Rules layer code, and the interface adapters layer code, with the domain layer contain elements organised according to DDD [202].

The bounded contexts thus contain the following elements (this is simply a summary of DDD and CA components), and I assign each element a bounded-context tag, a ca/ddd tag, and a more specific in-name role indication. E.g., if something is an output of a bounded context, it will contain the bounded

context tag and the word ‘output’. E.g., if something is an app business rule, it will contain the bounded context tag, a UseCase tag, and an indication of the more specific role of that use case. And I also add a FinApp1.0 tag when the element is specific to this app and not reusable in other services:

- ENTREPRISE BUSINESS RULES (also dubbed the Entities, but that is not to be confused with the Entities concept from DDD). This is the CA category I subdivide using DDD’s Tactical design elements, namely [207]:
    - Models: Models are aggregates (which are made of multiple DDD entities and DDD value objects, but with at least one DDD entity, which acts as root) or entities and value objects not part of any aggregates, and they must contain rich behaviour [88]. An entity is an object whose attributes are mutable and whose ID is immutable (e.g. a process instance object with a changing ‘status’ attribute, a ‘student’ object with an email address which might change), therefore defined by its ID, while a value object is an object whose attributes are immutable and whose ID is immutable, therefore defined by its attributes (e.g., an address is a value object, because it is unique not because of its ID, but because there are no similar address in the world) [170]. The rules they must follow are as follows. First, ‘Entity relations should not hold a reference to another entity class; it can only keep the ID of another entity. This entails that an aggregate can only depend on another one by citing the other aggregate root’s ID as a foreign key’ [88]. Secondly, ‘An aggregate describes a cluster or group of entities and value objects that can be treated as a cohesive unit’ [81]. Thirdly, ‘in order to maintain separation of aggregates and keep clear boundaries between them, it is a good practice in a DDD domain model to disallow direct navigation between aggregates’ [81]. “Aggregates should remain unaware of each other; they are independent units. They serve as a facade to hide the internal workings and specific business logic from anything outside their boundaries. However, it's acceptable for any object inside the aggregate to hold a reference, specifically the unique ID, to the root of another aggregate” [53].
    - the domain events: ‘Domain events capture moments, representing something that has occurred within the domain in the past, and which influences the rest of the system’ [131].
    - the DDD repositories (aka concrete repos’ interfaces): DDD repositories (aka concrete repos’ interfaces) abstract data persistence. “Ddd repositories (aka concrete repo’s interfaces) separate data access logic from the domain model. They provide a consistent interface for querying and storing domain objects. DDD repositories (aka concrete repo interfaces) hide the specifics of how data is retrieved or stored. They encapsulate the translation between domain objects and underlying data storage

methods, such as databases or external services" [221]. This means that they contain the abstract representation of concrete methods living in the interface adapters layer, because the interface adapters layer is outer to the layer calling the methods (Application Business Rules layer), enticing that calling the methods in the Application Business Rules layer requires an abstraction at the use case layer level or the entities layer to ensure we respect the dependency inversion principle (and in practice, putting such abstraction in the domain layer is preferred here, because I have domain services which might one day be written and which will call these interfaces, so they cannot sit a in an outer layer than the domain) [48].

  - the domain services: "In DDD, a Domain Service is a stateless, operation-centric class that encapsulates domain logic or operations that don't naturally fit within the boundaries of an Entity or a Value Object. These services are all about performing actions or providing domain-specific functionality that doesn't belong to a single entity" [4].
  - the factories :objects that have the single responsibility of creating other objects [19].

- APPLICATION BUSINESS RULES, which use ENTREPRISE BUSINESS RULES elements and tools to act on specific observations (e.g., missing sw_inputs or sw_outputs, or seeing a sw_instance with a certain completion status, or seeing a certain value object being raised with ; the related actions are to get the inputs or the outputs, move to the next logical sw, update sw_instance completion status, etc.).
- INTERFACE ADAPTERS (also called Anti-Corruption Layer in DDD [84]): these are "adapters that convert data from the format most convenient for the Application Business Rules and entities, to the format most convenient for some external agency such as the Database or the Web" [103]. Interface adapters include:
  - Controllers: Controllers receive an HTTP request, hand it to a use case, and (in some systems) return a response. a controller contains: 1) Endpoint definitions: the route, the HTTP method, the path parameters and query parameters; 2) Request parsing and validation: in FastAPI this is largely done by Pydantic input models declared on the endpoint signature, but the controller is where you decide which model to use; 3)Dependency injection declarations: in FastAPI, Depends() calls that inject ddd repositories (aka concrete repo's interfaces), services, or handlers into the endpoint function; 4) Calling the appropriate use case or handler: one line, passing the parsed input; 5) Mapping the result to a response model: taking what the use case returned and passing it through the Pydantic response model [124].
  - Presenters: "the Presenters repackage the OutputData into viewable form as the ViewModel", which, in the context of FastAPI, is either a Python dataclass or a Pydantic Model [124].

- Gateways: the bridge/translator between the core code app and an external service it talks to [65].

Note that Clean Architecture also contains a fourth layer called FRAMEWORKS & DRIVERS, but these are the elements such as FastAPI and python-OracleDB driver, which are made of code, but which are not like the previous CA layers' elements, in so that they are not part of bounded contexts, so they are not following a per-bounded-context tag system unlike elements in the previous layers, although they live in distinguishable files too [58].

Consequences: now that we know what elements classification I want, I must determine what pythonic form they will take (see 'ADR: function or object?') and how they must work together so as to obtain high cohesion and low coupling (see 'ADR: design patterns and solid principles').

4.3. ADR: function or object?

Date (dd/mm/yyyy): 10/10/2025

Status: Accepted.

Context: Python code consists of functions and/or objects [199]. I must decide which to use and when, with the goal of achieving the lowest coupling and highest cohesion [215].

Decisions and rationale:

I turn these Clean Architecture/DDD elements into these Python code elements for these reasons:

CA / DDD Element	Python construct	Key reason
Ddd Entity	class	has identity and mutable state [149, 152].
Ddd Value Object	@dataclass(frozen=True) or Pydantic model	Element is defined by attributes [23, part 1.1]. Element is immutable (immutability is not common to all dataclasses but can be set) [301].
Ddd Aggregate Root	class (entity + enforcement)	Since, in DDD, the aggregate root controls its members, but these members can change over time: this is best represented as a class, because it makes the aggregate root an entity, and its members the internal mutable attributes [175].
Domain Service	class (stateless, injected deps)	a domain service has no state of its own, but it depends on repos or other services, which act as interfaces between domain and DB in order to ensure dependency injection [132]: - UserRegistrationService depends on UserRepo to check user

CA / DDD Element	Python construct	Key reason
		info - user projectService depends on OracleUser projectRepo to load and save the User project aggregate , and of EventBus to save events - UniqueUserSessionPolicy depends on OracleUserSessionRepo to check whether an active session already exists for that user - UniqueUser projectRuntimePolicy depends on OracleUser projectRepo to check project status (if already opened somewhere) Such dependencies are cleanly written by making the Domain service a class with the given other service/repo as a constructor injection [132]. Classes also give you named, testable units, which is also the right practice, since test-driven and test-driven is good practice [144].
Application Service / Use Case	class with a single public method,	Making the element a class makes it independently injectable, testable, and named. If no dependencies, a function works. Classes are for state-focused workflows, while functions suit better the action-focused code elements [152]. Moreover, “, testing is simpler when you use functions. Functions that don't modify the global state can be tested in isolation, making it easier to verify the correctness of each component. This leads to more reliable, testable code without the overhead of additional abstractions” [152]. However, my Application Business Rules all comprise multiple named, injectable, mockable dependencies that benefit from being swapped in tests (e.g., testing the branch where get_cmsssw_input_id() returns nothing, vs. the branch where it returns something). This sort of use case requires controlling these dependencies, and since I will use dependency injection to ensure clean repo method calls via an interface, this justifies sticking to writing Application Business Rules as classes [152].
DDD repos	Protocol class or ABC class, or	First, we know this must be an abstraction a method’s signature attaches to. In Python, this is expressed as a protocol or ABC class

CA / DDD Element	Python construct	Key reason
	Duck typing	or with Duck typing because Python's type system has no other construct to attach method signatures to [23, part 1.2, 61].
Repo Implementation (Gateway)	class implementing Protocol (Its methods are the same signatures as the Protocol)	The element has state [152].
Domain Event	@dataclass(frozen=True) subclassing base or pydantic model	has No behaviour, no mutation [23, part 1.1].
DomainEvent base class	class	metadata enforcement + serialization hook= its __init_subclass__ or to_dict() logic may need to be non-trivial [142].
Controller / Route	FastAPI APIRouter class + async def functions	Controllers control incoming HTTP requests and redirect them to the right orchestration code. Doing so, it depends on that recipient. In FastAPI, controller logic can live in a function, and dependencies stay attached to an APIRouter object. The advantage is that this single object can 'keep' the dependencies of multiple routers, so it is slightly configuration-time saving, without losing any other advantages. But the controller in my case must be async, because it waits on the I/O taking place with the use case querying the db or the AI agent (except for some minor HTTP requests), and that asynchronicity lives at a function, so that all satisfies async needs and routing configuration ease [7, 302, 303].
Presenter / Response Model	Pydantic BaseModel	That pydantic model acts as final type and shared information validator before the information sends to the UI, and it also translates the response info from its fastapi-friendly form to its UI-friendly form (serialisation in other words) [304].
Factory	class with static method, or plain	My factories are: PaymentGatewayFactory.

CA / DDD Element	Python construct	Key reason
	def	PaymentGatewayFactory relies on a Stripe Gateway. A Stripe gateway needs at minimum a Stripe API key and probably a webhook secret, which will come from either a settings object, environment variables, or a secrets manager [305]. This means the factory depends on other elements to configure its own role. StripeGateway will need an HTTP client to call Stripe's API. In a properly testable system, that HTTP client is injected rather than instantiated inside the gateway, so the factory receives it and passes it through. That is an injected dependency. Considering these dependencies of the factory, the latter shall be a class.

4.4. ADR: design patterns and SOLID Principles:

Date (dd/mm/yyyy): 02/02/2026 (with some added sources in August).

Status: Accepted.

Context:

Computer science authors (most famously the ‘Gang of Four’) wrote ‘design patterns’, i.e., ‘ready-to-use’ solutions to solve a problem with cleanly designed object-oriented or functional code [49, 173, 189]. These patterns notably align with other well-known, more recent object-oriented programming principles, such as the SOLID principles [215]. In the current context, these can help me make my CA and DDD elements work together in the cleanest way (max cohesion, minimum coupling).

Decisions and rationale:

First, in FinApp1.0 code, although I did not really plan it at first, there is no inheritance, only composition. And I want to be careful to keep it that way as I fine-tune the code. Why? Inheritance is defining objects (e.g. a Dog object) as a type of a more abstract object (e.g., an Animal object). Composition is defining objects (e.g. Dog object) by its attributes (e.g. Leg object). Both methods introduce coupling, but inheritance does so at compile time, whereas composition does so at runtime. The coupling composition introduces is looser than the coupling inheritance introduces, because an objects the composed object uses (e.g. Legs) is less susceptible to change, and if I want to change the composed object (Dog) I can do it without affecting the ‘components’ (Legs), while a superclass

(animal) is more likely to change and changing it has effects on all its subclasses (dog, cat, snake, etc) [257].

Also, Application Business Rules must call a repo method via an interface. This dependency injection avoids that the app's business-rules layer directly depends on an outer layer's element, and instead depends on an abstraction in the core layer of the CA (the interface), thereby enforcing the dependency inversion principle [141].

Also, a use case must call another use case via an anti-corruption layer (ACL), and neither is a child class of the other, nor is either a child class of a common parent (whether a common 3rd class or separate parents). The use case call to another use call acts as a way of ending/pausing the workflow in one context and starting/resuming workflow in another context (in my case, this means that usecase of bounded context 1 either created bounded context 2's elements by calling bounded context 2 repo method interfaces create_instance create_input, or looked for a certain instance id of bounded context 2, and then simply calls usecase of bounded context 2 to execute for instance_id = bounded context 2's newly created instance's id). This is a way to ensure delegation/orchestration of work between objects while respecting the principle that modules of a modular monolith (here, my bounded contexts) should not directly depend on each other to ensure low coupling.

I avoid global variables because they increase coupling [128, 195].

Also, the HTTP request-response system is based on, either one of the subtypes of the MVC (Model-View-Controller) pattern [27, 52]. It works as follows (and this is what I adopt):

- "1. The user interacts with the view.
- 2. The view creates a request (object) which is passed to the controller.
- 3. The controller converts the request into a request model and passes it to the use case interactor through its input port (the controller returns nothing to the presenter it only triggers a use case, and so even if the use case does nothing more than calling a repo method interface, the flow must go through the use case first).
- 4. The use case interactor processes the request model and creates a response model which is passed through the output port to the presenter.
- 5. The presenter converts the response model to view model which is then passed to the view.
- 6. The user sees the result of his interaction in the view" [124].

In Clean Architecture, the request-response system comprises a Use Case Output Port, a Use Case Input Port, a Use Case Interactor, a Response Model, a Request Model, found under the Application Business Rules layer, as well as the Controller and the Presenter, the latter two found under the interface adapters layer [103, 124]. The output port is 'the API that the app sends output through, for this particular use case' [27]. In Python, the input port is usually the execute() method signature [306]. The output port is a presenter. The interactor is the use case itself [124]. The request model is the

object the controller constructs from the HTTP request and passes into execute(). It is a plain domain-friendly object, not a FastAPI request object. The Response model is the plain object the use case returns – not a FastAPI response, just a domain result. That response model is often separated from the request model. However, in a FastAPI project, having the controller both call the use case and format the HTTP response (without a separate presenter class) is a legitimate and widely used simplification [125]. So I use that simplification.

In terms of where the repo method abstraction should live, I found that if the repo method exists to enforce or evaluate a domain rule, the abstraction must be in ENTREPRISE BUSINESS RULES, because it must be in the same layer or an inner layer than the element consulted (here the domain rule in the corest layer), and if the repo method exists to fetch data so the use case can decide what to do, the abstraction can be APPLICATION BUSINESS RULES [134, 176]. But note that it also makes sense to group all abstractions in the core layer, since it is easier to find them with my by-layer folder structure in my FastAPI app. So I group all repo method abstractions in the enterprise business rules layer.

Note the abstraction topic can be confusing. One might wonder, why in clean architecture, saying “a L2 component (app use case) who wants to use a L3 (interface adapters) repo method, must be first dependent on a abstraction in L1 (entreprise business rules)” is clean, because it makes it look like, eventually this L1 element must know of the L3 repo method to use that L3 repo method. But in fact, the L1 element does not know of the L3 element: only the L3 element is aware of contracts happening within the inner layers. And having this L3 element does not make the code more capacity-consuming than having no L3, because the L3 element is not like an active watcher in that sense: it only runs at runtime of the other elements.

Lastly, “every API call costs network latency (50–200ms minimum), connection overhead, TLS handshake (first request), serialization/deserialization, database roundtrip, and authentication/authorization check” [115]. So ideally, I need to reduce the number of API calls. One way to do so is to collapse more processes into fewer endpoints: for example, if 4 endpoints are used to access a given page instead of 6, the user has to make 4 clicks to access the page instead of 6, reducing API call costs [115]. Additionally, fewer API endpoints mean less maintenance, while the backend produces the same results [115, 161]. So I keep the number of endpoints ot a minimum.

4.5. ADR: Use uv as dependency manager

Date (dd/mm/yyyy): 11/05/2026

Status: Accepted

Context: The project needs a Python dependency manager to ensure quality of migrations and a potential access by collaborators in the future [226].

Options: pip + pip-tools, Poetry, Pipenv, PDM, uv, conda.

Decision: Use uv (Astral).

Rationale:

- Significantly faster resolution and installation than pip/Poetry,
- uv.lock provides deterministic, cross-platform lockfile,
- Native virtual environment management,
- Compatible with pyproject.toml (PEP 517/518 standard),
- simplest and most advanced solution [208, 226].
- I use Docker containers (see deployment ADRs) but using uv causes no issue, and uv lives within the FastAPI container [307].

Consequences: UV does not appear as a separate C4 element, and uv.lock must be committed to version control [308].

4.6. ADR: DB queries code

Date (dd/mm/yyyy): 11/05//2026

Status: Accepted.

Context: My program must contain queries to communicate with the database.

Options: FastAPI allows querying tables in several ways: raw SQL (in code or in stored procedures), an ORM (ORM models can be combined with Pydantic models, but not necessarily), or a SQL query builder like the non-ORM SQLAlchemy [196].

Decision: I write my queries as raw SQL. Also, the SQL text for each query must include placeholders, and the Python code that sends the query must specify the value the placeholder takes.

Rationale:

The software relies on static queries [237]. My queries are mostly process-state management queries (ensuring workflow execution), with a minority of entity-state management queries (managing the lifecycle of objects such as users, user projects, etc.) [196, 205]. My queries are almost all very simple insert, update, and delete commands. The statements contain lookups, joins, filtering, JSON construction, and existence tests. Often, these require additional relational queries because the relevant data is not in memory. The Insert and Update statements all share the same “methodology”: besides

the primary key of the new/to update row, they find in other tables the values to put in that new/to update row.

I use Pydantic models, but they differ from ORM object mappings, so I don't need ORM object mappings to use Pydantic [296]. It is possible, however, to combine data validation, serialisation, and object mapping for an ORM with SQLModel [205, 309]. But eventually, with an SQLModel ORM-validation combination, or with the ORM alone, I might have fewer query lines, but I would lose the speed of raw SQL or stored procedures [95]. I need to prioritise speed in the querying system because my program is so I/O-bound, and because I/O is not only the main inter-requests bottleneck but also occurs in large quantities even within a single HTTP request-response. Securitisation against SQL injection is a concern, but since my queries are rather straightforward, SQL injection security will be as well [196].

My queries do not contain looping mechanisms or other elements that can only exist in the SQL dialect of my DB; all can exist as ANSI SQL [191]. Keeping raw SQL in-code queries avoids db-app navigation just to troubleshoot my code: all is in one place in the app.

FastAPI allows me to use an ORM and in-code raw SQL queries and stored procedures in the same project [310]. This means I can always switch to an ORM system later on if needed.

I use an Oracle DB. Oracle keeps parsed SQL statements and their execution plans in memory in the library cache, which is part of the shared pool (SGA). "When an SQL clause comes into the Oracle database, Oracle first checks whether it has been executed or not by looking at the library cache. If the query has been execute before, Oracle uses the previous execution plan without parsing this SQL. This process is called soft parse. If the query has not been executed before, Oracle will parse the executed SQL and save it to the Shared SQL Area in the library cache. This process is called a hard parse" [42]. If a query comes back in the DB with variable values concatenated inside it, it always forces a hard parse, whereas if the variables come separately, the DB only needs a soft parse. The soft parse is less cache- and hardware-intensive. Thus, to ensure shorter, lighter query execution, I must ensure that Oracle receives each request with the passed values sent separately rather than enclosed within it. To pass the value separately (this is called passing the value as a bind variable)**,** the SQL text of each query must contain placeholders, and the Python code sending that query must specify the value for each placeholder. This ensures the Python-DB driver understands these are two different elements, and sends them as such [42, 311].

For my program, a clean way to use raw SQL queries is with the '*cursor.execute'* method, which ensures that the query is executed with specific parameter values only once per call, which is what the use cases require (unlike cursor.executemany or others) [312].

Consequences: the only intermediary between FastAPI and the Oracle DB is the python-DB driver.

5. Deployment: example of the US deployment.

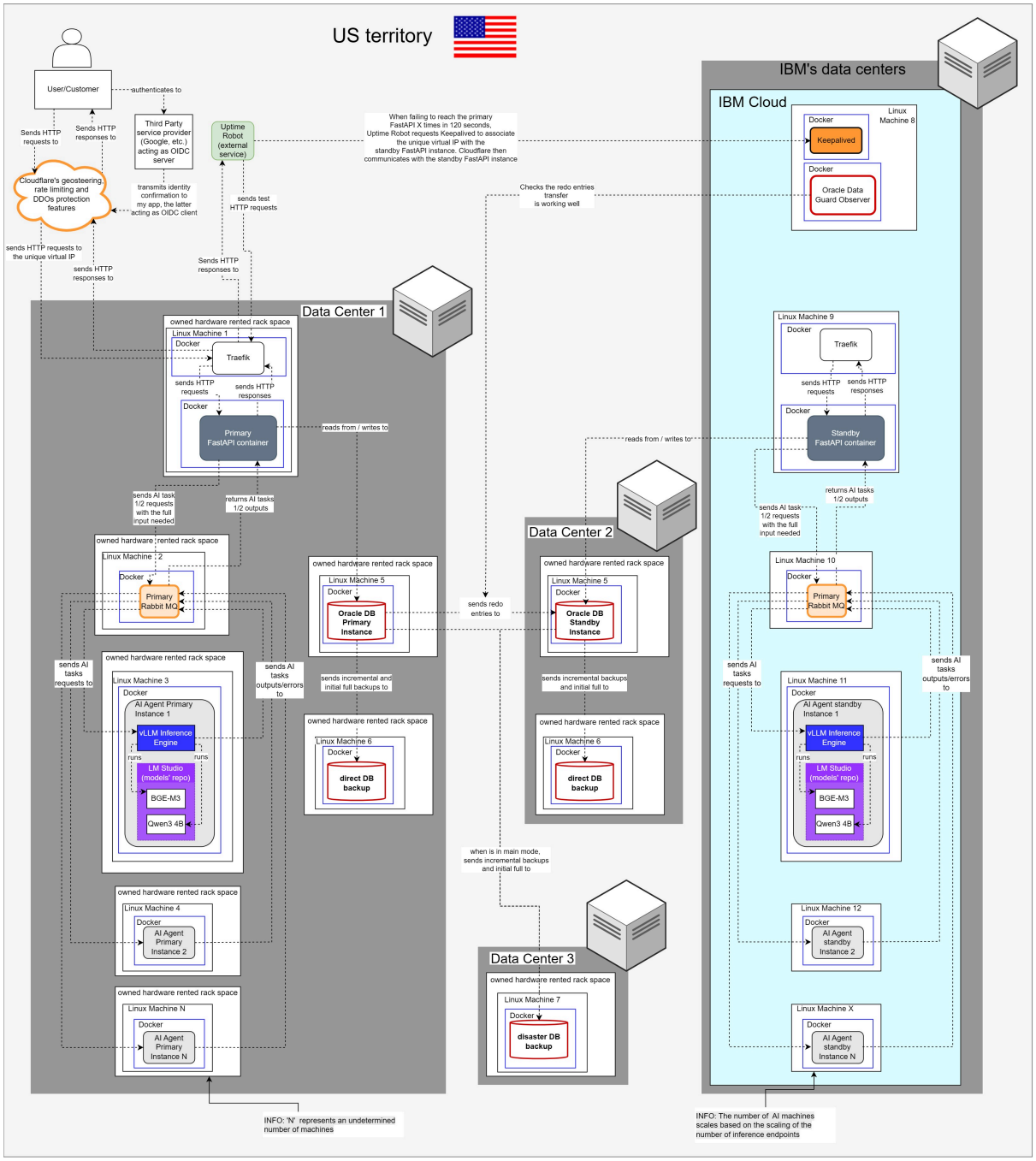
I am currently improving the code, meaning I don’t know the eventual latencies. Instead of running tests on this code I’m not satisfied with, I use latencies from comparable third-party projects (which contain equivalent tasks but cleaner code) and the program's maximum acceptable latencies to craft this illustrative deployment.

Remember, the domain can be used by other digital services or not. So I could craft a deployment as if the program was a modular monolith, or as microservices. And for an as-microservices deployment, there could be many configuration options because there are almost-infinite potential microservice scenarios (depending on what needs to become a microservice, which companies exploit the domain, whether services other than FinApp1.0 face the same regulatory constraints, etc.). I choose to focus solely on the as-monolith deployment, with FinApp1.0’s domain used by no other service, because I’m curious whether the service would be profitable when its domain provides no digital services other than FinApp1.0 (I will discuss this aspect in the “consequences” subsection of this ADR).

We will discuss later that the deployment must be per-country. And since costs and profitability of a deployment solution can vary by country, and in my scenario the data is mostly from the US anyways, and the location of that data impacts the relevance of the scenario, instead of imagining an optimised deployment for each target country, I choose to focus on the US deployment.

5.1. Diagram

Please zoom in to see the details.



5.2. ADR: OS

Date (yy/mm/yyyy): 01/04/2026.

Status: Accepted.

Context: My program must run on an OS (operating system). I need to choose which OS the program will run on.

Options: there are various providers, but the main ones are Mac OS, Windows, and Linux. And each has its different OS versions.

Decision: I choose a Linux OS.

Linux performs better on given hardware than the alternatives, it is also very stable, secure, and open-source, unlike the alternatives, plus it exists also in specific versions for Oracle DBs [92, 150].

The DB's block size must be equal or a multiple of the OS's block size [197]. Since my DB block size is 8kb, that gives 8kb, 4kb, 2kb, 1kb as OS block size options (keep in mind 8kb, 4kb, etc., are just shorter names for the exact size, which is always a multiple of 2).

5.3. ADR: using Oracle-python DB driver:

Date (dd/mm/yyyy): 10/11/2025

Status: Accepted.

Context: A domain code and a DB communicating need a clear interface to do so [242].

Options: The only one I found for Python app-Oracle DB communication is the Python-OracleDB driver [16]. It comes in two modes: Thin and Thick. Thick mode simply refers to using specific Oracle Client Python libraries that provide additional features [313].

Decision: I choose Thick Mode.

Rationale: It seems that Thick Mode adapts better to changing deployments (due to load-balancing or network-encryption features), which might come in handy for me. So I choose Thick Mode, but note that this requires installing Oracle Instant Client [313].

5.4. ADR: AI-FastAPI communications and merger?

Date (dd/mm/yyyy): 22 July 2026

Status: Accepted.

Context: AI agent sitting inside or outside of FastAPI, and using an AI API or an MCP for communication of the AI models with the FastAPI instance if sitting outside FastAPI.

Decisions: AI agent sitting outside of FastAPI, using an AI API to communicate with the FastAPI instance.

Rationale:

I do not embed my agent in-code. The core code will communicate with it via an AI API or MCP because that allows changing AI models more easily than if the model was embedded in the core code. The reason for why changing models is attractive in my case is that the type of documents that users will upload might change in formats and layout overtime (because of industry standards' evolution), and AI models perform well only on the data they have been trained on: a model trained with documents from before 2026 might not perform as well in 2028 if the documents' formats and layouts change by then, and so I would ideally change my model or its version by then [104]. Access via an AI API/MCP is also beneficial because it likely means that the model can be loaded on its own server or in its own Docker container, and thus remain loaded even if the server/container of the core code goes down, allowing both a faster restart of the work when the app restarts, and vice-versa allows the FastAPI server/container to keep running when the ai server/container goes down.

An API enables communication between a program's components by exposing endpoints: an endpoint specifies who can talk to whom and about what. An MCP is specifically used for software with AI models. The MCP enables more informed communication between an AI model and another component than an API does by exposing the component's full context to the model [254].

In my case, my program contains the AI models. And the requests contain all the info the AI model needs: the AI model does not need to understand how the entire core code works to answer the request appropriately. So I don't need an MCP and stick to an AI API.

Consequences: Since FastAPI does not host the models (see ADR AI-FastAPI communications), something else must host them, and this is usually simplified by using a tool that acts as a model repository. And it also needs its own engine to run. And I need a GUI preferably.

5.5. ADR: AI models' management and storage

Date (dd/mm/yyyy): 22 July 2026

Status: Accepted.

Context:

Each AI model response is always routed to the core app for orchestration. So the two models do not communicate with each other. But since FastAPI does not host the models (see ADR AI-FastAPI communications), something else must host them, and this is usually simplified by using a tool that acts as a model repository. And it also needs its own engine to run. And I need a GUI preferably.

Options: there are different engines, GUIs and repositories on the market. The list is long.

Decisions: I use the vLLM engine and the LM Studio repo/GUI.

Rationale:

The server engines for running large-scale AI models are vLLM, SG Lang, and TensorRT-LLM. The best seems to be vLLM because, while it is roughly on par with the others in terms of performance, it deploys more easily and is compatible with more hardware brands [76]. And it also serves its own API, even if the deployment is local and offline [314].

That server engine vLLM and these 2 models are available together only in the Hugging Face AI repo (the other repos have one or the other) [235, 240]. Hugging Face seems to offer an AI API, but it's not free, whereas using its repo and other AI APIs and GUIs locally, like LM Studio, is free [250, 315]. LM Studio has an intuitive interface, and I can import the two models from Hugging Face into LM Studio [250].

There are some all-features providers that concentrate on GUI, API, model selection, and storage. They seem to support my task 1, task 2 system, because they let me define different request types that the core code app can call [316]. Now the issue with those is that they are not free, even when deployed on my own hardware rather than called via the web.

Consequences: vLLM is something to run myself. So I not only need to include them in my C4 model but also plan capacity for them, which might influence which overall deployment hosting (cloud, on-prem, collocated) is cheapest. If I have multiple instances of the AI agent per program instance, vLLM is the element sending AI responses to |FastAPI and receiving AI requests from fastAPI, because vLLM acts as inference endpoint.

5.6. ADR: by-FastAPI/in-FastAPI TLS decryptor:

Date (dd/mm/yyyy): 19/08/2026

Status: Accepted.

Context: Nowadays, browser HTTP request encryption is TLS-based, and “TLS termination is the process of decrypting encrypted HTTPS traffic (encrypted by the browser [317]) at a specific point in the network, usually a load balancer or reverse proxy, before passing the request to backend servers” [61]. Since my program receives TLS-encrypted HTTP requests, I need to decrypt them with a reverse proxy [72, 318]. Cloudflare decrypts TLS-based requests, but for security purposes it must re-encrypt them before sending them to my FastAPI instance. Thus, I need an additional TLS terminator at the location of my FastAPI instance.

Options: Nginx or Traefik [72, 318].

Decision: I pick Traefik.

Rationale: Traefik supports enough HTTP requests for my deployment (it supports 45,000/s; I have a max of 3,000/s in my deployment), and is easier to set up than Nginx [6, 86]. I found no source stating that Traefik does not support my service-side Jinja2 rendering solution; thus, it seems compatible [6].

Consequences: it must show as a distinct component of the app's c4 container, since it sits in front of the fastAPI app [318], but it can live inside the same docker/Kubernetes container in the deployment.

5.7. ADR: personal data encryption

Date (dd/mm/yyyy): 20/08/2026

Status: accepted.

Context: since users share information that must remain unreadable for potential attackers, I need some form of data encryption.

Options: using a DB feature or an external encryption add-on.

Decision: I rely on the Oracle features.

Rationale: Oracle security features are one of the two main reasons I chose this DB. This includes encrypting data and controlling its readability per DB access/accessor.

Consequences: no need to have an additional component for this task.

5.8. ADR: user input format validation solution

Date (dd/mm/yyyy): 20/08/2026

Status: accepted.

Context: To protect user accounts against attacks, I want to enforce a minimum password strength requirement that includes password strength analysis and blocking the saving of passwords that are not strong enough. Also, when users answer questionnaires or fill out name bars, etc., it usually must adhere to a given format (e.g., if the program asks for a number, the user must provide a number, not words).

Options: front-end solution, or in-app solution, or DB solution.

Decision: front-end HTML validation for input constraining, and DB validation for final verification.

Rationale:

First, for password control, this solution provides the same eventual result as the others. But avoids a trip to the app. To enforce password constraints without a strength meter (the bars that show how strong a password is), an HTML frontend page suffices [319, 320].

For the answer bars format control, the challenge is that the next question is unpredictable from a program point of view. But since the program relies on sub-workflows and that in each sub-workflow, given types or one given type of answers are expected (e.g. sub-workflow X expects only numeric answers), there is still some general logical structure on which to base user input format control (e.g. when the code knows it is in sub-workflow X, it uses the numeric answer frontend template, which contains a "only numeric answer" HTML check upon whatever user writes in the input frontend cell).

The unpredictability is absent from the other user-input parts of the usage cycle (namely, payment, user registration, customer info entry, user project creation, or name edit). So these can use the frontend solution just like for the password creation/modification validation. I suspect Stripe to intervene in payment info filing checks [321].

The FastAPI container must eventually run its own check as well, because client-side validation in the mentioned solution can be bypassed [320]. This FastAPI-level validation, for all fields, including the password creation field, is done with Pydantic because it can validate data efficiently, including user input types, min-max lengths, and specific admin-defined character patterns [156, 210, 322–325].

Consequences: no need to have an additional c4 component for user input validation.

5.9. ADR: Endpoint caching?

Date (dd/mm/yyyy): 20/08/2026

Status: accepted

Context: I try to optimise HTTP request-response latency. I came across endpoint caching as an optimisation method. Endpoint caching means caching the HTTP response [256]. It makes the response faster because the response is already prepared and waiting to be delivered, without needing to run FastAPI logic. I wonder if I should use it.

Options: endpoint caching or not.

Decision: no endpoint caching.

Rationale:

Endpoint caching means placing an endpoint and its response in a cache in front of FastAPI, so FastAPI does not need to generate the response when an HTTP request hits that endpoint. Most of my requests (90%+) provide user- or user-project-specific HTTP responses, and these responses always differ from one another as well within each user project (a user will not obtain the same HTTP response twice in a user project unless they cancel a step and redo it exactly identically, which is rare). The only times an HTTP response is similar are when the program must display the website homepage, pages like legal terms, the login page, and other such universal non-user-specific pages.

These few universal HTTP responses (they are few in type, but of course, they are called often) could gain from a caching system, because caching them gains time on delivering them, but this would increase latency for all other requests because they lose a bit of time going through the Redis or equivalent HTTP caching system every time, despite 90% of my requests not needing so.

Consequences: I do not need a system in front of FastAPI to cache HTTP responses (e.g., Redis).

5.10. ADR: system failure management

Date (dd/mm/yyyy): 20/08/2026

Status: accepted

Context: When the system running the program shuts down or significantly slows down, I want users to continue enjoying the service smoothly despite the failure. That requires a specific backup plan.

Options: cold start backup, multiple active instances.

Decision: I opt for a cold start backup, and an Oracle Data Guard persistence option plus a disaster backup storage.

Rationale:

When having multiple running instances of the program, the total user activity is the same as with one active instance, but the costs of running multiple active instances are higher because of the baseline costs of running additional cloud VMs, or additional racks, and because it duplicates the instance-wide activities (health checkups, cybersecurity features) [326].

For the DB specifically, a backup implies two instances of the DB. Also, for failures which do not require a switch to the standby DB, I want that the primary and standby DB instances both have a main backup locally on a specific disk, and I want a disaster backup in another physical location (in another data centre), so the total number of backups is not 4; it is 3. The standby DB, when it is running as the in serving mode DB (meaning the primary instance stopped working and now the standby takes over), needs the same capacity as the primary DB instance, since it must do exactly what the primary DB was doing when it was not down. And since I want high availability, with at most some minutes of downtime, when the standby DB is in standby mode, it must have the required capacity, so that if it becomes the in serving mode DB, it will be in a state which is the same as the state of the previously in serving mode DB when it went down. We will see later that due to the very large size of the DB and write volume, writing everything from the backup to the standby would be too time-consuming, and probably as lengthy as, or longer than, the time it would take to troubleshoot the primary server and relaunch it. Therefore, the solution is to keep a standby copy which does the

writes over time. In Oracle, this system is known as Oracle Data Guard, available in the Enterprise Editions of Oracle DB [327].

Since the Oracle Data Guard feature is something constantly running, one could say the Db element of the standby instance is not completely cold, but its use by the FastAPI standby instance is still happening upon a cold startup.

Consequences: I must determine what tools are used to ensure this cold startup when needed. But I discuss this in the next ADR because it is also linked to load-balancing decisions.

5.11. ADR: load balancer for HTTP requests, DNS, and eventual failover system?

Date (dd/mm/yyyy): 20/08/2026

Status: accepted

Context: users send requests to the FastAPI container, and I want to use a cold-startup version of the program that takes over in the event of a failure of the primary program instance instance. I need to determine whether this necessitates a form of load balancing.

Options: load balancer or not.

Decision: I opt for no load balancer for HTTP requests, and I adopt a Keepalived failover mechanism.

Rationale:

Load balancers dispatch user requests to the right recipient [328]. User requests go only to my FastAPI container, which is a monolith. My FastAPI container seems to have no limits on the number of uvicorn workers or on allocated capacity, according to online sources, and the program can run entirely on one machine. Also, Docker containers I will use know no limits in size [17, 329]. Thus I need only one FastAPI instance. Therefore, a load balancer is of no use for load balancing to the in serving mode program, because there is nothing to balance: only one destination.

I use a standby program that runs only when the in serving mode program experiences a system failure and shuts down. That does not justify a load balancer redirecting requests to the backup, since I can reconfigure routing via KeepAlived, which is open-source, runs on Linux machines (good since that’s the OS I selected), and uses the Virtual Router Redundancy Protocol (VRRP) for routing [89].

The DNS maps a domain name to an IP address, ensuring that the internet interface users use to access an online service points to the program that runs it [330]. Changing the IP address linked to a given domain in the DNS takes about the time to access the space to do that change, plus the time before the domain name links to the new IP address, and that can be very quick, a matter of a few minutes [28,

331]. This setup works but also requires a small website, in addition to the DNS, to display a fallback message to users, such as “the service is temporarily down for a few minutes”. And it also leaves some users in the dark until the new DNS update. So I prefer simply using VRRP, as it allows the domain to be linked to a virtual IP that can be paired with a new program in an instant [29].

Note that Cloudflare includes a cold backup failover mechanism via its load-balancing functionality [246]. And since geosteering, which I use from Cloudflare, is part of Load Balancing, I already pay for load balancing. However, the fees for Cloudflare load balancing increase with the number of origins (i.e., in my case, the backend instances) and the number of requests [179]. If failover ‘logic’ is in Cloudflare, this implies Cloudflare must know the standby origin and the primary origin for each country. But if I use Keepalived, Cloudflare only needs to know one origin per country: Cloudflare always performs the same task (this request goes to this vip) and never changes the vip destinations, and keepalived, which acts as a watcher, sends dummy test requests to check reachability of the primary app instance, and if reachability fails X times, assigns the VIP to the backup deployment. So this reduces the Cloudflare bill. And since Keepalived is free and performs failover just as well, it is a great option for reducing costs.

Consequences: Keepalived is software I must install to use. So it must be in the C4 deployment diagram, but it must live separately from the program instances, otherwise it cannot work if the program instance goes down. I propose to host it on Cloud.

5.12. ADR: Hosting and virtualisation decisions

Date (dd/mm/yyyy): 02/08/2026

Context: I must determine the hosting solution of my program (where do my elements discussed in the C4 model’s previous chapters physically and logically live).

Options:

- Cloud solutions: Oracle Cloud. AWS Cloud. Microsoft Azure. Google Cloud, etc.[3]
- Serverless, serverful.
- Containerisation solutions (docker, kubernetes, etc.), virtualisation solutions

[3] Note that FastAPICloud does not integrate Oracle DBs yet [332], but if it later does, it could be a viable cloud solution as well.

Decisions:

- Primary program instance (except KeepAlived and the Data Guard observer and external services) are deployed on my hardware on rented rack space in one data center;
- Standby program instance are deployed on Cloud, except the standby DB instance and its direct backup which live on my hardware on rented rack space in a different data center than the primary program instance's;
- KeepAlived and the Data Guard observer are deployed on Cloud too.
- The disaster DB backup lives on my hardware (if possible, otherwise on rented hardware of the data center directly for security reasons) on rented rack space in another data center than the primary program instance's and than the standby program instance's.

Rationale:

I first discuss the relevant requirements in logical order to progressively eliminate ineligible solutions, and then estimate the cost of the cheapest of the retained solutions.

This thought process will notably require calculating needed capacity. I calculate the needed capacity based on a quali-quanti prediction of average and peak loads and load variations, dubbed "Scenario 1," which I assessed personally, drawing mostly on US data but also on some international data. I can disclose only certain parts of that scenario's methodology and metrics in this paper. I predicted all these data points because they help predict the profitability of dynamic vs static capacity deployments, and because this app must scale with load with minimal latency, since low latency is business-critical here (each user sends hundreds of requests a day). In scenario 1, the app's activity is indexed by the number of concurrent usage cycles and the requests within those cycles, all based on what I call the total number of customers. Customers are not singular persons who might consider using the project. Instead, each 'individual' is either one person alone, or one organisation delegating part of that task to an employee who sends all the HTTP requests necessary to receive 1x the service that my app provides, assuming using my service is cheaper than the competitors' (whether in digital or human delivery of the service), and as or less bothersome as the competitors'. In this scenario, the system's components are used only for FinApp1.0; no other commercial endeavours use them.

In scenario 1, I slightly inflate the approximate numbers to ensure the capacity is not in survival mode. One metric I inflated are requests per second. Why? Many processes occur a steady number of times per user project; if the number of processes P1 decreases per request, this increases the number of requests. This is because the program relies on asking the user questions only when the task 2 of the AI agent (triggered by a "ask model to find this info" use case loop) cannot find the answer itself. I thus had to select a number of requests and, consequently, size the number of processes (indexed by the number of use-case loops) per request. For 90+% of the app's processes, the AI should generally

find all it needs, thereby requiring less user intervention, fewer HTTP requests, and more “ask model to find this info” loops per HTTP request. But it is better to model things as if the number of requests was higher than expected, because scaling for more requests than expected, even if background work of the ai agent, the use cases, and the DB is similar than for less requests, allows to give some headroom to the capacity needed for the rest of the api work (intercepting HTTP requests/sending HTTP responses), and to cope more easily with unexpected days where the ai agent cannot perform its task (document info extractions) due to very unreadable documents.

Despite inflating I use a request/s ratio all along, because even though all users of a given 125 s request-to-request usual interval can click at once, if they do, and if the capacity is sustainting only the /s load, then they will not each receive their HTTP response at once, but one after the other, making it more likely that the next HTTP requests come evenly spread over the usual request interval (125 seconds), and so on.

In the calculations, I also often round up results to the next higher decimal place to accentuate number inflation.

Solutions retained based on legal and industry requirements:

My software expects public and private clients. Private and public clientele require that the data they send to my program resides in sovereign storage, i.e., local physical storage, operated by local citizens, hosted either via a local cloud provider or no cloud [54, 169, 198]. They do not require the physical and logical separation of data between the country's public and private clientele [198]. What could justify it is if I desired to apply different security standards to the data of the private and public clientele, but both require the highest levels of security anyway. Thus, I need at least one DB deployment per country.

The deployment tools must comply with SOC 1 and SOC 2 (for the US market) and ISO 27001, all of which are compliance certifications that clients will demand [333]. The deployment tools must also comply with ISO 42001, as I, as system owner, will use an AI Agent and monitor/fine-tune its nature and use (which is all within the scope of that ISO) [334]. The deployment tools must also comply with local and regional regulations, such as FedRamp. Indeed, if hosted in the cloud for the US market, my service’s cloud requires FedRAMP authorisation in the US because of the nature of the customer/user project data my program uses [123]. Our own on-premises systems, or on-premises VMs, or cloud VMs, must comply with these rules. For the US, I did not find specific storage location requirements besides those discussed two paragraphs ago.

If we merge all the requirements, my deployment options would each need per-country-based hardware, with all components grouped co-locally. The options meeting these requirements are:

- About hardware access: Uptime Robot is strictly SaaS [335]. Same for Stripe and OAuth. So I do not control what hardware I access to use them. Oracle DB, KeepAlived and fastAPI are strictly deployable on IaaS or PaaS Cloud solutions [177], or on capacity accessed via another network (vps, ssh), or on-premises on hardware I own, or on-premises on hardware I rent, or on hardware rented or owned but stored in a third-party rack space provider's data centre. The AI agent can be SaaS, since some remote AI inference providers (companies running models for people to access via API calls) commercialise BGE-M3 and Qwen3 4B inference [235, 240, 336], and it can also be deployed on PaaS, IaaS, and the on-prem and renting solutions I mentioned in the prior sentence.
- About virtualisation: in a VM or directly on the physical machine;
- About containerisation: in containers or not [209, 337, 338].

I'm already planning for peak usage (since I consider the US deployment at peak usage), not for scaling up later. So the decisive criteria for selecting among these solutions are reliability (performance and security), user traffic patterns (how many requests/s, when), capacity limits, processing speed, debuggability, the needed level of control I need, and price.

Solutions retained based on backup plan and app's statelessness

The app is stateless (see ADR data storage solution) and HTTP request-driven and monolithic. I can deploy a Stateless app either via (a) server(s) that I configure and/or maintain (serverful deployment), or that I don't configure nor maintain (serverless deployment, also known as FaaS) [110, 223]. That statelessness eliminates none of the options cited in the previous subsection.

Also, the cold start backup option chosen for backup (see ADR: system failure management), does not eliminate any of the options either, since a cold start backup can exist in either type of deployment retained so far.

Finally, I want a different location for the primary and the standby servers and the disaster backup, because placing the disaster backup in the same data centre as the other components is a useless backup if the entire data centre is compromised, same for placing the standby elements in the same data centre. This means that if I use the cloud for both primary and standby elements and distaster backup, I want a cloud provider with multi-location guarantees. But as such, the backup plan eliminates none of the options cited in the previous subsection.

Next, to determine the solutions eligible based on capacity requirements I need to calculate the capacity usages of the components, and before that I must mention the expected traffic, and the chosen containerisation solution.

Containerisation solution chosen:

Common containerisation options are Kubernetes and Docker. Since I deploy everything co-locally, and since the app is a modular monolith, it appears Kubernetes would be overkill, since its advantages over Docker are to manage microservices across machines and with multiple images per microservice, as well as managing a large number of containers (which is not my case, since I will have a maximum of 3 containers) [339]. Kubernetes and Docker both perform automatic health checkups and restarts of images which run abnormally, which is a feature I need [17, 340]. Thus, I do not lose that feature by choosing either. Finally, there seem to be no container size limits, which is great for avoiding load-balancing issues (one FastAPI container means one FastAPI destination, same for DB, same for AI agent) [329].

Capacity estimates

Before starting I point out a term which will often be used here: serving/ serving mode. This term designates an element of a program when it is actively in charge of delivering the service to the user, whether as the single or co-deliverer of that service (e.g. a db performing db tasks co-delivers a service with the fastapi app and other services). This is opposed to a dormant, or in-dormant mode element, or non-serving element, which can be part of the architecture, can do some work of its own (e.g. a db mirror instance which also does some writes) when dormant or no active work, but which does not directly provide the service to the users (in this example, the primary instance of the db would be serving, while the mirror copy would be non-serving).

So in simpler terms, just know that when I designate a standby element as serving, it means the primary instance of that element failed and the standby had to fully take over.

Expected User traffic info (scenario-based):

Duration of a usage cycle

A usage cycle is the entire set of requests made by the same customer (a customer being a logical group and domain value object of one or more users sending HTTP requests as part of the same usage cycle) that are performed to benefit from all the app's features necessary to receive the service 1x in its integrity.

In my scenario, **a usage cycle lasts 1 workday.**

Times of usage and impact on capacity calculation:

Targeted clients mostly work 9-to-5 or 8-to-6. In the case of the US, if we have a single place where the primary instance of the program resides, and let’s say we set the program’s time to that time zone’s, because of the US time zone landscape (7 hours of difference from most east to most west),

the program does not experience high traffic only from 9-5 where it is located, but more from 6 to 8 pm of 5 am to 9 pm for example (if located in GMT -7). But each worker in each time zone still works a 9-5 or 8 to 6 shift, so the total amount of requests and usage cycles is comparable to what it would be if every client from a country was doing a 8-5 shift exactly at the same time. And because the US time zones imply that there will be a time all US clients from all the US time zones work at the same time, considering capacity needed for a peak of time zone concurrency is the best thing to do. Lastly, for dynamic capacity solutions like serverless cloud, it is not an issue to calculate it all as if users were all exactly working at the same time, because these solutions' fees are based on capacity and hour/min of use of the capacity: so from their point of view, it does not matter whether user traffic comes one the exact same hours or not, the eventual price is rather the same.

HTTP requests per second per million inhabitants:

Based on Scenario 1, we have an average of 214.14 concurrent users per million inhabitants if the load is evenly distributed throughout the week. Each user triggers, on average, 230.4 requests per day (the number of requests for 1 usage cycle, which usually lasts one workday), so that's 49,337.856 requests a day.

Divided by the number of seconds in an 8-hour workday, that's an average of 49,337.856 /28800= **1.71312 HTTP requests/s/million inhabitants**.

Earlier, we saw that the maximum number of concurrent users is the average number multiplied by 5. Following the same rationale, if all concurrent users for one week start a usage cycle on the same day, this would raise our number of requests to **1.71312×5=8.5656 HTTP requests/s/million inhab. On a workday maximum**. On off days and nights, it drops to close to 0.

Concurrent users, and new usage cycles per day per million inhabitants:

In scenario 1, there are = **214.14 new usage cycles per million inhabitants per workday**. For the US deployment, that's 214.14×350= 7,4949 new usage cycles per day

Since usage cycles last on average 1 day, there will likely be **214.14 concurrent usage cycles, and the same number of users[4] during working hours on weekdays**, assuming the pool of users starts their usage cycles at a constant daily frequency during the week. But that's not the maximum. Indeed, users work only on one usage cycle at a time. They usually complete at least one usage cycle per week due to the imperatives of their work schedule. The above number of HTTP requests assumes the week's pool of users' usage cycles start evenly spaced throughout the workweek. Since they can start their minimum 1 usage cycle on any of the 5 days in the workweek, they could all start and complete 1 usage cycle at the same time of the week. That would amount to **a maximum of 214.14×5 = 1,070.7**

4 I use the term user instead of customer, because whether or not a customer includes one or multiple users, only one user of the group is intervening for one usage cycle at a time, so I can use the term user here interchangeably

concurrent usage cycles and the same number of concurrent users for a million inhabitants on the same day (thus, an equal number of connections).

Users and usage cycles drop near 0 outside work hours.

Needed capacity of the DB, its backup space and a standby db:

I must determine the peak CPU usage, peak read I/O and write I/O, peak connection counts, and storage growth rate to estimate the capacity my DB needs [188, 194]. I discuss all that can lead me to determining them, then I determine them, and then I size the DB.

Some sizing data can only be estimated from historical data in the Automatic Workload Repository (AWR) Report for a DB. I don't have my DB yet running, so no AWR Report. Thus, I asked an Oracle DBA I know for one of his DBs' AWR Reports and compared them with data I already have about my DB's activity to estimate missing values, such as redo writes and physical writes/sec. Obviously, this is an approximate comparison because the DB can size redo entries differently across projects, resulting in different redo and physical write frequencies. But it is still informative in some aspects. I look at his DB's two 19-Jul-26 05:00-19-Jul-26 06:00 instances. Why two and not just one? Because some data I need for my comparisons (number of transactions) is not divided per instance in the report but is cluster-wide (because group 0 and group 1 LGWRs are not mapped to a single instance, but to a single worker, which can work for either instance) [341].

Since I use Oracle data Guard, the DB capacity I calculate includes the capacity for the in serving mode DB (see Glossary definition), at least two identical backup data storage spaces per DB (and the storage process to these spaces), and a standby DB instance. And Data Guard requires the standby DB to have at all times, at minimum, the capacity required to perform the writes based on the archived redo logs shipped to it [327].

Disk type:

I use SSD storage for the in serving mode DB and the standby for its speed advantages and environmental resilience (notably resistance to temperature changes and extremes) over HDDs. For the backups, as long as I have sufficient write bandwidth, I can opt for HDDs because, although slower, they are much cheaper than SSDs [33, 38, 217].

Optimisations:

My queries contain one or more of the following: equality lookups, range lookups, prefix matching, sorting operations using ORDER BYs, JOIN operations between large tables on indexed columns, filters on multiple fields. All these operations execute faster and with fewer IOPS when using b-tree indexing (a sorting method). Since I can use that indexing in Oracle and need to optimise query speed, I will use it [55, 342].

Also, sometimes the database stops using indexes when it evaluates that using the indexing (with the subsequent random read triggered) is more bytes-costly than a direct sequential read, assuming the program runs on an HDD (on which, even when the indexed random read is way faster than the direct sequential read and if the program runs on an SSD) [60, 130]. To ensure the DB does not lose speed by entering what it sees as cost-efficient direct reads (which end up being much longer), I must write my queries using FORCE_INDEX and PREFER_INDEX, or dbms_SQLdiag_internal.i_create_SQL_patch [147, 343]. But that also means that the more queries I want to execute using random reads, the more RAM I need (as that RAM is the cache that the random read accesses), and I need to enforce a KEEP pool (to retain important small indexes/blocks that large queries could push out of the cache otherwise), and optimise clusters [120]. Note that I especially use indexes for the tables which the users' requests only trigger reads. This is because, in the tables that user requests trigger writes to, indexes can significantly slow performance, since each write incurs indexing overhead (whereas reads do not) [183].

I also optimise the size of my cache, including the redo log buffer cache, so it does not become a bottleneck and waste time.

Finally, I pick the best database block size. Block size tuning can introduce problems with design adaptation when applied to an already working program and DB, but seems to have a positive impact when set up from the start [155]. There are 5 options available in Oracle: 2k, 4k, 8k, 16k, and 32k bytes [344]. Bigger blocks mean more data is read in fewer I/O operations (1 block read is one i/o), and that's highly favourable for read-heavy programs [185]. On the other hand, since each block gets 2 transaction slots, and the higher the number of slots, the more writes can happen concurrently, and the smaller the blocks, the more blocks there are if all other things being equal, smaller blocks are best for write-heavy DBs [185]. Therefore, dividing the DB into 2 tablespaces: one for write-heavy tables with one block size, and one for read-heavy tables with another block size, could increase performance. Some of my tables are really only queried by the program, so this could work, and Oracle allows such a double-sizing system [159]. But I could not find definitive sources stating exactly what problems it would bring (maybe double maintenance issues, etc.). Therefore, in doubt, I abstain from a double system.

My tables have a row size of 1kb or less. This eliminates the need to choose a high block size just so that one row can fit in a single block, because a 1kb row will always fit even in the smallest block. And I use optimised indexing on the read-heavy tables to ensure efficient random reads. These two factors push for a small block size, but not so small that my usual number of rows pulled (4+) requires too many blocks [345]. The in serving mode DB expects 1.32078125 loops of write transactions per use case loop, and 4.945775 read loops per use case loop, for the main use case (see Appendix 1), whose executions represent 80% of the program's use case executions. The writes are usually 90% altering, inserting, or deleting 1 row, but the rest of the time they easily go over 4 rows. Also, since

the DB's block size must be equal to or a multiple of the OS's block size [197], picking something like an 8,192b block size for the DB gives more OS options and gives enough space to write 6 or 7 rows (considering there is always some overhead). Therefore, in theory, 8192b, the recommended size, seems like a good size. I might revise after subsequent tests.

Nature of queries, number of writes and number of 'user write' commits:

In my program, queries are lightly joined with a small-to-moderate row count per call (one user_project's or one sw_instance's worth of instances/inputs/outputs) (my usual get query is trying to find all rows from a given table, whose value in column c matches the value in columns d and e from another table whose id=x.). In my case, a "logical" (in the sense of what the use case ordered) write writes one row at a time, taking ~7–10 values from other tables, dates, and (for the inserts) generating a unique integer PK. A read can query 100 blocks and usually return 10 rows or fewer.

Reads and writes can either go by loading data into the cache (also known as a buffer) and reading/writing in the cache, or by bypassing the cache and doing this directly on/from disk [138]. In my case, all of the DB's working set is eligible/likely to get loaded in cache for reads and writes (as much as cache size allows), because data blocks of the working set are accessed a lot of times and usually represent not many (less than 200) rows, and so the DB deems it more resource efficient to load it all in cache [138]. Even when a DB is fully eligible to be loaded into its cache, the DB will sometimes perform reads and writes directly to/from disk, bypassing the cache, when it deems it more resource-efficient [15]. One cannot constrain the DB to never bypass the cache. But there are ways to incite the DB to always do so. The main way is to keep a cache large enough to host the entire working set. An additional way is smart indexing. With modern SSDs, if indexing eliminates many rows and selects only 100 or fewer, the DB considers it more resource-efficient to load the remaining rows into cache. But if, despite the indexing, the eventual reduced selection is tens/hundreds of thousands of rows, the DB bypasses the cache to find info it wants [60]. Now, this indexing strategy is even more performant if we avoid large table scans in the SQL text itself: asking for something small that can follow the index path will avoid them. Finally, I can use the *cache* attribute for tables which I want to be always loaded in cache. Also, it's important to avoid direct-write SQL. Now, in my case, I will very likely manage to avoid direct reads and writes. Direct reads/writes comprise some direct plain and some temp reads/writes [3]. The temp ones result from using temp space due to insufficient RAM to perform certain operations (especially sorts, hash joins, order bys, group bys, and temp index creation) [3, 186, 227]. I don't have hash joins, because these are valuable on unindexed data, whereas I use indexing [57]. I don't have temporary indexes because I already have fixed indexes. And the other operations can be memory-intensive, but I aim to have enough memory to host them. So I avoid temp direct reads/writes. And I avoid the other direct reads/writes with the optimisations evoked earlier in this paragraph. But later, we will see that for certain sizing purposes, we cannot completely rule out direct reads and writes. **For the remainder of this ADR, I specify cache reads/writes as the**

total number of reads and writes in my program, assuming I can almost always avoid cache bypassing; for unavoidable bypasses, I will remind the reader of potential bypasses.

My app's workflow is 90% centred around the execution of a given use case dubbed main use case, which contains, on average, 1.32078125 loops of transactions (transaction in the Oracle sense of the term, i.e., DML [346]) (when a user commits one or a series of transactions at once) per use case loop (see Appendix 1), and that the use case has 20 loops per HTTP request, and each transaction loop contains a maximum of 100 1-row writes, we have around 1.32×20 = 26.4 separate transaction loops per HTTP request, at least. I deliberately use the average number of loops per HTTP request because it is a 'steady' value for our rationale: if we reduce it, the program logically requires more HTTP requests, thus keeping the total number of loops steady (but since I don't change the number of HTTP requests in this rationale, this stays steady as such as well). Using the average number of redo writes is deliberate too: this is because, if the use case is made in a way that only certain paths reach more redo writes than that average, but if the use case takes either of these paths, often the looping ends, and thus if we take the maximum amount of transactions, from the first loop, the use case could end, which goes against our 20 loops amount used across this rationale. Now, since 26.4 does not include the transaction loops of other use cases, which could execute aside the main use case, and which have around 15 transaction loops altogether max per HTTP request (1.5 transacton loop/use case loop × 10 use case loops) (here we can use 10 loops max, because it does not amount to the shortening of the logic executed), we have **around 39.6 transactions loops per HTTP request on average, which I round up to 40**. Each transaction loop contains a maximum of 100 1-row writes. So that raises our **total number of 1-row writes per HTTP request** to 40 × 100 **= 4,000**. Knowing we have a maximum of 8.5656 HTTP req/s/mill. inhab., we should have a maximum of 8.5656×4,000 = **34,262.4 1-row writes/s/mill. inhab** (**2,055,744/min**).

Because there is only one commit per use case loop, and on average 30 use case loops per HTTP request (either with the main use case + satellite ones, or just the satellite ones), we have around 30 user commits per HTTP request, reaching a maximum of 8.5656×30= **256.968 user commits/s/mill. inhab**.

Peak datafile reads:

In Oracle, reads include logical reads (reads of cached blocks) and physical reads (reads of uncached blocks). Logical reads are the DB block gets (read from the buffer) and the consistent gets (read from a process private memory) [3].

Accessing data from the buffer cache requires, first, if not yet in cache, a physical read that loads the block into cache, then one DB block get for each index it uses to reach the block (if any), and then a DB block get for the block of interest [10, 14].

Based on the main use case's probabilistic tree (Appendix 1), we know the main use case triggers, on average, 4.945775 datafile read loops[5] per use case loop[6], thus around 4.95×20 use case loops=99 datafile read loops per HTTP request for the main use case. If we add around 30 datafile reads at least for the remainder of the response process beyond that main use case, we have around 130 read loops per HTTP request, which I round up to 150 to be safe. Times our 8.5656 max HTTP requests per sec per million inhabitants, we have a maximum of 1,284.84 datafile read loops/s/million inhabitants. Each loop comprises searches for an average total of 25 rows. We know these rows are not necessarily in datablocks containing only them, because they are written wherever there is space in their target table blocks at the time of redaction. But some will likely be grouped, so I imagine we would have around 12.5 datablocks per read loop. This makes for a total of 1,284.84 ×12.5= 16,060.5 end data blocks (as in 'not the index blocks') targeted by SELECTs/s/million inhabitants. To that, we add that the DB has, on average, 3 indexes to go through before reaching its data block of interest, and knowing that all I considered as reads are DB block gets, I would not have any consistent gets, we obtain 16,060.5+ 3×16,060.5= **64,242 total number of logical reads/s/million inhab**.

I add the physical reads, which load the end and index blocks into the buffer cache. Their number is rather low, because most data is either loaded once for everyone (when the table stores rules), or once for a user project but only in small quantities, because usually the resumed or the started user project has few rows to (re)load, then does inserts which make for new rows. So it is likely 20 rows per each of the max 1,070.7 concurrent usage cycles per day, plus 10,000 rows for each of the 35 rules tables, all that divided by the number of seconds in a workday (28,800), divided by the potential number of these rows found in the same datablock (because the read loads the block in cache, not just the row) equalling (20×10.70.7+35×10,000)/28,800/2= **6.4482 physical reads/s/million inhabitants**.

My block size is 8192b, with <1kb rows. Oracle leaves 20% of a block empty for elements other than rows [347]. We thus have 6.4482×8,192b= **52.824kB/s/mill. inhab. of physical read size, and 64,242×8,192b= 526.2705MB/s/mill. inhab. of logical read size.**

The US deployment, with 350 million inhabitants, **thus expects** 526.270 MB×350= **184.1945 GBps of logical reads for** 64,242×350= **22,484,700 logical reads/s, and 52.824 kB×350= 18.4884 MB/s of physical read size (which is thus the disk read throughput) for 6.4482×350=2,256.87 physical reads/s.**

Peak data file random writes (physical writes):

DML transactions can persist physically either directly to disk, or first in the buffer cache [3]. In the latter case, DML changes are made to copies of database blocks in the **buffer cache, making them**

---

5 e.g., "for each …, read…", but sometimes there is no "for each", just "read..", usually at the beginning of the use case loop.

6 The execution of a use case before it loops back to a previous step of that same use case

‘dirty blocks’ [348]. DBWn later physically persists dirty blocks to disk, either one or more per physical write, within the limits of what the OS allows [10].

In my case, I want physical writes to never bypass the buffer cache. So all my writes are what we find under the AWR Report under “physical writes from cache” [3].

An online source reports that a DB had a physical write throughput of 141.7 MB/sec, with 18,135.2 blocks/sec and 15,096.9 I/O write requests per second [51]. The number of block changes/sec is 120.13% of the number of i/o write requests/sec (15,096.9×100/18,135.2= 83.26). This suggests that some physical write requests led to more than one block change (consistent with the fact that a single physical write can persist one or more dirty blocks). Also, with the probable block size in that source being 8192b, we get 18,135.2×8192b = 148.564MB, which is almost the execution size of 141.7MB/s of writes. This suggests that throughput is the number of block changes of a given size the program can sustain in a given period. So the physical write throughput is DB’s block changes/s × block size. This substantially eases physical write size prediction, because no matter how many blocks are persisted per physical write, the size of writes is the size of the blocks they persist.

In my DB, logical writes are 1-row each, but each write transaction loop affects the same buffer cache blocks (as long as the original inserts of these target rows could populate blocks that had multiple rows available), not one block per row of the transaction loop. Due to block overhead, my blocks contain 6 or 7 rows of data (I use 6 rows to avoid undersizing). The DB triggers physical writes when a certain percentage/number of buffers (a buffer is the cache container of one block) are dirty [10]. Inserted data rows are placed into data blocks with available space. Thus, a single insert of X rows into blocks of Y rows will require at least X/Y blocks to be populated, but it could require more blocks when filling some blocks that were already partially filled from previous transactions. My writes per request do not thus necessarily affect the entirety of each block it touches, but the entire daily activity; the writes are condensed into blocks fully modified on each row or near that. My blocks are each 6 rows. Thus, **max physical to-write blocks/s/million inhab.**= 1-row transactions per second /6 = 34,262.4 1-row writes/s/million inhabitants /6 **=5,710.4**, for a total of 8,192b×**5,710.4=46.78MB physical write size/s/mill. inhab**. per DB, **and that is without the archlog writes (which I discuss later)**.

So how much physical write IOPS is that? As noted earlier, to-be-written dirty blocks might get grouped into a single physical write I/O [180, 349]. The criterion(a) for grouping is undocumented. So I must assume that, **at most, for my DB, the number of physical writes equals the number of blocks to be written**. This is a practical approximation, because it is probably an overestimation, which ‘balances’ to an extent the potential underestimation of the number of blocks to write.

For **the US deployment, that amounts to 1,998,640 physical write blocks/s, for a physical write size of 16.373 GB/s, for a max of 1,998,640 physical writes/s (physical write iops), and that is without the archlog writes (which I discuss later)**.

Peak redo write size:

Redo writes are the physical persistence, by the LGWR, of the redo log entries found in the redo log buffer, to the online redo log files (which are called online files although they are not online but on disk). The LGWR writes redos at least every 3 seconds. The number of redo writes the LGWR performs per second is called the redo write size and is determined by the machine's capacity and whether there is space in the online redo log files to write redo entries [3, 8].

We have a maximum of 2,055,744 1-row writes/min/mill. inhab. hitting the redo buffer (see precedent earlier). A 1-row transaction usually changes around 4+ blocks (the target table block, the undo block, the block of the transaction table for the undo segment, and the index leaf block at the very end of the index tree branch), and all that fits into 1 redo entry: so in my case, one 1-row write equals roughly one redo entry [1, 94]. Redo entries are not fixed in size [1]. But I noticed that for simple-to-medium writes in my contact's report and online sources, a 1-row transaction usually requires a 500B+ redo entry. So I estimate it at 600 bytes just to be large. So I would have around 600 B ×2,055,744 = **1.23345 GB of redo entries/min/mill. inhab. (20.5575MB/s/mill. inhab.).** I want a DB backup space per DB instance on a separate local disk, and a disaster backup in a specialised data centre. Each backup receives archlogs from the in serving mode DB at all times, as well as full and incremental block backups. When the primary instance of the DB is in serving mode, the Data Guard standby DB receives redo entries in real time. And for each deployment, I also have a disaster-recovery backup space that receives writes. So my number of concurrent writes is 4 for the primary DB, and 2 or 3 for the standby DB, which raises the total needed redo write bandwidth of the in serving mode LGWR to 20.5575 MB/s×4 **= 82.23 MBps/million inhabitants for the primary DB, and for the Data Guard, when it takes over (since it might also write to the backup)** 20.5575MB/s×3**=61.6725 Mb/s/mill. inhab. For the US deployment**, that's respectively **82.23MB×350=28.7805 GBps** for the primary DB, and 61.6725×350=**21.585375 GBps of redo size for the standby DB (that's when it runs as the main; it's two thirds of that when it is in standby mode)**.

Online redo log files, their size, and redo archive space size

Let's first discuss how transaction logging and consistency work, to understand why redo log files and redo archive space matter.

First, as noted in the previous subsection, the disk contains 'online redo log files', which store redo entries from the log buffer. Ideally, I choose to archive redo log files because it ensures better data and process recovery in case of failure: the process can check in the archives if all previous logs' writes have been physically persisted. In a DB with log archiving, a log file fills up, is archived, and then

becomes available for reuse (for new redo writes) only when the physical writes it notifies are performed and its contents have been fully archived. Therefore, a DB with redo log archiving must have at least 2 online redo log files [350].

Also, a log group is one instance of the same redo log process (the writing of given redo entries). In my case, I have an in serving mode DB and a standby DB, on different disks. Each must contain the same online redo log files, since I want the backup to take over immediately in the event of a failure. Each DB must contain at least two redo log files. Also, it seems that groups are not designed for in serving mode/standby backup purposes: the in serving mode DB has its own groups and members, and the standby or backup has its groups and members. Thus, I use a minimum of two log groups, with each group having at least 2 members on the in serving mode DB's disks, and the same config on the standby DB [184, 350, 351].

Thus, on each disk, I must set the size of the online redo log files, their number, and how many redo files the archive space holds at a time.

The redo archive space stores archived redo log files until they become obsolete. The company using the DB determines what obsolescence means [350]. In my case, "obsolete" means the backup space has received the full DB snapshot. Since I want a snapshot every 1 hour, I need enough archive space to store one hour's worth of redo archive. Thus, ARCn and DBWR must together persist the physical writes and redo log file 'R1' as fast as LGWR fills the online redo log file, which precedes R1 in the LGWR queue.

Archiving (ARCn) and physical writes (DBWR) occur simultaneously [352]. In my case, they both write to the same disk (the in serving mode DB's disk and the backup DB's disk). One disk expects 46.78MB of physical write size/s/mill. inhab., for 20.5575 MB/s/mill. inhab of redo log entries. So ARCn and DBWR together take 67.3375MB/s/mill. inhab. of disk write throughput per disk, while LGWR (which also writes to disk, since online redo log files are on disk) uses 20.5575 MB/s/mill. inhab. of write throughput per disk. **For my US deployment**, that requires **disk write throughput of 23.568125 GB/s for ARCn + DBWR, and 7.195125 GB/s for LGWR (so 23.568125GB + 7.195125 GB = 30.76325GB total write throughput, which we can round to 31 GBps)**. SSDs write up to 12GB/s [79]. Therefore, I will need to split my in serving mode DB (same for the backup) onto multiple disks (using a process called ASM stripping: one DB, multiple storage disks logically grouped as an ASM disk [353]). But that also means that the write speed will be met.

Changing the redo log files' size does not affect this throughput requirement, because a single write does not become heavier/lighter as the redo log file size grows/shrinks. But what size to pick? A checkpoint occurs when ARCn finishes archiving a redo log file. "A checkpoint can momentarily reduce runtime performance" [354, 355]. Thus, more archiving processes create more checkpoints, which slows overall performance. But archiving small redo log files still has advantages because, on

restart for recovery, a DB checks archived redo logs to verify that all have been performed. The bigger the archive, the slower the restart. That's why Oracle recommends a log file size that allows a log switch at most every 20 minutes, with a maximum of a few GB and a minimum of 4 MB [356].

If my DB switched log files every 20 minutes with a redo write size of 20.5575 MB/s/mill. inhab., one redo log file would be 20.5575 MB×60seconds×20minutes = 24.669 GB/mill. inhab, so 24.669 GB × 350 = 8.6341 TB for my US deployment. That is way over the "few GB" recommendation. Therefore, I set the redo log file size to **4GB**. And to have some LGW free space in case writing and archiving slows down for any reason compared to how fast LGWR fills up redo log files, I select **8 online redo log files per DB**; 4 log groups of 2 members, all part of the in serving mode DB, and 4 groups of 2 members each in the standby db (that's a total of 8×4GB= **32GB per DB**). I want a full DB snapshot backed up to my backup storage every hour. Therefore, I need to archive an hour's worth of redo entries. 20 minutes' worth of redo entries for my US deployment is 8.63415 TB. Thus, I need **redo archive space of** 8.63415×3= **25.90245 TB per DB instance**.

DB Physical disk storage space size:

For the DB, the disk (in my case, multiple disks, as mentioned in the previous subsection on ASM stripping) stores tables and DB work elements. Let's discuss the total disk size needed for the in serving mode DB, and note that the backup DB's disk is of the same size.

I have 71 business tables. 11 are of fixed size, 24 are of evolutive size but are the same size no matter the deployment and the number of users (mainly some law-related tables, but they do not change in size drastically over time; they just require a very large sizing), and 35 are of evolutive size depending on the number of users.

Each of the first 35 tables expects 400,000 rows of 500 bytes each. That's 35×400,000×500 bytes = 7GB of raw data, and that's per deployment, not per million inhabitants. With 20% of the table size worth of datablock overhead [347], and 100% of the table size in indexes (very likely in my case, because they are read-heavy tables, except when admin commits), and at least 20% of the undo tablespace, which totals **16.8 GB per deployment**.

27 other tables have each up to a thousand < 1 KB rows on average per user project I want to hold info of at a time. 1 other table has 5 <1kb rows per user project whose info I hold at a time. 5 other tables will have up to 20 1kb rows per user project whose info I hold at a time. 1 other table has up to 50 <1kb rows per user project whose info I hold at a time. The number of user projects whose info I hold at a time is the sum of the uncompleted user projects from the last 3 months (arbitrarily chosen duration) and the completed ones from the last 7 workdays. The completed ones represent a workweek's worth of user projects, which is equal to the maximum number of usage cycles in a workweek, i.e., 214.14 new usage cycles per day, times 7 (since a usage cycle takes on average 1 workday to complete, so those completed from D-8 until D-1 are those finished in the last 7

workdays), so around 1500. In 3 months, at most 1,000 user projects would not have been completed per million inhabitants (an arbitrary estimate). So these tables store info for 2,500 user projects. Together they thus represent 2,500×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)= 67.8875GB per million inhabitants. With 20% of the table size worth of block overhead [347], and 40% of the table size in indexes (fewer indexes here because these are write-heavy tables, and, as explained in the Optimisations subsection, indexes in write-heavy tables slow down performance), and at least 20% of the undo tablespace, which makes for **122.2GB per million inhabitants**.

1 and last other table has 1 <1kb row per customer I want to hold info for at a time, and another with 10 <1kb rows per customer I want to hold info for at a time. For a 1-million-inhabitant population, we expect at most 214.14 (new usage cycles per million inhabitants per workday; one new usage cycle can mean no new customer) times 261 (number of workdays per year) customers per year, i.e., 261×214.14 = 55,891 customers max per year. This seems large enough to hold and should stay rather steady over time, because the customer pool does not grow and is expected ot use the service every year. So I retain for an equivalent of 65,000 customers. That amounts to 65,000×(10+1)×1kb = 715MB per million inhabitants. With 20% of the table size worth of block overhead [347], and 40% of the table size in indexes (fewer indexes here because these are write-heavy tables, and, as explained in the Optimisations subsection, indexes in write-heavy tables slow down performance), and at least 20% of the undo tablespace, which totals **1.287GB per million inhabitants**.

Finally, we add space for the index blocks, which store information about the indexes to be enforced on a given table.

Each index block has a fixed size, just like the other data blocks (in my case, 8,192b). But the number of index blocks is fixed by three factors: the number of indexes, the number of rows per table, and the datablock size. An index block stores rows, each containing an index key, a row ID, and the related value, which is the subject of that index. There are as many index blocks as needed to store the value (that matters for a given index) of a given row. An 8,192-byte block uses 80% of its capacity to store this index info, so 6553.6 bytes. Each index entry is likely around 50 bytes max. So each index block contains around 131 index entries. My tables usually have a maximum of 3 indexes. So for 131 rows of a table, one can count 3 index blocks. I have 75 tables. Among these, 35 tables will contain 400,000 rows each. So these 35 tables require (400,000×35)/131×3= **320,611 index blocks per deployment (320,611×8,192bytes=2.626GB/deployment)**. Then, 34 other tables require 2,500 user projects. Together, they thus represent 2,500×(27×1,000 + 1×5 + 5×20 + 1×50)/131×3= **1,554,676 index blocks/million inhab (12.7359058GB/million inhab)**. Then the last 2 other tables expect 65,000×(10+1)/131×3=**16,374 index blocks/million. inhab. (134.135808MB/million inhab.)**.

For the US deployment, that's a total of 16.8 GB + 122.2 GB ×350 + 1.287 GB × 350 + 2.626 GB + 12.7359058 GB ×350 + 134.135808 MB ×350 = 47.7443906 TB disk space for the table blocks and index blocks.

To that, we add capacity for the online redo log files (**32GB per DB for the US deployment) and their archives held in both the in serving mode and backup DB's disks until deemed obsolete (25.90245TB of space per DB for the US deployment)**.

There is also size needed for temp, measured by the size and number of physical reads to and writes to the direct temporary tablespace [3]. Sorting, joining, aggregation, temporary index creation, or other operations which require more memory than available, trigger the creation of temporary tablespaces (or the use of the default created one) to store intermediate results [186, 227]. As indicated earlier, I want my DB to have enough RAM to run all its current-session work in memory. And thus temp would be nonexistent. However, I want to reserve some disk space for temp in case direct reads and writes occur anyway (due to a cache issue or otherwise). Let's see how I estimate it.

My contact's DB had 45 physical writes to direct temporary buffers out of 49 physical writes direct (see Appendix 2): that's 91.8% of the direct writes. It made 45 direct physical writes out of the 10,518 it had to make during the session of interest: that's less than 0.43% of direct physical writes. The least executed (2 SQL texts) were executed fewer than 49 times together, and there was only one other SQL write text. Writes did not represent a minority of the scripts; the former is logically not present because that DB (just like mine) had almost no physical reads. The size of these buffers was 8,192 bytes each, because their size equals the size of the data blocks (that's what Oracle internal AI services told my contact), and the data blocks were 8,192b (see Appendix 2). So for 2.92/s, my contact's DB had a temp of 45×8192b=368.64kB. But we don't know what 45 temp. buffers meant in terms of how many blocks were written. Since I have 5,710.4 blocks physically written/s/mill. inhab., I need a temp of (5,710.4 /2.92) ×368.64kB= 720.918444 MB/mill. inhab, thus, for my US deployment, 720.918444 MB×350=252.3215 GB. This is honestly a bit small, because my contact's DB had so few direct writes. I'd rather oversize storage and reduce it later. And we need to take into account potential overflow of read operations. So I choose **2 TB of temp space for the US deployment**.

Putting it all together, for the US deployment, the primary DB and the standby DB (because it must have all when it takes over) each require 47.7443906TB disk space for the tables and index blocks + 2TB for temp +8×4GB for redo log files +25.90245TB for their archive = a little under 76TB (let's round up to **76TB) of disk space.**

For backup, I choose to use Oracle Recovery Appliance, which, unlike conventional RMAN (which keeps full DB backups, incrementals, and arch logs), keeps 1 full backup, only incremental backups (backups of only what changed), and arch logs. I choose it because my DB is very big and Recovery

Appliance's approach (namely 'incremental-forever') can reduce storage needs and also speed up recoveries [13].

The storage size for that backup solution is determined as follows.

First, I want enough backup space in each backup storage to store the unique full DB backup. This full DB backup is the backup of the DB at whatever retention limit I choose (e.g. with a 15-day retention, the full is the DB as it was 15 days ago). So this full equals the size of the DB on that day, i.e., max 76TB.

Secondly, I want each backup storage to hold 1 incremental backup per day for the recovery period. Sizing for the incremental backups requires knowing each incremental backup's size and the number of backups held. I choose a 30-day retention policy. Activity is almost null on weekends. And I add 2 workdays' worth of admin changes per 30 days. Therefore, I need space to hold the equivalent of 29/7×5 + 2 = around 23 incremental backups. The physical write size per day is not the size of an incremental backup, because these writes for the same user project but for spaced use case executions often alter the same data blocks, and the incremental backups do not record all the changes, just the state of the changed blocks at the incremental backup time. So the right indicator is the compound daily edited blocks size in DB tables which received user project and user-specific info during the day. Earlier, user project and user info specific DB tables's size was calculated per million inhabitants as 2,500×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8 + 65,000×(10+1)×1kb×1.8, but 2,500 and 65,000 were user or user project counts for longer period than a day, while in practice, for a day, these two counts come down to 214.14. I add a little overhead for the indexes (worth 134.135808MB/million inhab) since they might change frequently. So the total incremental backup size for the US deployment is 214.14 ×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8×350 + 214.14×(10+1)×1kb×1.8×350 + 134.135808MB×350 = 3.712 TB. Therefore, the compound incrementals pool per backup storage is 3.712 TB × 23 days = 85.376 TB. This confirms I need less backup storage than with conventional Oracle backups, because 85 TB is nearly the size of one full backup (76 TB), and conventional Oracle backups would store 4 of these full backups for my 30-day retention.

Thirdly and finally, each backup retains the day's arch logs for each of the 30 days. We know the DB expects 1.23345GB of redo entries/min/mill. inhab (evoked earlier in the ADR), so a little less than 14 TB for the 30-day period (1.23345GB × 60 minutes per hour × 8 hours × 23 workdays of load).

So each of the 3 backup storage systems needs 14 TB + 76 TB + 85.376 TB = roughly 176 TB of disk storage space, but I plan for 230 TB to give each about 30% headroom.

DB Peak Storage bandwidth:

'Storage bandwidth is how much data can be read from / written to a disk on a given period' [31]. We know for the primary DB we have 31 GBps of write throughput per component (evoked earlier), plus 18.4884MB/s of physical read size/s. Therefore, **each of the five disk spaces for the components mentioned (the primary DB, the standby DB, the standby DB's direct backup, the primary DB's direct backup, and the disaster backup) requires 31 GB/s of write bandwidth. The standby DB, when in serving mode, and the primary DB each require 18.4884MB/s of disk read bandwidth. Also, the standby DB, even in standby mode, loads blocks from disk into its cache; since it writes to this cache first, it requires 18.4884 MB/s of disk read bandwidth.**

DB Peak memory:

The RAM for the DB contains everything that must live in the live memory for use.

One alive thing in memory is the redo log buffer and what it contains. The redo log buffer is the intermediate storage of redo log records before the LGWR writes them to the online redo log files (they are called online, although they are on disk) [341]. The LGWR writes the redo logs in the log buffer to the redo log files at least every 3 seconds [8]. So 3 seconds' worth of redo logs is what the redo log buffer must be sized for. In my case, 3 seconds' worth of redo logs equals 20.5575 MB/s × 3 = 61.6725 MB/s. But if we size the redo log buffer to that size, it would always be nearly full, and if DB processes behind that buffer checkpoint slow down, the buffer could overflow. So I size the **redo log buffer** a bit bigger, at **1GB per million inhabitants**.

Also, I want my reads to be in-cache, and writes to first hit the cached blocks. There are several things needed for that.

To have all my reads and writes in-cache, I first need all my currently in serving mode blocks and structures associated with them to be in-cache. We saw earlier what all tables weigh per million inhabitants in the physical disk. But that's for the total number of usage cycles and customer info held at a time in the physical disk. What I need in the cache is only blocks relevant to the usage cycles currently active on a given day, since these are the only ones that need to be loaded. If we load all tables which do not change in size **per deployment**, that requires **16.8GB of cache**. To that we add blocks storing info specific to the max 1,070.7 simultaneously active usage cycles/user projects. On the physical disk, you find 34 tables concerned by this rate (see earlier in the ADR). But in-cache, we likely need only 25 of those at a time. As noted in the disk storage subsection, the total weight of these 34 tables per million inhabitants is 2,500×(27×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8. The in-parentheses 27, 1, 5, 1 represent the tables, and the 5 and 1 are always loaded in cache, and 2,500 is a fixed number (the number of held user projects). So the calculation for the cache capacity for these 34 tables is 1,070.7 ×(18×1,000×1kb + 1×5×1kb + 5×20×1kb + 1×50×1kb)×1.8= **35GB of cache per million inhabitants**. To that, we add one that scales with the number of customers, with 10

< 1kb rows per customer, and one with one 1kb row per customer. Since we have a maximum of 1,070.7 users active at a time, this table requires 1,070.7×(10+1)×1 kb = 11.78 **MB of cache per million inhabitants**.

To that we add RAM capacity for loading their indexes. This is calculated as **2.626GB/deployment and [**1,070.7 ×(18×1,000 + 1×5 + 5×20 + 1×50) + 1,070.7×(10+1)]/131×3×8,192bytes + **134.135808MB** = **3.78307827GB/million inhab**.

To have all my reads and writes in cache, I don't need additional RAM capacity to perform logical reads and buffer writes, since they apparently incur no RAM cost [10]. But for operations that bypass the SGA cache and work in the PGA, there is a RAM cost [10]. My physical writes and reads do not bypass the SGA cache. So I just add an arbitrary RAM capacity of 20GB for exceptional cases.

For the US deployment with 350 million inhabitants, this amounts to 1GB×350+11.78Mb ×350+ 16.8GB+35GB×350 +**2.626GB + 3.78307827GB×350 +** 20GB = **a little less than 14 terabytes of RAM for the in serving mode DB.**

The backups do not need that RAM, since they just store copies of files.

When in serving mode, the standby DB will need as much RAM capacity as the primary DB. When in standby mode, it needs as much as well, because it always persists transactions to the cache first, just like the primary DB [126].

DB CPU:

Logical reads, direct reads, direct writes, buffer writes, physical writes of the dirty blocks, ARCn, redo log writes, are the Oracle activities that consume CPU.

For a given number of operations, DB CPU usage can vary greatly depending on the complexity of the operations (e.g., aggregations, sorting), since CPU time increases with the number of SQL executions [148]. One way to estimate it is to look at the CPU Time (the total time it worked asynchronously) and the total CPU enqueue time (the time CPU would have worked if running synchronously each process sent to it) in AWR Reports of comparable DB loads. Another way is to look at ranges of CPU time a given DB task takes, based on load simulations done by other people and pick a CPU time within each range based on the level of complexity of my tasks, then multiply each CPU time picked by the amount of that DB task my DB does [24, p.5, 108]. SLOB is one of such tool for load simulation. It does not mimic my DB activity exactly: instead of running my code and my tables, it is a simulator of activity for a given schema size, block size, concurrency level, and quantities of given types of SELECTs and UPDATEs that SLOB itself provides [32, 182]. I suspect INSERTs and DELETEs to have a different CPU usage than those two statements. Since my DB has all 4 types of statements, I need a tool to simulate CPU usage for a DB that runs them. Such tools are Hammerora or Swingbench [164]. A third way is to actually run my own SLOB (or Swingbench, etc.)

test, specifying the parameters I want it to use (the number of query types and schema/table sizes) [253]. Tools like Swingbench require specific configuration, which can take time and some Java knowledge to run my own queries on my own tables [181]. A fourth way is to simply run my queries atomically in Oracle and review the CPU time for each run in the AWR report [133]. That way I obtain the CPU time per query, and can determine the average CPU time per main use case loop based on my probabilistic tree of occurrences of these queries in one main use case loop, then apply the same value to loops of satellite use cases, then aggregate CPU time of all loops of one HTTP request, then multiply that by the number of requests/s. This 4th solution applies only to a running app with populated tables. A fifth way is to approximate (with AI or otherwise) the average types of reads, writes, and redo writes my program executes, check their CPU requirements, and then simply multiply by the number of occurrences.

My DB tables are not populated, which eliminates option 4. My load will include other writes than just UPDATEs, which disqualifies SLOB. And I don't want to spend too much time setting up other benchmarking tools, since the current deployment is more of a training exercise than anything else. Finally, I did not find benchmarks with very comparable queries. So I choose the fifth option.

Each write to the cache creates/alters one row with ~7–10 values pulled from other tables, some dates, and an unique integer PK. So each takes around 2 µs. Each logical read searches all rows from a given table, whose value in column c matches the value in columns d and e from another table whose id=x. that costs 10 µs max. Each DBWR block write, LGWR write, block ARCn archives, and physical read, takes on average 5 µs max. In my US deployment, the DB expects 22,484,700 logical reads/s, 2,256.87 physical reads/s, 1,998,640 physical writes IOPs (same number of blocks to write), 2,055,744 redo entries per second, and 1,998,640 archived blocks per second. So in total, we would have 22,484,700×10µs + 2,256.87×5µs + 1,998,640×2µs + 2,055,744×5µs + 1,998,640× 5µs = 249.12748435 seconds of compound (i.e., if everything was dealt in a single thread) DB CPU time/s. If I use 256 double-threaded-core CPUs (such as AMD EPYC 9996 with its 256 cores and 512 threads [357]), the US deployment would need 249.12748435/512= approximately 0.48657711787 seconds to handle the per-second workload of the DB. CPU does not need headroom. So I only need half of 1 AMD EPYC 9996 CPU's capacity for the primary DB. Therefore, **my primary instance of the DB for the US deployment requires 1 AMD EPYC 9756 CPU (128 cores).** Given the bandwidth limitations, I wouldn't worry that this processor isn't enough, as it has 1k+ GB/s bandwidth (see AMD's website).

In serving mode, the standby DB requires just as much CPU. In standby mode, it performs the same activities, except for logical reads. So instead of 249.12748435 seconds of compound DB CPU time/s, it expects in standby mode 2,256.87×5µs + 1,998,640×2µs + 2,055,744×5µs + 1,998,640 × 5 µs = 24.2804844 compound DB CPU time/s (10.2604000911 times less than when in serving mode). So **the standby DB in standby mode needs** only something 10 times smaller than the AMD EPYC

9756, so approximatively something like **the AMD EPYC 9116 with its 16 double-thread cores (since there is not one processor exactly 10 times smaller)** [357, 358].

Needed capacity for the AI Agent:

My AI agent relies on 2 models. A model's context is information specific to the given request (in our case, the documents and files the AI agent works on, plus the request). The parameters are all the AI model's logic capacity and internal knowledge acquired over time/training. Tokens are the units of the data that the model processes [47]. The AI model requires space (ideally in a GPU's VRAM) to load all this information, as well as some processing power (CPU and GPU usage) to execute the logic using the necessary knowledge, and some disk storage to hold data that it loads into VRAM [241].

Note that a standby instance of the AI agent is also needed; it will have the same capacity as the primary instance in serving mode but requires only some storage capacity when on standby. But I do not want the standby to also contain user-imported contexts, since files can sometimes carry malware, and I do not want it to spread to my standby, which must remain clean.

AI agent Disk storage needs:

On disk, an AI agent stores model parameters and sometimes some context.

The AI agent task 1 (i.e., extracting the entire document as a JSON) uses 10mb worth of user-input documents[7] per usage cycle × maximum number of usage cycles in one day for a million inhabitants= 10MB × 1,070.7 = max 10.71Gb per million inhabitants a workday worth of original-form documents for the AI models. Task 1 produces JSON extracts of these documents. The extracts likely weigh 2 MB per user project [359]. So we expect 2 MB × 1,070.7 = a maximum of 2.15GB of JSON files per million inhabitants per workday for the AI models. Task 2 simply uses sections of the JSON extracts Task 1 produces, thus producing or storing no additional context. I need to hold the extracts for at least a full workday because Task 2 processes for a single-user project occur during almost the entire 1-workday usage cycle. Holding them in memory would be expensive. Therefore, I store them on disk. Task 2 does not store anything on disk. A usage cycle usually lasts 1 workday. But some users might pause and resume a usage cycle many days or weeks later. To avoid accumulating original documents to astronomical proportions, and because the AI agent is fine with just the JSON extracts (even if Task 2 fails to find info from these extracts, the app asks the user), I decide to retain only the JSON extracts for 14 days. For the US deployment, the IA agent thus requires 14×2.15GB×350=10.535TB of disk storage for the contexts or more precisely 13.6955TB with 30% headroom.

---

7 This is because the concerned documents are financial reports in their non public form (just the numbers, not the detailed public explanation), and from looking at different ones online, this seems to be the trend size if we group all potential years of coverage + the size of the documents [62, 172].

My models are a BGE-m3 and a Qwen3 4B. Their parameters weigh a little differently depending on the language used to interact with them, but since my code contains human wording in English, I know that BGE-M3 (which I choose because it is the bge model supporting the most input tokens at once, which I require for the input token load discussed later in this subsection) has 569M parameters, totalling 2.27GB [360]. For Qwen, it appears that at the highest precision level, 1 parameter takes 4 bytes [78]. So it would seem that for Qwen3 4B, because it has 4 billion parameters and I choose the highest precision, the parameters of my Qwen3 weigh 4billion times 4 bytes = 16 GB. That's a total of 18.27GB of storage needed for the parameters (let's round up to 19 GB to be large), or, more precisely, 23.751GB if we add 30% headroom.

And we can add an extra 5 GB for the installations of vLLM and LM Studio and their headroom.

Thus, my **US deployment requires** 23.751GB + 13.6955 TB + 5GB = roughly **13.8 TB of disk storage**.

Secondly, when a model stores or retrieves data from the SSD, this produces read/write throughput. Task 1 processes per day are concentrated in a short time window, and Task 2 processes are evenly spread with no real break. Thus, the models load their parameters once per day max, and then just keep using them. Thus, even if loading these parameters is a bit slow, it is fine because it happens almost never. Thus, the read throughput of parameters is negligible, or we can set it to 100 MB/s to be safe. To that, we add read and write capacity for the contexts. Requests for task 1 come at a rate around 10/125s/million inhab, so very few per second, but we can round up to 1 per second for the current task to account for higher peaks. Task 1 writes the JSON extracts to disk. Thus, Task 1 writes for the US deployment and thus requires 2 MB of JSON extracts × 1 Task 1 request ×350 = 700 MB/s of disk write throughput. Task 1 has no context read throughput requirements, since it deals with info loaded directly from the user into the VRAM. Task 2 has only read throughput impact on disk, as it writes nothing to it, and simply loads into cache a small section of the JSON extracts, worth a maximum of 4 KB per Task 2 request.[8] The software expects around the average number of requests per second per million inhabitants × number of use case loops requiring info extraction × number of info extracted per loop = 8.5656×22×1 = 188.4432 Task 2 requests per second per million inhabitants.[9] Therefore, Task 2 processes for the US deployment require a maximum disk read throughput of 263.82048 MB/s (4KB×188.4432×350). So **the US deployment requires 263.82048 MB/s of disk read throughput and 700 MB/s of disk write throughput for its AI agent.**

[8] This is estimated based on comparison to sizes of JSON documents found online [359].
[9] 20 is the number of loops in the use case representing 90+% of the use case workflow, and I add 2 loop for the other tasks which can run in the same HTTP request-response run.

AI agent's VRAM:

Task 1 of the AI agent's work, i.e., the full info extraction and JSON file conversion, must have a context big enough for each potential concurrent request for task 1. Requests for task 1 come at a rate around 10/125s/million inhab. Also, sometimes the task 1 process might last up to two minutes. So the compound size of the context for task 1 at a time per million inhabitants is 10mb×10 + the size of the data of the requests themselves (probably 10MB total per million inhabitants) plus the produced JSON extracts of 2MB each, for the 10 concurrent task 1 requests (20MB total). So we are near **130 MB max compound task 1 non-token data context size per million inhabitants (169MB with headroom)**.

The software expects 188.4432 task 2 requests per second per million inhabitants. And we saw in the AI agent ADR that when a request needs a document, it does not load its entire JSON or original document again, but just loads in context the section of interest from the target JSON document. A section is max 4kb.[10] Note that 188.4432 is a per/second rate, not a concurrency metric. Task 2 requests (188.4432/s/mil. Inhab.) each trigger a response process that takes around <10ms, ideally (I estimate this because, due to the JSON files' sections, the Task 2 model has very little JSON text to read). Thus, we expect 188.4432 / (1second / 10ms)= 1.884432 concurrent Task 2 processes per million inhab. So **the total Task 2 non-token data context size at a given time is** 1.884432 contexts of 4kB, plus a few bytes of info from the data table indicating potential formulations of the sought info (see ADR AI agent), so around 7.54kB **per million inhabitants (10.0534 with headroom)**.

Task 1 and task 2 also induce token creation in the context, so they represent the non-token data fed to them in a way the AI model understands. These tokens created make the KV Cache [256]. Task 1 documents do not generate KV cache a priori, because embedding models (which is what my be model is) do not generate it; but they do seem to generate a thin layer of it for the tokens for the question or code sent alongside the documents [137]. Knowing the request size is always very small, I just add 5MB of additional headroom for the KV cache on top of the 157MB. For task 2, there is more KV cache generation. Each task 2 token output (also called decode throughput [154]) is 1 float value or a couple of words worth of tokens (so around 20 tokens max). Each task 2's token input (also called prefill throughput [5]) is a max 4kb JSON file (800 words max, 1,100 tokens max) plus the text request (100 tokens max). The model for task 2 is Qwen3 4B. It requires 0.1737GB of KV cache per 1,000 tokens (and this seems to encompass both input and output tokens) [361]. Since one task 2 process produces a total of 1220 additional tokens, which is roughly 1.22×1,000 tokens, it requires an additional 0.1737GB×1.22=211.914MB. Since a million inhabitants corresponds to 1.884432 concurrent task 2 processes, task 2 processes require an additional 1.884432×211.914MB plus a 30%

10 This is estimated based on comparison to sizes of JSON documents found online [359].

headroom = 399.338MB of VRAM for the KV cache for a million inhabitants. So **the total KV cache of the ai agent for a million inhabitants is** 798.675046MB + 5MB= roughly **804MB**.

Each of my models is responsible for one of the two tasks. Thus, I do not duplicate the context size I mentioned so far. Also, Models ideally load all their context onto the V-RAM [250]. So **the needed task 1+2 context's GPU VRAM is 169MB + 10.0534kB + 804MB = 973.1MB per million inhabitants**.

To that, we add the other things the models must load into GPU VRAM, namely their parameters [250]. In my case, FastAPI would use multiple uvicorn workers due to the high volume of queries. FastAPI documentation seems to say that it loads an AI model once per uvicorn worker that uses it [362]. But it seems to assume that the AI model is used within FastAPI, while my AI agent will exist outside FastAPI, like an external 'service' the uvicorn workers reach out to, and it will ensure that multiple LLM requests can execute simultaneously by using the vLLM engine [127, 363]. So that would mean there is no per-FastAPI-worker model loading. Therefore, I assume the model's parameters are loaded only once per deployment, not once per uvicorn worker. Since ideally the model loads this fully on the GPU [85], and since we should count 30% of headroom for the system to run smoothly, using the parameters' size evoked earlier, the models thus need (16GB+2.27GB)+(1/3)× (16GB+2.27GB) **= 24.36GB of GPU VRAM for loading models' parameters**.

Thus, **the US deployment requires** 24.36GB + 973.1MB × million inhab. = 24.36GB + 973.1MB ×350= **364.945GB of VRAM**.

AI agent's CPU and GPU usage

Both AI tasks (task 1 with BGE and task 2 with Qwen I discussed so far) consume some CPU and GPU. Let's list all the factors which weigh in determining the CPU usage, which I directly calculate for the US deployment.

Each AI request by the FastAPI app requests one of the 2 tasks (never both). My AI agent requires 364GB of VRAM (24.36GB for the parameters + 973.1MB of "task 1 + task 2" compound VRAM per million inhabitants at a time ×350 (350 being the million inhabitants for that deployment). The AI agent expects 188.4432×350=65,955.12 Task 2 AI requests/second for that deployment, 28 Task 1 ai request/seconds for that deployment (10/125s/million inhab, divided by 125, ×350). The AI agent contains a Qwen3 4B model (handling task 2) and a BGE-m3 2.27GB model (handling task 1). Each task 1 request triggers one task 1 response. Each task two request triggers one task 2 response. The task 2 request is some code words saying "find this info in documents with doc id= x, etc.", joined to the given file section to parse. Each Task 1 request is some code words saying "extract info from this/these documents as a JSON with that structure", plus the documents. Note that I mention request texts as such because this is usually how they exist: the FastAPI calls an AI API endpoint, which runs the text with the specific values of the request in its placeholders. But all that to say that, if these texts

are the input tokens material, then it's very few input tokens per request. Each task 2's token output is 1 float value or a couple of words worth of tokens (max 20 tokens). Each task 2's token input is a max 4kb JSON file (800 words max, 1,100 tokens max), fetched by the FastAPI app to the AI model, plus the request text (max 100 tokens). Token input per task 1 request is max 700,000 tokens, same for token output (input of 10MB of PDF , output of 2mb JSON worth of text [77]). Ideally, a task 1 response takes a max of 2min, and a task 2 response takes a max of 10ms. Expected total concurrency in the US deploymentequals the number of concurrent task 1 processes at a time per million inhab×millions inhab + the number of concurrent task 2 at a time per million inhab×millions inhab = 3,500 concurrent task 1 processes (10 new per 125 seconds per million inhabitants, each lasting 120 s at most, so roughly 10 concurrent ones, times the 350 millions inhabitants) and 659.5512 concurrent task 2 processes (since we expect 1.884432 concurrent task two processes per mill. inhab. ×350).

US deployment's bge-M3 model (the one doing task 1 processes) deals with (input+output tokens included) max 700,000 tokens × 2 × 10 new jobs per 125sec per million inhab. ×350 (which is the million inhabitants) /125seconds = 39,200,000 tokens /seconds (half that of inputs, half that of outputs). US deployment total Task 2 processes expects (dealt entirely by the qwen3 model) 1,220 tokens × 65,955.12 new jobs/second = 80,465,246.4 tokens/second (1,200× 65,955.12= 79,146,144 input tokens/s, and 20× 65,955.12= 1,319,102.4 output tokens/s.).

Input token processing capacity is called prefill throughput, and output token production capacity is called decode throughput, both measured in tokens/s [5]. I'm looking at benchmarks to estimate GPU usage. Each model behaves differently. So I'm not sure using another model's benchmarks is insightful. Also, I did not find any source saying that the speed at which a single process completes, when considered in isolation (imagine we have a machine with just one task 1 running), influences the total GPU compute work for that step. Thus, a tokens/s benchmark always has the same relative value, all else being equal (except the required GPU workload). That said, let's look at benchmarks of the models I found.

For Qwen3 4B, I provide (for that US deployment) enough VRAM for a total KV cache of 659.5512 concurrent task 2 processes of 1220 tokens, so 1,609,304.928 tokens. In the online benchmark on an Nvidia H100 GPU (could not find benchmarks on more recent GPUs like the Blackwell family), Qwen3 4B's KV cache is 382,096 tokens (my 1,609,304.928-tokens aggregate context is 4.212 times that) [80]. In the benchmark, input tokens are also spread across multiple requests, as evidenced by the number of input tokens decreasing when the number of concurrent requests increases: the tables indicate 11,939 input tokens per request with 32 requests, 5,969 per request with 64 requests, etc [80]. Thus, the total input tokens is 11,939 × 32 = 382,048. Note, these input tokens are not /s rates. Thus, it probably implies that they were sent to the model all at once. And that implies that the /s throughput was the max capacity of this H100 GPU. And since they sized the KV cache perfectly for the expected load (382,048 input tokens, and the same size of KV cache configured), and since I configured it as

well to have enough KV cache, I can trust the benchmark gives a realistic KV bottleneck illustration for my case (since there isn't a KV bottleneck in either case).

However, these Qwen3 tests use a 25% input tokens - 75% output tokens split and a 50-50 split in simulations, whereas my model uses a 98.36/1.64 ratio (per request: 1,200 input tokens and 20 output tokens). In the benchmarks, with 32 concurrent requests on an H100 GPU, 2,985 input tokens and 8,955 output tokens get dealt with at a rate of 2,978tok/s, and 5,970 input tokens and 5,970 output tokens get dealt with at a rate of 3,586 tok/s [80]. Because the total tok/s throughput varies with the input/output token ratio, it's reasonable to assume that prefill and decode throughput stress the GPU differently. So we cannot safely apply the total tok/s metric directly to my case. But I can build a linear estimate of total tok/s seconds for a 98.36% prefill throughput based on these 25-75 and 50-50 ratio-based benchmarks. At 98.36% prefill throughput ratio (which is my qwen3 4B expected ratio), one H100 GPU can process together 4,762.1 tok/s for 32 concurrent requests of each 11,939 total (input + output) tokens (representing a total of 382,048 tokens). Since the tok/s almost linearly increase as the tokens are spread over more concurrent requests and linearly vary between the 25-75 and 50-50 ratio tests [80], I can deduce the tok/s per GPU if we adjusted the number of requests so that each request is 1,220 total tokens (but with the same total 382,048 tokens) for my 98.36 input ratio.

These are the benchmark's results [80]:

concurrency	32	64	128	256	512	1,024	2,048
25/75 ratio's total tok/s	2,978	5,267	7,894	11,176	14,962	22,358	23,798
50/50 ratio's total tok/s	3,586	5,872	9,952	16,393	22,648	32,544	32,280

If 382,048 tokens were distributed across 1,220 tokens-per-request, we would have 382,048/1,220 = 313.15 concurrent requests. Between 256 and 512 concurrent requests, neither ratio's tok/s increases by the same percentage (34% and 38%, respectively). Taking that into account and asking AI, I obtained that one H100 GPU can sustain 29,000 total tok/s of prefill-decode throughput when 382,048 tokens are spread over 313.15 requests of 1,220 tokens each, with 98.36% of these tokens being prefill throughput. In other words, **one H100 GPU can sustain 313.15 Qwen3 4B requests of my program per second**. Since the US deployment expects 131.910.24 qwen3 4B requests/s (which is 65,955.12 / 313.15 = 210.61829794 times more than 313.15), **my Qwen3 4B, as part of the US deployment, would require 210.61829794 H100 GPUs**. This metric is derived from the benchmarks, which obviously also run an LLM engine and repo on the GPU. Since any model needs an engine, **this metric likely also includes capacity for my vLLM and LM Studio**. Note that each model needs its own separate vLLM engine [224]. So my next model will also need its own vLLM sizing, but that is

okay because the benchmark used focuses on a single model, so there is always its own inference engine calculated based on its capacity.

How much CPU to pair to these GPUs? Usually racks for AI activities already include some CPU capacity. In my case, the AI agent in production is not training; it only performs inference [364]. Nvidia recommends a minimum of 6 CPU cores (of either NVIDIA Grace, Intel Xeon scalable processor or AMD Turin) for each GPU (GPU, not GPU core) for inference [365, 366]. An Nvidia Grace chip provides 144 cores [367]. Thus, **Qwen3 4B requires** 210.61829794 ×6 / 144**= 9 Nvidia Grace CPUs for this deployment**.

Now let's discuss GPU capacity for the bge-m3 model. Bge-m3 sustains a 90,000tok/s throughput on an Nvidia H100 PCIe GPU [106]. Since, in the US deployment, my bge-m3 expects **39,200,000 tokens /seconds of throughput, which is 39,200,000/90,000=435.56 times one H100's capacity, my US deployment requires 435.56 H100 GPUs of GPU capacity for bge-M3 (and that includes all it needs to run: its own vLLM instance** [224]**, LM Studio).** I apply the same CPU/GPU ratio, thus requiring 435.56×6/144= **19 CPUs.**

Note my choice of AI tasks was right. Task 1 consumes much more GPU, though it doesn't convert the document's content into their in-db variable names. So if task 1 was doing that for all document content, and task 2 only for searched variables, then task 1 would be even heavier and require even more capacity. We can ignore CPU usage because it is low for both tasks.

So in total, my ai agent in the US deployment requires 435.56+210.61829794 = 646.18, if running at 100% capacity. GPUs and CPUs do not need headroom (the GPU notably, because it can run perfectly fine at 100% and because my VRAM needs are way under the VRAM that 647 GPUs provide), so I can simply give it a very small amount of headroom for rare cases: a total of 650 H100 **GPUs**. For CPUs, the agent AI requires 9 +19 = **28 Nvidia Grace CPUs.** I do not need CPU capacity for disk reads and writes because Nvidia GPUs do not need CPU mediation [368].

Needed capacity of the FastAPI container:

The FastAPI container (note when I say FastAPI container I mean the FastAPI web framework structure, and what runs behind it, i.e. Starlette, Uvicorn, and UV [369]) contains my clean architecture code and executes as follows: ASGI server (Uvicorn / Hypercorn), then FastAPI router, then Dependency resolution (Dependency Injection), then Validation and parsing with Pydantic, then Type conversion, then Response serialization, then sending the response [41]). Uvicorn workers execute this work [362]. The FastAPI documentation says that Uvicorn workers are 'processes' and that 'processes consume RAM and CPU' [362]. So this rather confirms that FastAPI itself consumes RAM and CPU. I say 'confirm' because by nature this seemed rather obvious. First, the processes I just described consume CPU. And FastAPI does not send any hard storage; the info on the async processes runs in the async event loops, where uvicorn workers' coroutines queue, so these loops and

processes have a live presence in memory [281]. So I must calculate the CPU and RAM usage of the FastAPI container. I also need to gauge the system workspace and disk storage required to install FastAPI and store the installation, code, and structure of the current FastAPI project.

Note that a standby instance of FastAPI is also needed; it will need the same capacity as the primary instance when the primary instance is in serving mode, and will only need some storage capacity when it is on standby (for storing the FastAPI installation).

Number of FastAPI instances per deployment:

There is no max number of workers per FastAPI container instance [370]. Thus, I can set up a single FastAPI container and provide it with everything it needs to handle the expected workload.

Disk storage:

There are multiple smaller FastAPI versions than the exhaustive one, but while they make for smaller installs, each loses some functionality that seems useful in my case [165]. So I stick to the classic install, which, without any project but with Starlette, Pydantic, and UV, is only a few MBs (as I observe when creating a FastAPI project via PyCharm). And the code and files I build for the project are a couple of MB as well, based on examples of a FastAPI project with nearly 88,000 lines of code and thousands of files, which weighs 3 MB [99, p. 79]. Now, note that the FastAPI installation, especially when packaged as a container image, can easily reach 1+ GB [165]. So I plan at least **4GB of disk storage for an entire FastAPI c4 component**. And since I only need one instance of that component, that's all the disk storage space I will need for the US deployment.

FastAPI container's CPU

Here I use an approximate value because the only correct metric would be the one of my eventual code up and running. And this is fine because, as mentioned in the introduction, this deployment crafting is best taken as a practice exercise, and dummy values are provided when lacking to complete the entire exercise.

My FastAPI container expects 8.5656×350= 2997.96 HTTP requests/s for the US deployment, and this likely implies 2997.96×5×0.04= around 600 of concurrency because we estimate roughly that the HTTP request-response latency would be around 5s[11] max on average, and 96% of the latency is on the components external to the FastAPI container (DB, AI agent, Stripe, OAuth). According to the last two TechEmpower benchmarks, one FastAPI container instance, when the code is well written and the surrounding architecture is well built, can process around 65,723 single-row select-then-json-serialisation requests per second at 512 concurrency, on a single Xeon 6330 box with 28 cores (56 threads) [34, 143, 200, 371]. This could go down a little when coming from 600 concurrent users: I

11 It is usually less, but the minority requests which lead to a task 1 ai agent request can take a minute or more to complete a task 1, thus raising the average latency.

estimate 60,000 of such requests. I take the smaller number of the two, which is the number from the 256 concurrency, as it inflates my estimation a little, and I prefer oversizing. Now my FastAPI processes triggered from a single HTTP request are expected to conduct around 250 times the equivalent of a single-row select-then-json-serialisation request (I base this on the 30 use case loops per HTTP request mentioned earlier, with 39.6 transaction loops and the 150 read loops per HTTP request, plus the rest of the activity the FastAPI will perform besides creating this transaction commits, and since the benchmark metric includes DB latency time, I figured the added complexity of my project would make for my FastAPI's CPU time would include also that DB latency time this benchmark included). Thus, in one second, for the same setup, my FastAPI container instance would deal with 60,000 / 250= around 240 HTTP requests per second. So my app's traffic is 2997.96 / 240= around 12.5 times what this Xeon processor can process; the US deployment requires a 28 × 12.5= around 350 cores + 30% of headroom = around 455 (or above) cores processor for performance/core equivalent to the Xeon 6330. Therefore, to be large, **I choose 2 Xeon 6980E+ processors, each with 264 cores** [372]**, to run the FastAPI container for the US deployment**.

FastAPI container's RAM:

I use an approximate value, for the same reasons as for the CPU usage.

From my understanding, the RAM FastAPI uses is dedicated to storing the live info of the Python application, the runtime, the currently active connections/tasks and their state/data (this last part notably because of the aync/io system, which makes the FastAPI layer await certain external actors to return something in order to resume a given uvicorn process).

I know my FastAPI instance handles around 3,000 requests per second in this US deployment, and the program takes up to 5s to return an answer. There's up to 10MB per HTTP request, so task 1 AI request's worth of user input is held in case there are retries (these 10MB are the weight of the documents the AI agent receives and needs for task 1). And there are a maximum of 3500 concurrent task 1 AI pending responses at a time in this deployment (see earlier in this ADR). Therefore, for task 1-bound HTTP requests, and assuming state info for each awaiting FastAPI co-routine takes 10kB of RAM, the FastAPI container requires 3500×10MB + 3500×10kB = 35.035GB of AI task 1-related RAM. Also, there are as many as 659.5512 concurrent task 2 processes in this deployment, for which FastAPI must hold state info (at most a few kB). So the FastAPI container requires 659.5512× 10 kB = roughly 7MB of AI task 2-related RAM. There is at most one wait at a time per HTTP request on the other components (DB + oauth + stripe), with as much coroutine info held in RAM (10 kB each), so that's an additional 3,000 × 10 kB = 30 MB of RAM needed. If we add as much RAM as the baseline system uses for FastAPI processes and some overhead, this deployment requires (35.035GB + 7 MB + 30 MB)×2 = **71 GB of RAM**.

Needed capacity for the Docker overhead:

While I calculated the capacity of what will be found within the Docker containers, running Docker and the container structure also requires some capacity planning, because the container infrastructure and the Docker engine require capacity. I did not find a definite capacity requirement, so I give it per instance 5GB of disk storage, 1GBps of disk bandwidth, an AMD EPYC 9016 of 8 cores [357], 5GB of RAM.

Needed capacity of the machine and OS runtime

Since I already sized all the components with 30% headroom for each, and since the sizing was based on benchmarks that all run on the same system, the capacity listed in the previous subsections should be all the program needs.

Solutions retained based on capacity requirements:

Based on peak capacity estimates we just read, we know that the US deployment has capacity requirements equivalent to (this means the given products I choose can be different but their capacity must be equal to) the following:

- The primary instance of the DB requires 14TB of RAM and 1 AMD EPYC 9756 CPU and 76TB of disk storage with 31GBps of write throughput, 18.4884MB/s of read throughput.
- The standby instance of the DB, both in serving mode and in standby mode, requires just as much capacity as the primary DB, with the only difference being that, for CPU usage, the standby DB in standby mode only needs the equivalent of an AMD EPYC 9116 CPU.
- Each DB must have its own hardware.
- Each of the 3 backup storages (one co-local with the primary DB, one co-local with the standby DB, one disaster backup storage in another data centre) requires 230TB of disk space, and 31GBps of storage bandwidth.
- the primary instance of the AI agent in this deployment requires 364GB of GPU VRAM, 650 H100 GPUs of GPU usage (that total amount of gpus contains that needed vram by far), 28 Nvidia Grace CPUs, 13.8TB of data disk storage space, 263.82048MB/s of disk read throughput and 700MB/s of disk write throughput. And I want to keep this GPU model because it avoids CPU mediation for disk reads/writes (thus reducing my CPU needs).
- The standby AI agent requires as much capacity when in serving mode; otherwise, it requires just the space to store installations and parameters (23.751 GB + 5GB = roughly 29 GB)
- The primary instance of the FastAPI container requires 2 Xeon 6980E+ processors, 71 GB of RAM and 4GB of disk storage.
- The standby FastAPI container when in serving mode requires as much as the primary instance; otherwise, it requires only the 4GB of disk storage.

- The Docker overhead primary instance and the standby instance both require 5 GB of disk storage, 1GBps of disk bandwidth, the equivalent of an AMD EPYC 9016 of 8 cores, and 5GB of RAM.
- KeepAlived is very thin, so negligible.
- Since I sized all the components with 30% headroom for each, and since the sizing was made based on benchmarks which run on a given system eventually, the capacity listed in the 3 previous bullet points should be all I need.

Thus, the biggest unit of my architecture (biggest in processing time, and more important than storage since storage is extensible more easily), i.e., the primary program instance instance, requires in total the equivalent of: a little more than 14TB of RAM, 650 H100 GPUs, 1 AMD EPYC 9756 CPU, 28 Nvidia Grace CPUs, a little less than 320TB of disk storage. Which of the so-far-retained solutions can support that capacity and how?

As evidenced later in the price analysis subsection, the program instance, when running directly on bare metal or on virtual machines, requires capacity spread across multiple chassis. As such, it does not eliminate any of the hosting options retained so far, because whether as one supercomputer or as multiple network-connected computers, each can provision the needed capacity (I give some examples later). Let's discuss the next determining selection criterion.

Solutions retained based on the level of control I need

If the AI agent is a SaaS service I pay to use, I lose control over the location of the hardware running the models and the data my FastAPI sends to it, since the inference provider chooses where its capacity is allocated. Since I want to be sure my data always remains on national soil, and I do not want to have to 1) constantly verify where the providers have their capacity, nor 2) reconfigure in an emergency in case the providers switch to an abroad-based capacity, I avoid this solution. Avoiding SaaS for the AI agent also means it can be deployed locally alongside the other elements, which is valuable in my case, as explained in the next subsection.

Also, Decentralized/peer-to-peer compute marketplaces (DePIN), such as vast.AI and Akash Network, is risky because it has no SLA layer, is often not compliant with my needed compliance requirements, and is likely more costly because it functions based on bids for hardware usage, and thus makes hardware usage for my app uncertain based on potential colocators' bids [44, 129, 160, 373]. And the same uncertainty applies to spot cloud (access to cloud resources as long as a more privileged customer 'steals' the capacity). So I eliminate these resource-access solutions from my list.

Also, I could choose either private or public cloud prices, but private cloud requires a lot of set-up and managing work while public cloud is managed and maintained and updated by the provider (e.g.

AWS) [374], and high security. Therefore, I rule out private Cloud and retain public cloud for control, convenience, and security.

Finally, I read some articles about Cloud lock-in, i.e, the idea that it is often difficult or costly to switch cloud providers. In my case, the migration aspect is not impossible; the hard part is migrating the DB data, because it is large and the data keeps changing, while the backup can write to my new target cloud deployment. Now the real issue is the payback of switching. The only thing I actually must migrate as such is the DB data, and that's around 340-350 TB, and that costs some thousands to 20 thousand dollars for the technical migration itself. But if you count 50 thousand dollars or more for the labour cost of the operation (because this requires some people to supervise and plan), then payback is only profitable after some years, and that's if the previous provider did not lower its pricing in the meantime. But I consider this lock-in is not an eliminatory argument, because bare metal also has same-effect lock-ins: if I want to buy components which consume less electricity, or switch to more performant c4 components, that switch implies similar costs (in bare metal element or/and in labour costs for the migration supervision).

Solutions retained based on processing speed and performance

My system needs to be very fast because users complete 200+ requests per day; long waiting times directly impact customer satisfaction and retention. The underlying capacity is the same across hosting options, but the HTTP request-response latency varies because each option has different network latency costs. A network latency difference between one hosting config and another ranges around 1 microsecond per stream all other things being equal, plus the bandwidth difference: for example, if communication used to go through X cables, and now it goes though X other cables, each less performant cable has a 1 microsecond higher stream cost, and that's X more microseconds of latency, all other things being equal, and we also need to take into account bandwidth limitations. So do network latency differences between deployment options impact my program?

I calculated the required capacity based solely on the processing requirements. Therefore, the capacity I calculated is sufficient: adding more will not make a deployment with higher I/O latencies faster. My FastAPI container, the primary instance of the DB and KeepAlived fit on one machine. That's optimal for communication between these components, and it's important to choose so, because the number of communications between them is gigantic (see capacity estimates). Another machine can host the backup of the DB's primary instance, and with a 400Gb/s wired Ethernet link which provides 50GB/s of network bandwidth, there is enough network bandwidth to sustain the write load. Network fabric is stuitable too [75]. Non-wired communication to the backup DB is not really suitable (cause not as effective or as reliable), and laser transmission (FSO) is overkill for two servers on the same rack (more technology to deploy, more environmental constraints). Wired communications do not require servers to be adjacent, as data centres can wire-link two servers even when in opposite corners of the

same facility [9]. Communication between Cloud VMs of the main providers can have similar network bandwidth, but it requires multiple parallel flows that are guaranteed to be provisioned at all times, which is a guarantee the providers do not give, since bandwidth can be lower when the network gets busy [375, 376]. Because my backups are asynchronous periodic backups, some small additional latency is fine. But I would then need to give the primary DB machine more storage so it does not overflow in ARClogs when sending backup data to the backup gets slower. Same logic applies to communications between the primary DB/the standby DB's backup, and to the disaster backup (which is on another machine in another data centre).

However, the AI agent cannot live on one single machine, whether virtual or not. I can distribute AI requests among these AI agent machines via a message broker, provided each machine has the AI models and engine installed; each engine instance then serves as one inference endpoint. Or it can send all requests to one AI agent machine, which then delegates to other AI agent machines. FastAPI does not need to know which AI agent machine processes the given AI request. And the requests are independent, as long as each machine receives the document to parse, coming from a common document storage or a per-machine copy of these documents. Thus, a message broker can dispatch requests to multiple inference endpoints (I discuss later which message broker is best).

The question is then, where do I store documents, and who is in charge of loading them (either in the request or the model contexts directly), and where does the broker sit in the stream flow? In either case, an app (FastAPI or other) must load documents in the request or in the inference engine, because the broker nor the AI inference engine can do it. Thus, here are the options:

- requests can be shipped from fastAPI already with the document/JSON section (here the documents live in the FastAPI's direct storage), hit a broker, the hit the available inference endpoint;
- requests can be shipped without the document/JSON section, hit an app in charge of loading them in the request (the documents are stored with that app), then hit a broker, then hit the inference endpoint available;
- requests can be shipped without the document/JSON section, hit a broker, hit a per-machine loading app instance (the documents are stored on each machine by duplication, or on a common storage), then hit the inference endpoint on that machine.

I choose the first option. That avoids installing some app or multiple app instances outside the FastAPI container. Adding a single outside-fastAPI app induces routing existing already with my chosen option, but between the outside app and its document storage (instead of between fastAPI and its storage in my chosen option). Adding one app instance per machine is a lot of extra work and also induces equivalent routing, whether documetns are duplicated on each machine or stored in a single storage. I expect 65,955.12 Task 2 AI requests/second for that deployment, 28 Task 1 ai

request/seconds. Task 1 is the one with 10MB doc, task 2 loads 4kb json section. Per second, adding the very thin request text as well, and counting the response messages too, the broker in my chosen option receives in total 65,955 task 2 req/sec × 4 KB + 28 task 1 req/s × 10 MB + 28 task 1 resp/s × 2MB + 65,955 task 2 resp/s × 100b= 606.5MB/s of throughput spread across near 132,000 messages. Per message, the per-message payload does not exceed most brokers' capabilities, since the biggest payload size is around 10MB per message (since task 1 comes with a 10MB document) and the brokers usually accept up to largely over 10MB per message [377]. An the total payload is also completely achievable. Shipping documents in the requests from FastAPI through the broker has memory and bandwidth weight. But knowing the bigger documents (the 10MB one and the 2MB extract of it) are shipped only once, the bandwidth weight is the same for shipping these documents in another storage than fastAPI's direct storage if we were placing that storage with a document loading app living on the side of the broker opposite to FastAPI's. The memory weight would also be the same : either by the document loading app on each machine, or on a single document loading app (and if that second option is chosen, then there is also memory costs in the broker which would need to exist between the document loading app and the AI mahcines). Therefore, the overall operational costs is the same in either option, or bigger in the two last ones, but the first option induces less configuration than the two others. That's why I choose the first option. This entices the 13.8TB of disk storage needed for te AI agents will thus live with the FastAPI container.

Finally, note the backup instance can work a little slower than the primary instance, since its use is exceptional.

Solutions retained based on the level of isolation I want, and containerisation

With each added layer of containerisation or virtualisation, latency time increases [234]. But adding one layer does not suddenly make the program slow or unprofitable, and it can bring many advantages.

I choose to use containerisation because it keeps the dependencies required to run a given component with that component. This eases debugging, migrations, and duplication (the latter, notably, for developing new versions and integrating them smoothly into the main workflow), and there is no size/capacity limit, because what is containerised is not he CPU/RAM/disks torage used by a given element, but the eleement's environment and dependencies [378].

But using containers does not mean the running program is always fully portable. In some configurations and with certain container providers, data populating the DB after the container image's installation (so in other words, the runtime data from user sessions) might be stored straight to the machine in a way which would require the OS to be migrated too in case I want to migrate my

program to another server. Luckily, with my chosen containerisation solution, i.e., Docker (see next subsection), the data of the DB tables can be persisted in a Docker volume, which will not be lost or erased when the container restarts or is deleted (unlike when the data is written in the writable layer of the container) [379]. Moreover, volumes are fully managed by Docker and can thus be easily migrated without having to ship them with the machine's OS [158]. And in my case, that actually makes VMs unnecessary for migration purposes because I expected unfrequent migrations, and reinstalling elements missing on the machine (the OS and the docker engine) on the migration destination plus migrating my few (4 to 5 for the primary program instance instance, same for the standby) docker containers and volumes is very quick, so not that time expensive over time for unfrequent migrations, while avoiding the small latency and infrastructure costs of adding a VM layer. And also, **"**Virtual machines can take up a lot of storage space. They can quickly grow to several gigabytes in size. This can lead to disk space shortage issues on the virtual machine's host machine" [24]. Containers do not cause these sizing issues or significantly increase capacity costs because they are thin forms of isolation. So unless the cheapest solution involves virtualisation by default (e.g., using the Cloud), I do not use a VM.

Now let's eliminate the remaining options based on price.

Solutions retained based on price:

Deployment prices comprise prices for deploying the capacity itself, and prices of necessary memberships/usage fees that one must pay depending on the deployment option. In my case, services I pay are Stripe, and Oracle Database (OAuth2.0 protocol is free to use, and as seen in the ADR on security decisions at system level, I do not pay to any provider for the implementation of the identity management and security) [228, 380]. Stripe fees do not vary by deployment option, as fees are indexed on user payments [380]. But Oracle prices do vary, as dicussed later. Therefore, to determine the cheapest option out of the remaining ones, I first discuss prices of capacity aside from membership and Oracle fees, then I add these Oracle DB purchasing fees parameters into the equation.

Based on peak capacity estimates, and adding a message broker for communication between FastAPI and AI agent instances, the US deployment has capacity requirements equivalent to (this means the given products I choose can be different but their capacity must be equal to) the following:

- The primary instance of the DB requires 14TB of RAM and 1 AMD EPYC 9756 CPU and 76TB of disk storage with 31GBps of write throughput, 18.4884MB/s of read throughput.
- The standby instance of the DB, both in serving mode mode and in dormant mode, requires just as much capacity as the primary DB, for the only difference that for CPU usage, the standby DB on standby mode only needs the equivalent of a AMD EPYC 9116 CPU.
- Each DB must have its own hardware.

- Each of the 3 backup storages (one co-local with the primary DB, one co-local with the standby DB, one disaster backup storage in another data centre) requires 230TB of disk space, and 31GBps of storage bandwidth.
- The primary instance of the AI agent in this deployment requires 364GB of GPU VRAM, 650 H100 GPUs of GPU usage, 28 Nvidia Grace CPUs, 13.8TB of data disk storage space (but this lives with FastAPI except for the 29GB of AI agent install size), 263.82048MB/s of disk read throughput and 700MB/s of disk write throughput. Note: I want to keep this GPU model because it allows disk reads/writes to not need CPU mediation (thus reducing my CPU needs).
- The standby AI agent requires as much capacity when in serving mode; otherwise, it requires just the space to store installations and parameters (23.751 GB + 5GB = roughly 29 GB per machine)
- The primary FastAPI container requires 2 Xeon 6980E+ processors, and 71 GB of RAM and 4GB of disk storage.
- The standby FastAPI container requires as much as the primary one when running; otherwise, it requires only the 4GB of disk storage.
- The fastAPI-AI agent message broker requires 606.5MB/s of network interface capacity, 16 CPU cores, some GBs of disk storage for storing the broker install, and 48GB of RAM for the retries info and broker overhead (65,955 task 2 req/sec × 4 KB × 10 seconds of depth for task execution and response arrival to the broker + 28 task 1 req/s × 10 MB × 121 seconds of depth for task execution and response arrival to the broke + 28 task 1 resp/s × 2MB × 1 second for response processing by FastAPI + 65,955 task 2 resp/s × 100b × 1 second for response processing by FastAPI + 30% of all this for broker overhead = 48GB of RAM, and I do not add some longer queueing capacity, because if all that comes within a second failst).
- The Docker overhead primary instance and the standby instance both require 5 GB of disk storage, 1 GBps of disk bandwidth, the equivalent of an AMD EPYC 9016 of 8 cores, and 5GB of RAM.
- Since I sized all the components with 30% headroom for each, and since the sizing was made based on benchmarks which run on a given system eventually, the capacity listed in the 3 previous bullet points should be all I need.
- There are Cloudflare costs, but they are steady no matter the deployment solution. Therefore, although they influence service profitability, I do not consider them as part of my deployment solution selection.
- And we remember the physical storage behind the deployment, whether based on cloud or not, must be in the US (which disqualifies all resource providers whose capacity are abroad).

While the backup DB storage must always be running because it receives constant data from the in serving mode DB (since it copies massive amounts at an hour interval), the other elements (standby ai agent, standby FastAPI, standby db) need to run (and thus need capacity) only when the in serving

mode program cannot run anymore. This makes the backup program components, besides the standby DB, very eligible for an in-cloud deployment because the cloud can provide dynamic capacity-based deployments which imply high fees only when the primary instance fails. Let's first compare prices per element by deployment option, and then I will compare overall solutions based on requirements such as logical location, together with price.

For the solutions where I own hardware (the first two in the following table), I must assume a hardware renewal rate of 5 years (10 years for the racks). Thus, the annual price of a hardware component is its purchase price divided by 5. Also, I pay maintenance for anything I own. Elements are facing near zero traffic outside the 8-hour workday period, except for the DB, which runs incremental backups at a fixed night hour. I also assume at least 3 days per month (36 days per year) of activity in which the standby components are in serving mode, but that is not 36 days of usage less for the primary instance, because while the primary instance is down, it will undergo internal/admin activity for troubleshooting purposes. Finally, as noted earlier, DB backups can be on HDDs as long as the required write bandwidth does not cost more than the equivalent capacity on SSDs.

Note that for Cloud pricing, I use the prices of on-demand and local hardware solutions, since these are requirements (see earlier).

Also, in the subsequent tables I ignore on-prem building space rental prices, and add this parameter afterwards.

Lastly, these components and CPUs and GPUs are indicative examples. I can use different models, but I base prices on the technologies outlined in the capacity estimates, as this gives us clear evidence of what the set-up could cost.

Tables: cost comparisons of hosting solutions per system component (before Oracle solution purchase costs):

	Primary DB
Price on-premise/y	CPU: $15,000/5= $3,000. RAM: max $100, 14TB costs max $500, and we divide by 5. SSD storage: 10 Dell 7.68TB PCIe NVMe RI 2.5" SSD Drives provide the needed capacity and cost a total of 10 × $8,000/5=$16,000/y [381, 382]. A specialised 1U or 2U rackmount server chassis with direct-attached PCIe Gen 4 or Gen 5 NVMe backplanes and robust multi-lane support costs $3,000 to $5,000 (so $800/y) [145, 383]. $25,000 of employee work, external maintenance and energy needs for the on-prem infrastructure per year. A thousand dollars for wiring. 20,000 for security procedures.

	We can add 10,000 dollars for facility rent or purchase price per yet Total: $75,900/y, ignoring building rental prices.
Price on my hardware in rented rack space/y	The same prices for the components since I own them, plus $100/month for housing [384, 385] (likely more for security purposes, and since I probably need 2U, let's count $700/month), and there is probably still $3,000 of component maintenance per year. No cable surplus because the data centre provides it. 20,000 for security procedures. So that's a total of 3,000+16,000+800+700×12+3,000= $51,200/y. **This is the cheapest option.**
Price on rented hardware and rented rack space/y	I can imagine it costs a bit more than the above option, since renting the components is expected to cost the same as or a little more than owning them, because rental providers must be profitable and incur the same purchase and maintenance fees. However, renters buy items in larger quantities, so they get larger discounts. So I still checked to be sure. However, it seems that the CPU plus a fraction of the RAM (between 1 and 3 TB, out of the 14 TB needed) and storage I need is already as expensive as the housing solution (around $4,000/month ×12 months) [386].
Price on Cloud/y	An AWS U7i-12TB solution costs $125.58/h and provides all my capacity except 2 missing TB of RAM [387]. Capacity of the DB is rather down on weekend days, and outside work hours users do not much solicit the service, but the DB has off-workhours activity due to backups, and we can count within week hours an hour or 2 for the thin weekend incremental backups. So we can count around 16 hours of activity per workday. This solution would then cost $125.58/h ×16 hours ×261 days also compensates for the ignored thin incremental backups of the weekend days) = around $524,000/y. And that's without the missing 2 TB of RAM.

	Standby DB (considering 23 days/y of in serving mode activity)
Price on-premise/y	$75,900/y, because downtimes are mostly unpredictable, thus requiring always to dispose of the "as if in serving mode" capacity. This ignores building rental prices.
Price on my hardware in rented rack space/y	$51,200/y., for the same reason. **This is the cheapest option.**
Price on rented hardware and rented rack space/y	Same as for the primary DB, so costing over the two above.

Price on Cloud/y	Unlike with the primary DB, CPU usage is much lower when the db instance is on standby mode. But RAM statys as high as for the primary db instance, and that drives the cloud prices to still be so costing over the two first options.

	the 3 DB backup storages
Price on-premise/y	We need (230TB of disk space, and 31GBps of storage bandwidth)×3. 10 Exos M HDDs of 24TB and 6GB/s per backup provide exactly what I need, and 10 together cost a total of 10× $1,100.22 [388] / 5 years of use = around $2,200/y. To that, we add at least $10,000 in maintenance and energy costs per backup per year: only two are on-prem, since the disaster backup is in a different dedicated location, but I count for its maintenance too as 10,000. Also, the rack install medium for each backup, i.e., a JBOD enclosure with 3.5-inch bays [389], costs max $100/y each. But then we add quite some price for security of the data (could easily rise to 60 thousand dollars in total). We can add 10,000 dollars for building space renting since it requires 3 locations but of small size. Total: 10,000×3 + 2,200×3 + 100×3 + 50,000 + 10,000 = $ 106,900/y,
Price on my hardware in rented rack space/y	$2,200/y + $100/y+ $700/month for 2U rack space/month (since this is all supposedly needing a 2U rack space), and there is probably still $3,000 of component maintenance per year per backup. I can add $20,000 each per year for additional security procedures. Total: ($2,200 + $100+ $700×12 + $3,000+ 20,000)×3= $101,100/y. **This is the cheapest option.**
Price on rented hardware and rented rack space/y	Rental prices are $29.96/month for each of my 10 drives [390]. That's 29.96×12×10×3 backups= $10,785.6/y. To that we add $700×12 for the rack space renting per backup, and $117.66/mo per JBOD [391]. And still we must add 2,000 of maintenance per year per backup. I can add $20,000 each per year for additional security procedures. Total: $10,785.6 + ($700×12 + $117.66+ 2,000+20,000)×3= $102,339/y.
Price on Cloud/y	With pricing around $0.1/GB/month [392] (and since we have 230TB per DB backup, which is 235,520×1GB), so the total would be $0.1×235,520×12 months × 3= $847,872/y.

	Primary AI agent instance
Price on-premise/y	With 650 H100 GPUs for all my AI work, knowing each H100 has 80GB of VRAM [187], my AI rack provides 80GB×650= 52TB of VRAM, : largely enough for my 364GB of total VRAM needed in that deployment. Each GPU costs around $25,000 [83]. So that makes for a total of $25,000×650/5 years= $3.25M/y. Each Nvidia Grace CPU superchip costs around $83,500 [393], and I need 28: that's 28×83500/5y=$467,600/y. I can mount 8 GPUs per Supermicro HGX100 SYS -421GE-TNHR2-LCC chassis (which I read is good because it's 4U and easier to rack than 6 or 8U ones, and provides better cooling [212]). Therefore, I would need 650/8= 82 of these. They each cost around $29,000 [394]. So that's 29,000×82/5= $475,600/y. Each Nvidia Grace superchip is 1U [393]. But it seems I can put 4 of these CPUs in a 2U H263-V60-AAW1chassis [96, p. 5]. Each such chassis costs around $72,000 [395]. So that's a total of 72,000×28 CPUs/4 CPU per chassis /5 years = $100,800/y. Looking at the components' energy consumption and the price of electricity in the US, we add at least 700KwH for the entire AI agent, which costs about 130×12 months= 1,560/y [396]. We can count $20,000 of maintenance for the on prem infrastructure per year, and $20,000 for security. Disk storage: only 29 GB per ai agent instance (since the document storage is with the fastapi container), which is negligible cost. Finally, we can add a million and half + dollars per year to run the infrastructure necessary to such system, which requires water cooling aqnd specific facility charateristics. Total is : 3,250,000 + 475,600 + 100,800 + 1,560 + 20,000×2 + 1.5 million = $5,369,960/y. it could be slightly more due to the renting or purhcaisng the necessary facility and water costs.
Price on my hardware in rented rack space/y	The same components price minus electricity bills plus at least half of the maintenance bill (5,000 then) + rack space renting bill. We know that rack space renting facilities usually charge 196 dollars/kwh/month, and this is is paid even if the activity goes below that on multiple days in the month: it prices on what the rack would usually consume when functioning [397]. My AI agent requires around 700KwH. So that's a total of 700×196×12=$1,646,400/y for rack space renting.

	Putting it all together, we get 3.25M + 475,600 + 100,800 + 5,000 + 1,646,400= $5,477,800/y. I ignored the small additional costs for the HDD storage space. But **this is likely the cheapest option.**
Price on rented hardware and rented rack space/y	We already know the colocation price: 1,646,400/y. Then we have the components' rent prices. I did not find definite prices, but I expect it to be pricier than the previous options.
Price on Cloud/y	For cloud capacity, we must pay between $1.19 and $6.50/h/GH200 [251, 398, 399]. It seems that due to the on-demand needs (I must conserve this capacity over co-consumers of the provider) and dynamic pricing. I would likely pay closer to the 6.50/h/gh200. But a GH200 contains 1 GPU and 1 CPU chip [398]. I need way fewer CPUs than GPUs. So I cannot simply use the GH200 cloud pricing. But I can more or less halve the price tag, and that will account for added security features (since my data is sensitive) and the storage, and use half separately when calculating for the CPUs as well. That's $3.25×8 hours a day×261days×650 GPUs= $4,410,900/y for my capacity requirements. This is already above the previous options. And I have not even accounted for the remaining capacity yet.

	Standby AI Agent (considering 23 days/y of in serving mode activity)
Price on-premise/y	As much as for the primary AI agent instance, since I must own/rent year-round because the primary instance's downtime is mostly unpredictable (except for the one the company plans for certain manipulations).
Price on my hardware in rented rack space/y	As much as for the primary AI agent instance, since I must own/rent year round because down times of the primary instance are mostly unpredictable (besides the one the company plans for certain manipulations).
Price on rented hardware and rented rack space/y	As much as for the primary AI agent instance, since I must own/rent year-round because downtimes of the primary instance are mostly unpredictable (besides the one the company plans for certain manipulations).
Price on	For reminder:

Cloud/y	- During the whole year it needs 29GB of disk storage; - for 36 days each year it needs: 364GB of GPU VRAM, 650 h100 GPUs of GPU usage, 28 Nvidia Grace CPUs, 263.82048MB/s of disk read throughput and 700MB/s of disk write throughput. We remember from earlier, GPUs costs $3.25/h/h100 GPU, and that includes some storage costs and some added security since I derived it from a GH200 price with different storage and CPU count than what I need. For the standby's activity, that's 3.25×8 hours per day ×36×650= $608,400/y (and that includes the ram that the GPUs offer). For CPU, basing myself from the same GH200 basis, the cloud CPU cost for this is is 3.25×8×36×28=26,208/y. And the year-round cost of the 29GB of storage is under 50 dollars [400]. So the total yearly price for a cloud deployment of this component is: 608,400+26,208+50= around $635,000/y. **This is the cheapest option.**

	Primary FastAPI container instance
Price on-premise/y	2 Xeon 6980E+ processors cost 2×$9850 [401] = $19,700, amortised on 5 years that's $4,000/y. 71 GB of RAM costs max $1,000 [18], so that's 1,000/5=200/y. 4GB of disk storage for the install is a couple euros per year max. The documents storage for the AI agent, 13.8TB on SSD costs around $250-2,000 per year. you add 3,000 per year for electricity consumption and maintenance and rack space. And we can add a few thousands for wiring. That's max $12,000/y total at least, ignoring building renting prices. **This is the cheapest but in any case this element would go where the others the primary program instance components are since this is of a small price.**
Price on my hardware in rented rack space/y	Rack space renting facilities usually charge 196 dollars/kwh/month [397]. My setup likely consume 2 kwh max per hour. So I would pay 196×2×12= $4,704/y. If we add my rack prices and some self maintenance, the total is: 4,000+4,000+200+1,000 + 2,000= around $11,000-12,000/y. **This is the cheapest but in any case this element would go where the other primary program instance components are since this is of a small price and for latency benefits.**
Price on rented	I did not find a tangible pricing but I expect the hardware renting to be more

hardware and rented rack space/y	expensive than buying the hardware. **This is not the cheapest but in any case this element would go where the other primary program instance components are since this is of a small price and for latency benefits.**
Price on Cloud/y	The processors would cost me around $1/h. for an activity 8 hours during the days 261 days per year , that's 1×8×261=2,088/y. and 71 GB of RAM would cost me around 0.4/h [326] ×8×261=835/y. Finally 4GB+ 13.8TB of disk storage has a yearly cost of 13,800GB×$0.10/GiB-month×12 months= $16,560, as evidenced with cloud price for storage for the ai agent on standby mode. **This is not the cheapest option, but in any case, the component goes where the other primary program instance components are since this is of a small price and for latency benefits.**

	Standby FastAPI container (considering 23 days/y of in serving mode activity)
Price on-premise/y	As much as for the main FastAPI instance, since I must own/rent year-round because downtimes of the primary instance are mostly unpredictable (besides the one the company plans for certain manipulations). **This is not the cheapest but this would go where the always-running standby DB instance is, for latency benefits.**
Price on my hardware in rented rack space/y	As much as for the main FastAPI instance, since I must own/rent year-round because downtimes of the primary instance are mostly unpredictable (besides the one the company plans for certain manipulations). **This is not the cheapest but this would go where the always-running standby DB instance is, for latency benefits.**
Price on rented hardware and rented rack space/y	As much as for the main FastAPI instance, since I must own/rent year-round because downtimes of the primary instance are mostly unpredictable (besides the one the company plans for certain manipulations). **This is not the cheapest but this would go where the always-running standby DB instance is, for latency benefits.**
Price on Cloud/y	For reminder: - For all year we need : 4GB of disk storage (very neglible yearly cost, as evidenced witht cloud price for storage for the ai agent on standby mode). - For 36 days in the year, we need: + 2 Xeon 6980E+ processor , and 71 GB of RAM and 4GB of disk storage, and 13.8TB of documents storage . The logic is that we need even less capacity than for the primary instance, and since cloud was already the cheapest for the primary instance, and the other options' prices

	are similar in each of the other 3 solutions than for the primary instance's, **cloud deployment of this component is the cheapest solution here again, but in any case, the component goes where the always-running standby DB instance is, for latency benefits.**

I do not calculate prices for Docker nor the message broker, as the Docker is thin, and as the broker, due to its RAM heavy nature, follows a similar pattern to other RAM heavy elements (for primary instance, better hard metal, and for the standby better on Cloud).

Last parameter to take into account: so far, I voluntarily neglected a part of the Oracle DB deployment dimension, which is that purchasing the solution and deploying it are tied concerns from a financial point of view (as I am about to discuss).

I use an Oracle DB Entreprise Edition solution, notably for its security features and its Data Guard feature (as discussed in earlier ADRs). The options to purchase that service, are a Traditional License purchase for on-prem/self-deployment, a *BYOL* (Bring Your Own License) Cloud deployment on Cloud infrastructure, and a *License Included* solution running on Oracle Cloud infrastructure. If I choose a Cloud solution, it would be 'Oracle Base DB Service – Enterprise Edition', because it seems to be specifically dedicated to my needs, especially security across multiple physically distinctly stored db instances, which work together (which is my case since I use Data Guard), and because it runs on Oracle's own Cloud infrastructure, unlike the other Cloud solution for Enterprise Edition, namely Cloud@Customer [11, 402].

Prices for the three purchasing options of that DB Edition are as follows [255, 403]:

Purchase Option	Purchase indexing metric	Enterprise Edition
Traditional License Purchase (only one metric to choose)	Named User Plus (NUP)	$950 / user one time, plus $209 / user/year (the latter being support fees)
	Processor , also called ECPU (what Oracle calculates as such based on equivalence tables [239])	$47,500 / processor, plus $10,450 / processor/year (the latter being support fees)
OCl BYOL (both rows apply together)	ECPU per hour	$0.0484
	Gigabyte storage capacity per month	$0.12
OCl License Included(both rows	ECPU per hour	$0.1075
	Gigabyte storage capacity per	$0.12

apply together)	month	

So how much would each option really cost in my case? Let's start with the Traditinal License purchase option for the primary DB instance and its direct backup storage. An NUP is any human user and device auhtorised to connect and manipulate data in the Oracle DB [70, 255]. In my cae, this number should not be over a hundred. But oracle fixes the number of NUP as minimum 25 per processor [205, p.42]. The number of processors is an Oracle-defined metric, calculated as the actual number of cores of the given processor model used times an oracle factor [239]. Oracle did not yet define how much Oracle processor equivalents AMD 9006 series (the one I used to measure the DB's needed capacity) equals in Oracle processor terms. The previous AMD generations however had all an 0.5 rate factor per CPU for this equivalence calculation [239]. The fact is that I chose the most recent processor capacity example to index my DB's capacity on, but an equivalent 9005 series processor (in cores number) could probably do the job too, and the 128 cores 9005 and 9006 series processors have close characteristics [404, 405]. So I keep this 0.5 per CPU rate factor for calculating the Oracle processor Licenses number. Thus, we have 128 cores × 0.5 = 64 Processor Licenses for the DB primary instance (same for the standby instance when in serving mode). Thus, the primary DB instance's pricing is based on 64 × 25 NUP per processor = 1,600 NUPs. With the NPU indexing, the primary DB instance as a Traditional License purchase would then cost $950 × 1,600 = $1,520,000 one time, plus $209 ×1,600 = $334,400 /year. with the processor indexing, the primary DB instance as a Traditional License purchase would then cost $47,500 × 64 = $3,040,000 one time, plus $10,450 ×64 = $668,800/year. This shows purchasing based on NUP count is the option to choose if I opt for a Traditional Licensing purchase. But this would just be the purchasing cost, not the deployment cost of running the DB instance nor the deployment cost of running the direct backup storage.

The OCl option with BYOL requires paying the License of $1,520,000 (since I do not yet possess one) and the same yearly support fees of $334,400/y, plus at least $0.0484 per ECPU per hour, and since I have 64 ECPUs, and 261 × 16 hours (see hour count mentioned earlier in the price calculation section of the ADR), and knowing the primary DB instance requires 76,000GB of inner storage plus 230,000 GB of direct backup storage space this purchasing option would cost $0.0484 × 64 × 261 × 16 + 334,400 + $0.12 per GB of storage per month × 12 months × (230,000 + 76,000) = $787,976 per year, plus an upfront cost of $1,520,000 , but these $787,975 per year include purchasing and deployment expenses.

The License Included purchase + deployment would cost $0.1075 × 64 × 261 × 16 + 334,400 + $0.12 per GB of storage per month × 12 months × (230,000 + 76,000) = $803,771/y.

Now let's weigh these DB options' prices against the prices yet calculated earlier. The cheapest eligible deployment of the primary DB instance, namely on my hardware in rented rack space, costs

around $51,200/y, and the direct backup co-locally deployment costs around 33k per year (around 100k for the three backups divied by three). Added to the yearly Oracle fees for a Traditonal License purchase of $334,400/y (I do not amortise the upfront license cost within the yearly fees because a license can always serve other ventures later in time and is a separate long term investment in my opinion), we are still under what the yearly cost of the other purchasing options would be, and would allow me to keep the rack co-locality (as dicussed earlier, connected by cables within the same rack pile or the same data center) with the pricier element, namely the primary AI agent instance, which is also deployed for the cheapest on my hardware in rented rack space. So the on-prem on my own rack deployment of the primary program instance is still the most profitable by far.

We still have to calculate the most profitable option for the standby DB and its direct backup storage, and for the disaster DB backup. Let's start with the DB standby instance and its direct backup storage. For all year (minus 36 days), this instance needs 14TB of RAM , 76TB of disk storage with 31GBps of write throughput, 18.4884MB/s of read throughput, and the equivalent of a AMD EPYC 9116 CPU (16 cores). For 36 days on average in the year, it needs exactly the same capacity as the primary DB instance. The Traditional License purchase would cost me just as much as the primary instance's, because I would buy one time the License for that max capacity needed for the 36 days, and each year I will also have the Support costs. The OCl option with BYOL requires paying the License of $1,520,000 (since I do not yet possess one) and the same yearly support fees of $334,400/y, plus $0.0484 per ECPU per hour, and $0.12 per GB of storage per month. That 16 cores processor capacity equals 8 ECPUs (16 × the 0.5 core factor). So we have the same calculation as for the primary db instance except with a 8 ECPUs config for all year minus 36 days, and then for 36 days the same config as the primary db instance. That gives us $0.0484 × 8 × (261 – 36) × 16 + $0.0484 × 64 × 36 × 16 + 334,400 + $0.12 × 12 × (230,000 + 76,000) = around $778,218/y, plus the upfront one-time License purchase of $1,520,000. The License Included purchase + deployment would cost us $0.1075 × 8 × (261 – 36) × 16 + $0.1075 × 64 × 36 × 16 + 334,400 + $0.12 × 12 × (230,000 + 76,000) = around $782,099/y with no upfront license costs. Here again, the traditional license purchase is the most profitable option, because, I can then deploy it with the cheapest eligible option evoked earlier (namely on my rack on rented rack space, since it must be in a different physical location than the primary db instance as evoked earlier), and once again the price over time is largely under the other options, as that most profitable option would cost me per year $334,400/y (once again I ignore the license one tme purchase cost) plus $51,200/y (see table on standby DB instance deployment prices), so still quite under the near $780k/y other options.

Finally, for the disaster backup, it would cost me around 230,000×12×0.12= $331,200/y with a License included Oracle Cloud or a BYOL Oracle Cloud deployment. Just as observed earlier with other cloud provider prices, the oracle cloud deployment of the disaster backup still costs way over

the expected near $33k/y deployment price of that backup if deployed on rented or owned rack in rented rack space (see earlier tables).

So in conclusion, the deployment architecture which is most profitable is the same before and after taking into account Oracle solution purchasing prices.

Consequences:

First, I must finetune the number of uvicorn workers manually, because otherwise the automaticly set up number might and cause traffic issues [87].

Since some components are ideally deployed on dynamically-scaling Cloud, I must also choose such solution out of the available ones.

The message broker must be added to the deployment diagram, but I do not show it in the other c4 diagrams, because the c4 layers diagrams rather show the conceptual elements and their conceptual worklfow role/info exchanges, not the physical reality of the system, thus AI agent exists only in multiple instances in the deployment diagram, thus it needs a broker only in deployment. Moreover, due to its high resource use, especially RAM, I put it on a separate machine.

Also, since the user-input documents that the AI agent uses are stored and managed by FastAPI, it should be a component within the FastAPI container in the c4 model diagrams, since a container must include all its responsibilities [21].

Also, knowing I will deploy the two AI models in multiple pairs and one pair will have its own machine, each pair needs its own separate vLLM engine [224], but that can exist by default in the cloud vm config, since the scaling can simply replicate the AI agent docker container.

Finally, considering a yearly profit before expenses of $188,871,480 for the US (see Appendix 3), and yearly salaries expenses (around 10M dollars) + deployment costs per year (51,200 + 51,200+ 101,000 + 1,646,400+ 635,000+12,000+ 5,000 (max for the standby fastapi instance on cloud)) + 334,400*2 for the oracle fees per year (besides upfront purchase of the license) + Cloudflare fees which should not exceed the few millions + yearly taxes (minimum 21% of income tax in the US plus the additional average 6.57% of tax on revenue in the US [102, 204], plus at least 10% of other taxes), the business should still bring around $66,120,000/y of net profit.

5.13. ADR: Cloud provider

Date (dd/mm/yyyy): 05/09/2026

Status: Accepted.

Context: some of my components (namely the sandby AI agent and the standby FastAPI-AI broker) are best deployed on dynamically-scaling cloud capacity (see previous ADR). I must choose a provider of such solution to deploy these on Cloud.

Options: private or public cloud, and if public cloud, there are multiple providers and 3 main types of solution to choose between (IaaS, PaaS, SaaS) which are all dynamically scalable [98, 169, 223, 228, 320].

Decision: IBM Cloud

Rationale:

People can create their own cloud, but it brings no scale or security (it is rather less safe) benefits over third-party providers' clouds (public Cloud).

Also, for cloud hosting, there are spot and on-demand options. Spot capacity is usually much cheaper than on-demand, but there is a trade-off: the cloud provider can reclaim those instances with little notice, and specific instance types may not always be available immediately [97]. So I would need on-demand cloud whenever I use it.

Google, IBM, Microsoft and AWS solutions seem to have equivalent advantages and provide all the features I require (inference endpoints number-based scaling from zero to peak number of inference points needed, security, standards, endpoint get both GPU and CPU capacity). I also read about other less known providers like Runpod, which also provide zero-to-peak elasticity based on scaling the number of inference endpoints providing both GPU and CPU capacity for the models and their engine, and providing the most common transparency and data protection standards for rather cheaper than the other solutions [406–409].

I do not need a global reach since the deployment is with country-based capcity just for the clients in the country. The price remains rather low no matter the provider since the backup ai agent component is expected to run only on average 36 days or less per year. So price should not be the main decisive factor. Security is much more important since the parsed info includes financial reports and contracts. And surrounding services (DNS, etc.) are not a strong requirement either since for networking and load balancing for failover I already found my opens source solutions. I cannot investigate it too deeply due to time constraints but I choose IBM Cloud for its finance industry high compliance features, its FedRamp compliance [410], which matter a lot in my case, and because I can run specifically nvidia GPUs and CPUs, which alleviates the need for CPU mediation [229, 411], and because I can run Linux VMs on IBM Cloud [168] and because they have data centers in the US, which respects my geographical storage requirements [37].

5.14. ADR: Best fastAPI - AI machines message broker

Date (dd/mm/yyyy): 05/09/2026

Status: Accepted.

Context: I need to choose a broker which receives FastAPI's AI requests and dispatches them between the available AI inference endpoints, and then receives AI models' responses and sends them back to FastAPI.

Options: RabbitMQ, Kafka, NATS JetStream, Amazon SQS, etc.

Decision: RabbitMQ.

Rationale:

I do not need to hold messages once they have been processed by a 'consumer' (recipient); in my case, this means that an AI model instance needs to have completed its task. The FastAPI container can hold the correlation ID which allows to trace back an AI response to an AI request. An AI request is a single job, that only one AI model instance needs to carry out. So the right broker for my case is a queue-based one, i.e. the type which does not hold information once the message it brokes has been processed, but can still hold info for retries until processing completion is confirmed [26, 412].

The broker must be able to contain two separate queues, since I have two distinct takss with separate characteristics.

It must also be able to sustain my large bandwidth (606.5MB/s of throughput spread across near 132,000 messagess, and 48 GB of RAM at least).

Because the primary instance of the program is hard metal-deployed, and for I want elements to live close together, I want the primary instance of the message broker to be a hard metal-installable broker, while the standby can be on Cloud to dynamically scale its capacity alongside the AI agent's when needed. That standby option could potentially be an IBM solution if there is any, since I use IBM Cloud, if it represents advantages, but this is not mandatory, as other brokers can do the job as long as they can live on IBM Cloud. This also means the needed standby provider must be able to scale easily horizontally/vertically.

RabbitMQ is a queue-based broker that is open-source for the features I need (double queue, scaling, retries with something like a ACK), containerisable on bare metal and on Cloud [74]. I did not benchmark but I found a benchmark indicating that the 4.1 verion of RabbitMQ can deal with 5000 messages of each 1kB in 10ms, so 500,000/s and 500MB/s with only 10 queues, while there can be many more [214] . So I'm confident RabbitMQ is a good option.

Bibliography

[1] L. Ashdown A. Agrawal and Mark Doran. Managing the Redo Log. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/admin/managing-the-redo-log.html

[2] Khalid Abuhakmeh. 2025. OAuth 2.0 vs OpenID Connect: What Is the Difference? *Duende Software*. Retrieved September 9, 2026 from https://duendesoftware.com/learn/the-differences-between-oauth-2-0-and-openid-connect-and-why-they-matter

[3] Nipun Agarwal, Nigel Bayliss, Neerja Bhatt, Wilson Chan, Sumanta Chatterjee, Eugene Chong, Bruce Ernst, Thuvan Hoang, Mark Johnson, Jonathan Klein, Sushil Kumar, Tirthankar Lahiri, Bill Lee, Yunrui Li, Juan Loaiza, Rich Long, Catherine Luu, Neil MacNaughton, Gopal Mulagund, Subramanian Muralidhar, Ravi Murthy, Kant Patel, Mark Ramacher, Suvendu Ray, Debbie Steiner, Malai Stalin, Seema Sundara, Murali Thiyagarajan, Alex Tsukerman, Kothanda Umamageswaran, Steve Vivian, Eric Voss, Tak Wang, Steve Wertheimer, Andy Witkowski, Daniel Wong, Brian Wright, Qin Yu, Mohamed Ziauddin, and Mary Beth Roeser. Statistics Descriptions. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/refrn/statistics-descriptions-2.html?embed=1#GUID-2FBC1B7E-9123-41DD-8178-96176260A639

[4] Rubén Alapont. 2023. Domain Services and Factories in Domain-Driven Design. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/ruben_alapont/domain-services-and-factories-in-domain-driven-design-55lo

[5] Jim Allen Wallace. 2026. Prefill vs Decode: LLM Inference Phases Explained. *Redis*. Retrieved September 6, 2026 from https://redis.io/blog/prefill-vs-decode/

[6] Open Source Alternatives. Traefik vs Nginx: Which Reverse Proxy Wins in 2026? *Open Source Alternatives*. Retrieved September 5, 2026 from https://www.opensourcealternatives.to/blog/traefik-vs-nginx

[7] Jeremy Alva. 2024. FastAPI Backend Architecture: Model-Controller-Service. *Medium*. Retrieved September 4, 2026 from https://medium.com/@jeremyalvax/fastapi-backend-architecture-model-controller-service-44e920567699

[8] Rene Antunez. 2025. René Antúnez | What is an Online Redo Log? : A Back-To-Basics Guide. Retrieved September 5, 2026 from https://rene-ace.com/what-is-an-online-redo-log-a-back-to-basics-guide/

[9] Arista. 202AD. 7060X4 Series 100/200/400G Data Center Switches. Retrieved from https://www.arista.com/assets/data/pdf/Datasheets/7060X4-Datasheet.pdf

[10] Lance Ashdown, Donna Keesling, Tom Kyte, Drew Adams, Ashish Agrawal, Troy Anthony, Vikas Arora, Jagan Athraya, David Austin, Thomas Baby, Vladimir Barriere, Hermann Baer, Srinagesh Battula, Nigel Bayliss, Tammy Bednar, Virginia Beecher, Bjorn Bolltoft, Ashmita Bose, David Brower, Larry Carpenter, Lakshminaray Chidambaran, Deba Chatterjee, Shasank Chavan, Tim Chien, Gregg Christman, Bernard Clouse, Maria Colgan, Carol Colrain, Nelson Corcoran, Michael Coulter, Jonathan Creighton, Judith D'Addieco, Mark Dilman, Kurt Engeleiter, Bjørn Engsig, Marcus Fallon, Steve Fogel, Jonathan Giloni, Naveen Gopal, Bill Habeck, Min-Hank Ho, Lijie Heng, Bill Hodak, Yong Hu, Pat Huey, Praveen Kumar Tupati Jaganath, Sanket Jain, Prakash Jashnani, Caroline Johnston, Shantanu Joshi, Jesse Kamp, Vikram Kapoor, Feroz Khan, Jonathan Klein, Andre Kruglikov, Sachin Kulkarni, Surinder Kumar, Paul Lane, Adam Lee, Allison Lee, Jaebock Lee, Sue Lee, Teck Hua Lee, Yunrui Li, Ilya Listvinski, Bryn Llewellyn, Rich Long, Barb Lundhild, Neil Macnaughton, Vineet Marwah, Susan Mavris, Bob McGuirk, Joseph Meeks, Mughees Minhas, Sheila Moore, Valarie Moore, Gopal Mulagund, Charles Murray, Kevin Neel, Sue Pelski, Raymond Pfau, Gregory Pongracz, Vivek Raja, Ashish Ray, Bert Rich, Kathy Rich, Andy Rivenes, Scott Rotondo, Vivian Schupmann, Venkat Senaptai, Shrikanth Shankar, Prashanth Shanthaveerappa, Cathy Shea, Susan Shepard, Kam Shergill, Mike Skarpelos, Sachin Sonawane, James Spiller, Suresh Sridharan, Jim Stenoish, Janet Stern, Rich Strohm, Roy Swonger, Kamal Tbeileh, Juan Tellez, Ravi Thammaiah, Lawrence To, Tomohiro Ueda, Randy Urbano, Badhri Varanasi, Nick Wagner, Steve Wertheimer, Patrick Wheeler, Doug Williams, James Williams, Andrew Witkowski, Daniel Wong, Hailing Yu, and Mark Doran. Memory Architecture. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/cncpt/memory-architecture.html

[11] Michaël Aussems. 2026. Oracle launches Cloud@Customer for smaller local AI databases. *ITdaily*. Retrieved September 9, 2026 from https://itdaily.com/news/datacenter/oracle-base-database-cloud-customer/

[12] Auth0. JSON Web Token Introduction. *JSON Web Tokens*. Retrieved September 16, 2026 from https://jwt.io/introduction

[13] Andrew Babb, Donna Carver, Tim Chien, Sean Connolly, Donna Cooksey, Bill Fischer, Mahesh Girkar, Ray Guzman, Dah-Yoh Lim, Colin McGregor, Kant Patel, Chris Plakyda, Padmaja Potineni, Kathy Rich, Jony Safi, Toru Sasaki, Lawrence To, Randy Urbano, and

Steven Wertheimer. Oracle Database - Release 23.1 - 1 Introduction to Recovery Appliance. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/engineered-systems/zero-data-loss-recovery-appliance/23.1/amagd/introduction-recovery-appliance.html?source=%3Aow%3Alp%3Acpo%3A%3A%3A%3ARC_WWSA210218P00008%3ALPD400146646

[14] Hermann Baer, Bjorn Bolltoft, Ali Cakmak, Sunil Chakkappen, Immanuel Chan, Deba Chatterjee, Chris Chiappa, Dinesh Das, Kurt Engeleiter, Leonidas Galanis, William Endress, Marcus Fallen, Bruce Golbus, Katsumi Inoue, Praveen Kumar Tupati Jaganath, Mark Jefferys, Shantanu Joshi, Adam Kociubes, Keith Laker, Allison Lee, Sue Lee, Cheng Li, David McDermid, Colin McGregor, Ajit Mylavarapu, Ted Persky, Lei Sheng, Ekrem Soylemez, Hong Su, Murali Thiyagarajah, Randy Urbano, Sahil Vazirani, Bharath Venkatakrishnan, Hailing Yu, John Zimmerman, Frederick Kush, and Lance Ashdown. Reading Execution Plans. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/18/tgsql/reading-execution-plans.html

[15] Hermann Baer, Bjorn Bolltoft, Ali Cakmak, Sunil Chakkappen, Immanuel Chan, Deba Chatterjee, Chris Chiappa, Dinesh Das, Kurt Engeleiter, Leonidas Galanis, William Endress, Marcus Fallen, Bruce Golbus, Katsumi Inoue, Mark Jefferys, Shantanu Joshi, Adam Kociubes, Praveen Kumar Tupati Jaganath, Keith Laker, Allison Lee, Sue Lee, Cheng Li, Jiakun Li, David McDermid, Colin McGregor, Ajit Mylavarapu, Kantikiran Pasupuleti, Ted Persky, Palash Sharma, Lei Sheng, Ekrem Soylemez, Hong Su, Murali Thiyagarajah, Randy Urbano, Satya Valluri, Sahil Vazirani, Bharath Venkatakrishnan, Hailing Yu, Yuying Zhang, John Zimmerman, and Lance Ashdown. Query Optimizer Concepts. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/tgsql/query-optimizer-concepts.html

[16] vijay balebail. 2025. OracleDB Python Connections to Autonomous DB: Thick vs Thin Mode Explained. *Medium*. Retrieved September 5, 2026 from https://medium.com/@vbalebai/oracledb-python-connections-to-autonomous-db-thick-vs-thin-mode-explained-7e6146ff3001

[17] BinaryPatrick. 2025. Restart Unhealthy Docker Containers Automatically. *DEV Community*. Retrieved September 5, 2026 from https://dev.to/binarypatrick/restart-unhealthy-docker-containers-automatically-4c00

[18] Matt Binder. 2025. Framework raises DDR5 RAM prices again with per GB price hike. *Mashable*. Retrieved September 6, 2026 from https://mashable.com/article/framework-price-hike-ddr5-ram-memory-shortage

[19] Philip Brown. 2014. What are Factories in Domain Driven Design? | Culttt. *Cult3*. Retrieved March 6, 2026 from https://culttt.com/2014/12/24/factories-domain-driven-design

[20] Simon Brown. c4 model. *C4 model*. Retrieved September 4, 2026 from https://c4model.com/

[21] Simon Brown. Container diagram. *C4 model*. Retrieved September 4, 2026 from https://c4model.com/diagrams/container

[22] Simon Brown. Component. *C4 model*. Retrieved September 4, 2026 from https://c4model.com/abstractions/component

[23] Jason Brownlee. 2023. Asyncio I/O Libraries. *Super Fast Python*. Retrieved February 5, 2026 from https://superfastpython.com/asyncio-io-libraries/

[24] Ian Buchanan. Containers vs Virtual Machines. *Atlassian*. Retrieved September 6, 2026 from https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms

[25] Kushal Byatnal. 2026. Agentic Document Extraction Guide March 2026. *Extend*. Retrieved September 4, 2026 from https://www.extend.ai/resources/agentic-document-extraction-complete-guide

[26] Bindu C. 2025. Distributed System : Difference Between Message Queue and Streaming System. *Medium*. Retrieved September 11, 2026 from https://medium.com/@bindubc/distributed-system-difference-between-message-queue-and-streaming-system-a57c44ae3fb5

[27] candied_orange. 2017. Answer to “Clean Architecture: Use case containing the presenter or returning data?” *Software Engineering Stack Exchange*. Retrieved September 5, 2026 from https://softwareengineering.stackexchange.com/a/357066

[28] CaptainDNS. 2026. DNS Propagation Time: How Long Does It Really Take? *CaptainDNS*. Retrieved September 5, 2026 from https://www.captaindns.com/en/blog/dns-propagation-time

[29] Alexandre Cassen. Quick Start. *KeepAlived*. Retrieved September 5, 2026 from https://www.keepalived.org/documentation/user-guide/quick-start/

[30] Magomed Chatuev. 2020. DDD for microservices. *Medium*. Retrieved April 17, 2026 from https://medium.com/@chatuev/ddd-for-microservices-4778a363c071

[31] Joseph Chen. 2025. Understanding Hard Drive Bandwidth: Boost Performance Now. *ULINK DA Drive Analyzer*. Retrieved September 5, 2026 from https://ulink-da.com/hard-drive-bandwidth/

[32] Kevin Closson. 2012. SLOB 2. Retrieved from https://kevinclosson.net/wp-content/uploads/2013/12/slob2_readme-2012-05-04.pdf

[33] Thomas Coughlin. 2026. SSD Storage Capacity Prices Are Over 20 Times HDD Storage Capacity Prices. *Yahoo Finance*. Retrieved September 5, 2026 from https://uk.finance.yahoo.com/news/ssd-storage-capacity-prices-over-200117036.html

[34] Dikhyant Krishna Dalai. 2025. FastAPI Performance Bottlenecks: Why Middleware and ORMs Kill Throughput (and How to Fix Them). *Medium*. Retrieved September 6, 2026 from https://medium.com/@dikhyantkrishnadalai/fastapi-performance-bottlenecks-why-middleware-and-orms-kill-throughput-and-how-to-fix-them-a79924bfaebb

[35] Filip Danić. 2026. Do You Really Need Redis? When is an In-Memory Cache Enough? - Filip Danić. *Filip Danić's Website*. Retrieved September 4, 2026 from https://danicfilip.com/blog/do-you-really-need-redis/

[36] Erik Darling. 2021. I/O Bound vs CPU Bound Queries In SQL Server | Darling Data. *Darling Data*. Retrieved September 4, 2026 from https://erikdarling.com/i-o-bound-vs-cpu-bound-queries/

[37] Datacenters.com. IBM Cloud: United States Data Centers - Providers Map in United States. *Datacenters.com*. Retrieved September 15, 2026 from https://www.datacenters.com/providers/ibm-cloud/locations/united-states

[38] Dataprise. 2023. SSDs vs. Spinning Disks: Which Storage Solution is Right for You? *Dataprise*. Retrieved September 5, 2026 from https://www.dataprise.com/resources/blog/ssds-vs-spinning-disks/

[39] A. Jesse Jiryu Davis. 2022. Why Should Async Get All The Love?: Advanced Control Flow With Threads. *Empty Square*. Retrieved February 2, 2026 from https://emptysqua.re/blog/why-should-async-get-all-the-love/

[40] Parth Rajesh Dedhia. 2025. Batch processing at Scale: How PostgreSQL outperformed BigQuery for us. *Data Science Collective*. Retrieved September 4, 2026 from https://medium.com/data-science-collective/batch-processing-at-scale-how-postgresql-outperformed-bigquery-for-us-34b00c572039

[41] Agilista Desenvolvedor. 2026. FastAPI in Production: From the Default Framework to the Edge of Performance and a Fair Comparison…. *Medium*. Retrieved September 6, 2026 from https://medium.com/@agilista.desenvolvedor/fastapi-in-production-from-the-default-framework-to-the-edge-of-performance-and-a-fair-comparison-7d00524379b3

[42] Mehmet Salih Deveci. 2021. Hard Parse vs. Soft Parse and Bind Variable in Oracle Database. *IT Tutorial*. Retrieved September 7, 2026 from https://ittutorial.org/hard-parse-vs-soft-parse-and-bind-variable-in-oracle-database/

[43] Nawaz Dhandala. 2025. How to Implement Rate Limiting in FastAPI Without External Services. *OneUptime | One Complete Observability platform.* Retrieved September 4, 2026 from https://oneuptime.com/blog/post/2025-01-06-fastapi-rate-limiting/view

[44] Cem Dilmegani. 2026. GPU Marketplace: Vast.ai vs Shadeform vs Prime Intellect. *AIMultiple*. Retrieved September 6, 2026 from https://aimultiple.com/gpu-marketplace

[45] Sudha Duraiswamy, Michael Hwa, Sudha Iyer, Supriya Kalyanasundaram, Lakshmi Kethana, Peter Knaggs, Andrew Koyfman, Dah-Yoh Lim, Adam Lee, Adam Lindsey, Rahil Mir, Gopal Mulagund, Andy Philips, Preetam Ramakrishna, Philip Thornton, Peter Wahl, Lixia Yuan, Paul Youn, and Patricia Huey. Managing the Keystore and the Master Encryption Key. *Oracle Help Center*. Retrieved September 4, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/asoag/managing-keystore-and-tde-master-encryption-key.html

[46] Roman Dykyi. 2025. Understanding Validation Levels. *Medium*. Retrieved September 4, 2026 from https://medium.com/@dykyi.roman/understanding-validation-levels-74d0adecda5e

[47] Erichain. 2024. A Deep Dive into AI Large Models: Parameters, Tokens, Context Windows, Context Length, and…. *Medium*. Retrieved September 4, 2026 from https://medium.com/@jiarunhuang/a-deep-dive-into-ai-large-models-parameters-tokens-context-windows-context-length-and-8187fb393acb

[48] Vahid Faraji. 2026. Where Should Repository Interfaces Live in Clean Architecture? *Medium*. Retrieved September 4, 2026 from https://vahid-faraji-dev.medium.com/where-should-repository-interfaces-live-in-clean-architecture-7c12f74a4d7b

[49] Patricio ferraggi. 2020. Do you need Design Patterns in Functional Programming? *DEV Community*. Retrieved March 6, 2026 from https://dev.to/patferraggi/do-you-need-design-patterns-in-functional-programming-370c

[50] John Fincher. 2026. Sending Emails With Python – Real Python. *Real Python*. Retrieved September 4, 2026 from https://realpython.com/python-send-email/

[51] flashdba. 2014. Oracle AWR Reports: Understanding I/O Statistics. *flashdba*. Retrieved September 5, 2026 from https://flashdba.com/2014/02/26/oracle-awr-reports-understanding-io-statistics/

[52] Martin Fowler. 2006. GUI Architectures. *martinfowler.com*. Retrieved September 5, 2026 from https://martinfowler.com/eaaDev/uiArchs.html

[53] Rico Fritzsche. 2024. Bounded Contexts: Behavior Over Data Structures - Part II. *Rico Fritzsche*. Retrieved April 20, 2026 from https://ricofritzsche.me/ddd-modularization-concepts-aggregates-part-ii/

[54] Matthieu Gaillard-Midol. Souveraineté Numérique : L'ère du « Sovereignty First » et du Cloud de Résistance. *Devoteam*. Retrieved September 5, 2026 from https://www.devoteam.com/fr/expert-view/souverainete-numerique-lere-du-sovereignty-first-et-du-cloud-de-resistance/

[55] Jatin Garg. 2025. Types of Database Indexes: When to Use B-Tree, Hash, and Full-Text Indexes. *Gocodeo*. Retrieved September 5, 2026 from https://www.gocodeo.com/post/types-of-database-indexes-when-to-use-b-tree-hash-and-full-text-indexes

[56] Rahul Garg. 2022. Demystifying software architecture patterns. *Thoughtworks*. Retrieved April 17, 2026 from https://www.thoughtworks.com/insights/blog/architecture/demystify-software-architecture-patterns

[57] gelovolro. 2024. What are merge join, hash join, and nested loop? Example in PostgreSQL. *Medium*. Retrieved September 5, 2026 from https://gelovolro.medium.com/what-are-merge-join-hash-join-and-nested-loop-example-in-postgresql-29123ca18fd1

[58] George. 2023. Separation of Concerns between Use Cases and Entities. *Technical blog from UNIL engineering teams*. Retrieved September 4, 2026 from https://medium.com/unil-ci-software-engineering/separation-of-concern-between-use-cases-and-entities-e3a86cdee49e

[59] Iddo Gino. 2025. Post on LinkedIn. *LinkedIn*. Retrieved September 4, 2026 from https://www.linkedin.com/posts/iddogino_do-we-still-need-rag-in-the-age-of-large-activity-7326648596185915392-Q2M8/

[60] Marlon Giona. 2025. Sequential vs Random I/O. *LinkedIn*. Retrieved September 5, 2026 from https://www.linkedin.com/pulse/sequential-vs-random-io-marlon-giona-dkswf/

[61] Dinu Gitlan. 2025. What Is SSL Termination And How Does It Work? *Medium*. Retrieved September 5, 2026 from https://medium.com/@munteanu210/what-is-ssl-termination-and-how-does-it-work-a194a77c2974

[62] Grant Thornton. 2024. IFRS Example Consolidated Financial Statements 2024. Retrieved from https://www.grantthornton.global/globalassets/1.-member-firms/global/insights/insight-content-blocks-and-media/ifrs/example-financial-statements-2024/ifrs-2024-example-financial-statements.pdf

[63] Harry Percival and Bob Gregory. 2020. cosmic_python - Simple patterns for building complex applications. *Cosmic Python*. Retrieved March 16, 2026 from https://www.cosmicpython.com

[64] D. Hardt. 2012. *The OAuth 2.0 Authorization Framework*. Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

[65] Dzmitry Harunou. 2025. Gateway Unit in Clean Architecture for Frontend Applications. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/harunou/gateway-unit-in-clean-architecture-for-frontend-applications-137f

[66] Anca Hellmann. 2026. How to Choose Modular Monolith vs. Microservices Architecture. *DO OK*. Retrieved September 4, 2026 from https://dook.pro/blog/tips-tricks/how-to-choose-modular-monolith-vs-microservices-architecture/

[67] Hesbon limo. 2025. Understanding File I/O in Python: Reading, Writing, and Managing Files. *DEV Community*. Retrieved February 2, 2026 from https://dev.to/limoh653/understanding-file-io-in-python-reading-writing-and-managing-files-2dof

[68] hgraca. 2018. More than concentric layers. *@hgraca*. Retrieved April 20, 2026 from https://herbertograca.com/2018/07/07/more-than-concentric-layers/

[69] Kevin Hood. Scaling Stateless vs Stateful Services. Retrieved February 20, 2026 from https://blog.dreamfactory.com/scaling-stateless-vs-stateful-services

[70] Adam Hopkins. 2020. Oracle Licensing - NUP or CPU. *DSP*. Retrieved September 9, 2026 from https://content.dsp.co.uk/oracle-licensing-nup-or-cpu

[71] Andreas Hütter. 2022. Answer to “What actually is a subdomain in domain-driven design?” *Stack Overflow*. Retrieved April 17, 2026 from https://stackoverflow.com/a/73080428

[72] Katerina Hynkova. 2025. Ultimate guide to FastAPI library in Python. *DeepNote*. Retrieved September 4, 2026 from https://deepnote.com/blog/ultimate-guide-to-fastapi-library-in-python

[73] Mostafa Ibrahim. 2024. OAuth vs JWT: Key Differences Explained | SuperTokens. Retrieved September 4, 2026 from https://supertokens.com/blog/oauth-vs-jwt

[74] IcePanel. 2023. Top 6 message queues for distributed architectures. *DEV Community*. Retrieved September 12, 2026 from https://dev.to/icepanel/top-6-message-queues-for-distributed-architectures-3hmb

[75] IEEE. Network Fabric. *IEEE Technology Navigator*. Retrieved September 6, 2026 from https://technav.ieee.org/topic/network-fabric/

[76] Ignacy. 2026. vLLM vs TensorRT-LLM vs SGLang (2026): Which to Pick. *DTF*. Retrieved September 5, 2026 from https://decodethefuture.org/en/vllm-vs-tensorrt-llm-vs-sglang-2026/

[77] iku. 2025. Is JSON Outdated? The Reasons Why the New LLM-Era Format “TOON” Saves Tokens. *DEV Community*. Retrieved September 6, 2026 from https://dev.to/akari_iku/is-json-outdated-the-reasons-why-the-new-llm-era-format-toon-saves-tokens-31e5

[78] ImranzamanML. 2024. Here is how we can calculate the size of any LLM model: Each parameter in LLM…. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/posts/ImranzamanML/127269471333935

[79] IndustryNews. 2025. SSD Read and Write Speed Explained in Details - OSCOO. Retrieved September 5, 2026 from https://www.oscoo.com/news/ssd-read-and-write-speed-explained-in-details/

[80] Jackmin801. 2026. H100 throughputs. *GitHubGist*. Retrieved September 6, 2026 from https://gist.github.com/Jackmin801/e30f590920ccf33d03295704d79592a4

[81] jamesmontemagno. Designing a microservice domain model - .NET. *Microsoft*. Retrieved April 20, 2026 from https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/microservice-domain-model

[82] Thorben Janssen. 2023. Design Patterns Explained – Dependency Injection. *Stackify*. Retrieved March 1, 2026 from https://stackify.com/dependency-injection/

[83] Jarvislabs. 2026. NVIDIA H100 Price Guide 2026: GPU Costs, Cloud Pricing & Buy vs Rent. *Jarvislabs*. Retrieved September 6, 2026 from https://jarvislabs.ai/blog/h100-price

[84] Hossein Nejati Javaremi. 2026. The Anti-Corruption Layer: Protecting Your Domain from Legacy Systems. *Medium*. Retrieved September 4, 2026 from https://hosseinnejati.medium.com/the-anti-corruption-layer-protecting-your-domain-from-legacy-systems-6da58fc5f462

[85] Arvid Kahl. 2025. Local AI vs APIs: Making Pragmatic Choices for Your Business. *The Bootstrapped Founder*. Retrieved September 6, 2026 from https://thebootstrappedfounder.com/when-to-choose-local-llms-vs-apis-a-founders-real-world-guide/

[86] Ramesh Kannan s. 2026. Nginx Is Overkill for FastAPI: I Benchmarked Caddy and Traefik. *Medium*. Retrieved September 5, 2026 from https://medium.com/@rameshkannanyt0078/nginx-is-overkill-for-fastapi-i-benchmarked-caddy-and-traefik-846a0e6539eb

[87] Ayush Kaushik. 2026. How Many Uvicorn Workers Do You Actually Need? *LogicLoopTech*. Retrieved September 6, 2026 from https://www.logiclooptech.dev/how-many-uvicorn-workers-do-you-actually-need-fastapi-performance-guide/

[88] kdakan. 2020. DDD-Domain-Driven-Design/README.md at master · kdakan/DDD-Domain-Driven-Design. *GitHub*. Retrieved April 20, 2026 from https://github.com/kdakan/DDD-Domain-Driven-Design/blob/master/README.md

[89] KFSys. 2023. Navigating High Availability with Keepalived | DigitalOcean. *DigitalOcean*. Retrieved September 5, 2026 from

https://www.digitalocean.com/community/questions/navigating-high-availability-with-keepalived

[90] Kludex. uvicorn/uvicorn/supervisors/multiprocess.py at main · Kludex/uvicorn. *GitHub*. Retrieved September 4, 2026 from https://github.com/Kludex/uvicorn/blob/main/uvicorn/supervisors/multiprocess.py

[91] Ahmet Eren Lapanta. 2026. From Threads to Coroutines: Modern Concurrency and Parallelism Explained. *HesapTech*. Retrieved September 4, 2026 from https://medium.com/hesaptech/from-threads-to-coroutines-modern-concurrency-and-parallelism-explained-ac5484377722

[92] Ken Lee. 2024. Why Use A Linux Server In Enterprise Environments? | SUSE Blog. *Suse*. Retrieved September 10, 2026 from https://www.suse.com/c/why-use-linux/

[93] Newvick Lee. 2025. Python Concurrency: Threads, Processes, and asyncio Explained. *Newvick's blog*. Retrieved February 2, 2026 from https://newvick.com/posts/python-concurrency/

[94] R Leishman. 2007. Understanding Indexes. *OracleFAQ*. Retrieved September 5, 2026 from https://www.orafaq.com/node/1403

[95] Patroclos Lemoniatis. 2022. Stored Procedures vs ORM Framework: When to use each one and Why. *Medium*. Retrieved September 5, 2026 from https://patroclosdev.medium.com/stored-procedures-vs-orm-framework-when-to-use-each-one-and-why-b575600ee124

[96] long2ice. 2026. fastapi-limiter/pyproject.toml at main · long2ice/fastapi-limiter. *GitHub*. Retrieved September 4, 2026 from https://github.com/long2ice/fastapi-limiter/blob/main/pyproject.toml

[97] Zhen Lu. 2024. Spot vs. On-Demand Instances: What's the Difference? *Runpod*. Retrieved September 14, 2026 from https://www.runpod.io/blog/spot-vs-on-demand

[98] Bhavika M. 2026. Monolith vs Microservices: What Actually Matters (Lessons From Netflix, Amazon, Atlassian). *Medium*. Retrieved September 4, 2026 from https://aws.plainenglish.io/monolith-vs-microservices-what-actually-matters-lessons-from-netflix-amazon-atlassian-7cdc19dff4e4

[99] Cilia Madani. 2025. Making FastAPI Fast: A Beginner's Guide to Workers and Threads. *Medium*. Retrieved September 4, 2026 from https://medium.com/@ciliaMadani/making-fastapi-fast-a-beginners-guide-to-workers-and-threads-095d7f2b9575

[100] Sizan Mahmud. 2025. The Hidden Cost of Bad Caching: Why More Cache Isn't Always Better. *Medium*. Retrieved September 4, 2026 from https://medium.com/@sizanmahmud08/the-hidden-cost-of-bad-caching-why-more-cache-isnt-always-better-168b74af6715

[101] Shiladitya Majumder. 2026. Using SlowAPI in FastAPI: Mastering Rate Limiting Like a Pro. *Medium*. Retrieved September 4, 2026 from https://shiladityamajumder.medium.com/using-slowapi-in-fastapi-mastering-rate-limiting-like-a-pro-19044cb6062b

[102] Abir Mandal. 2026. State Corporate Income Tax Rates and Brackets, 2026. *Tax Foundation*. Retrieved September 16, 2026 from https://taxfoundation.org/data/all/state/state-corporate-income-tax-rates-brackets/

[103] C Martin. 2012. Clean Architecture. *The Clean Blog*. Retrieved from https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

[104] Lucas McGregor. 2026. Comparing Different LLM Models For Data Extraction. *Medium*. Retrieved September 4, 2026 from https://medium.com/the-leading-indicator/comparing-different-llm-models-for-data-extraction-f067d56bed54

[105] P M Mell and T Grance. 2011. *The NIST definition of cloud computing* (0 ed.). National Institute of Standards and Technology, Gaithersburg, MD. https://doi.org/10.6028/NIST.SP.800-145

[106] Mitrasish. 2026. Self-Host Embeddings and Rerankers: TEI on GPU Cloud (2026). *Spheron*. Retrieved September 6, 2026 from https://www.spheron.network/blog/self-host-embedding-reranker-tei-gpu-cloud/

[107] David Muraya. 2025. A Practical Guide to FastAPI Security. Retrieved September 4, 2026 from https://www.davidmuraya.com/blog/fastapi-security-guide

[108] ISAAC NDUBUISI. 2022. Should you use Raw SQL or an ORM? *CodeX*. Retrieved September 4, 2026 from https://medium.com/codex/should-you-use-raw-sql-or-an-orm-18468c11c1a9

[109] Nexora. 2026. I built a rate limiter that survives process restarts — no Redis required. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/nexoradev/i-built-a-rate-limiter-that-survives-process-restarts-no-redis-required-g4n

[110] Nitin. 2023. Serverful vs Serverless: Which One is Right for Your Needs? *Locofy Blogs*. Retrieved September 5, 2026 from https://www.locofy.ai/blog/serverful-vs-serverless

[111] Olivia Novak. 2025. Asynchronous vs. Synchronous Functions in FastAPI When to Pick Which. *Leapcell*. Retrieved February 1, 2026 from https://leapcell.io/blog/asynchronous-vs-synchronous-functions-in-fastapi-when-to-pick-which

[112] Nvidia. 2026. NVIDIA-based Enterprise Solutions. Retrieved from HTTPs://www.gigabyte.com/FileUpload/Global/WebPage/1052/NVIDIA_2026_1H_V2.pdf

[113] Romuald Oluwatobi. 2025. Python Concurrency Model Comparison For CPU And IO Bound Execution: Asyncio vs Threads vs Sync. *Medium*. Retrieved February 19, 2026 from https://medium.com/@romualdoluwatobi/python-concurrency-model-comparison-for-cpu-and-io-bound-execution-asyncio-vs-threads-vs-sync-35c114fc0045

[114] Alan Ouko. 2026. SQL ON DELETE CASCADE: Automatically Remove Dependent Data. *DataCamp*. Retrieved September 4, 2026 from https://www.datacamp.com/tutorial/sql-on-delete-cascade

[115] Devrim Ozcay. 2026. We Built a REST API with 47 Endpoints. Should’ve Been 12. *Let’s Code Future, on Medium*. Retrieved September 5, 2026 from https://medium.com/lets-code-future/we-built-a-rest-api-with-47-endpoints-shouldve-been-12-8185d99bc802

[116] Mehmet Ozkaya. 2024. The Bridge: Modular Monolith (Modulith) Architectures — Gateway Between Monolith and Microservices…. *Medium*. Retrieved September 4, 2026 from https://mehmetozkaya.medium.com/the-bridge-modular-monolith-modulith-architectures-gateway-between-monolith-and-microservices-069338211de7

[117] Maria Paktiti. 2026. Picking a password hash: A developer’s guide to argon2, bcrypt, and scrypt. *WorkOS*. Retrieved September 4, 2026 from https://workos.com/blog/picking-a-password-hash-argon2-bcrypt-scrypt

[118] Arpita Pal. 2025. LLM, VLM, and VLA. *Medium*. Retrieved September 4, 2026 from https://medium.com/@arpipal2/llm-vlm-and-vla-d758b91479eb

[119] pallets. jinja/docs/intro.rst at main · pallets/jinja. *GitHub*. Retrieved September 4, 2026 from https://github.com/pallets/jinja/blob/main/docs/intro.rst

[120] Bugra Parlayan. 2024. Understanding and Optimizing the “db file sequential read” Oracle Wait Event - Bugra Parlayan | Oracle Database & Exadata Blog. *Bugra Parlayan’s Blog*. Retrieved September 5, 2026 from https://www.bugraparlayan.com.tr/understanding-and-optimizing-the-db-file-sequential-read-oracle-wait-event.html

[121] Josh Parsons. 2025. Password Hashing: What Cyber Pros Need to Know. *Enzoic*. Retrieved September 4, 2026 from https://www.enzoic.com/blog/password-hashing-what-cyber-pros-need-to-know/

[122] Alejandro Penalba Tarrida. 2025. PyPex: Analizador Estático de Complejidad para Código Python. Universidad de Barcelona. Retrieved from https://diposit.ub.edu/server/api/core/bitstreams/fbc44f0c-3d69-40ff-8b7c-5cc1dd7e97f1/content

[123] pete@fedramp.gov. Scope of FedRAMP¶. *FedRamp*. Retrieved September 5, 2026 from https://fedramp.gov/2026/scope/

[124] Plainionist. 2018. Implementing Clean Architecture - Of controllers and presenters. *Plainionist*. Retrieved September 4, 2026 from https://plainionist.github.io/Implementing-Clean-Architecture-Controller-Presenter/

[125] plainionist. 2020. Answer to “Do I need presenter in Clean Architecture?” *Stack Overflow*. Retrieved September 5, 2026 from https://stackoverflow.com/a/59589084

[126] Padmaja Potineni, Kathy Rich, Andy Adams, Beldalker Anand, Chipper Brown, Larry Carpenter, Pieter Van Puymbroeck, Laurence Clarke Jin-Jwei Chen, Jeff Detjen, Ray Dutcher, B. G. Garin David Gagne, Mahesh Girkar, Joydip Kundu Yuhong Gu, Steven Lee, Steven Lim, Nitin Karkhanis, Jonghyun Lee Goutam Kulkarni, Bob McGuirk, Joe Meeks, Steve Moriarty, Muthu Olagappan, Ashish Ray, Mike Schloss, Mike Smith, Lawrence To, Doug Voss Stephen Vivian, Hongjie Yang, and Subhash Chandra. Getting Started with Oracle Data Guard. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/sbydb/getting-started-with-oracle-data-guard.html

[127] Ayush Raj. 2026. AI Agents for Ecommerce. *Exei*. Retrieved September 6, 2026 from https://exei.ai/blog/guide-to-handle-multiple-concurrent-llm-requests-with-vllm

[128] Ujjwal Raj. 2025. Volatility-Based Decomposition: A System Design Example. *DEV Community*. Retrieved February 26, 2026 from https://dev.to/ujjwall-r/volatility-based-decomposition-a-system-design-example-an3

[129] Bhagya Rana. 2026. 10 Decentralized GPU Clouds Taking on AWS. *Medium*. Retrieved September 6, 2026 from https://medium.com/@bhagyarana80/10-decentralized-gpu-clouds-taking-on-aws-7bfac993c86f

[130] Lilith Ray. 2026. SSD Sequential vs Random Speed: Which One Matters for Your PC? *OSCOO*. Retrieved September 5, 2026 from https://www.oscooshop.com/blogs/blogs/ssd-sequential-vs-random-speed

[131] Redis. Understanding Domain-Driven Design (DDD) for Developers. *Redis*. Retrieved September 4, 2026 from https://redis.io/glossary/domain-driven-design-ddd/

[132] Adejumo Ridwan Suleiman. 2026. Python Dependency Injection: A Guide for Cleaner Code Design. *DataCamp*. Retrieved March 16, 2026 from https://www.datacamp.com/tutorial/python-dependency-injection

[133] Mary Beth Roeser, Nipun Agarwal, Rick Anderson, Mark Bauer, Neerja Bhatt, Tudor Bosman, George Candea, Wilson Chan, Sumanta Chatterjee, Eugene Chong, Connie Dialeris Green, Harvey Eneman, Bruce Ernst, Ira Greenberg, Steve Harris, Thuvan Hoang, Lilian Hobbs, Namit Jain, Hakkan Jakobsson, Bob Jenkins, Mark Johnson, Jonathan Klein, Sushil Kumar, Tirthankar Lahiri, Bill Lee, Yunrui Li, Juan Loaiza, Rich Long, Diana Lorentz, Catherine Luu, Neil MacNaughton, Ari Mozes, Gopal Mulagund, Subramanian Muralidhar, Ravi Murthy, Karuna Muthiah, Arvind Nithrakashyap, Ronald Obermarck, Kant Patel, Christopher Racicot, Mark Ramacher, Suvendu Ray Shankar Raman, Ann Rhee, Kathy Rich, Vivian Schupmann, Debbie Steiner, Malai Stalin, Seema Sundara, Juan Tellez, Alex Tsukerman, Kothanda Umamageswaran, Randy Urbano, Steve Vivian, Eric Voss, Tak Wang, Steve Wertheimer, Andy Witkowski, Daniel Wong, Graham Wood, Brian Wright, Aravind Yalamanchi, Qin Yu, Mohamed Ziauddin, and Bert Rich. Oracle Database - Release 19 - V$SQLSTATS. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/V-SQLSTATS.html

[134] Moez SADOK. 2026. moez-sadok/chat-clean-architecture. Retrieved September 5, 2026 from https://github.com/moez-sadok/chat-clean-architecture

[135] Laurent Savaete. 2026. laurentS/slowapi. *GitHub*. Retrieved September 4, 2026 from https://github.com/laurentS/slowapi

[136] Deeksha Sehgal. 2025. Oracle Sharding:Enterprise-Grade Distributed Database. Retrieved September 4, 2026 from https://blogs.oracle.com/database/oracle-shardingenterprisegrade-distributed-database

[137] sgl-project. [Bug] Why is KV cache memory allocated when serving embedding models? Is it necessary? · Issue #9181 · sgl-project/sglang. *GitHub*. Retrieved September 6, 2026 from https://github.com/sgl-project/sglang/issues/9181

[138] Craig Shallahamer. 2016. What's The Difference Between Direct Path Reads And DB File Scattered Reads? *Orapub*. Retrieved September 5, 2026 from https://blog.orapub.com/20160316/difference-between-oracle-database-direct-path-reads-and-db-file-scattered-reads.html

[139] Sharyu. 2025. " When your apps goes from 100 to 10,000 users : Vertical vs Horizontal Scaling ". *Medium*. Retrieved September 4, 2026 from https://medium.com/@sharyup01/when-your-apps-goes-from-100-to-10-000-users-vertical-vs-horizontal-scaling-ced1e8fb2d46

[140] Oleg Sinavski. 2024. Interfaces: abc vs. Protocols. *Oleg Sinavski*. Retrieved March 17, 2026 from https://sinavski.com/post/1_abc_vs_protocols/

[141] Bhagya Sithumini. 2025. How To Implement Clean Architecture in FastAPI: A Step-by-Step Guide. *Medium*. Retrieved September 4, 2026 from https://medium.com/@bhagyasithumini/how-to-implement-clean-architecture-in-fastapi-a-step-by-step-guide-8b73a75c650b

[142] Brett Slatkin. 2019. Python Classes and Interfaces. In *Effective Python: 90 Specific Ways to Write Better Python* (2nd Edition). Retrieved March 16, 2026 from https://www.informit.com/articles/article.aspx?p=2992597&seqNum=2

[143] Mike Smith. 2026. tfb Archives. *TechEmpower*. Retrieved September 6, 2026 from https://www.techempower.com/blog/tag/tfb/

[144] Sobolevn. 2019. Enforcing Single Responsibility Principle in Python. *Sobolevn's personal blog*. Retrieved March 16, 2026 from https://sobolevn.me/2019/03/enforcing-srp

[145] NETservis s.r.o. AIC EB202-CP - XP1-E202CP03. *Anafra*. Retrieved September 6, 2026 from https://smicro.eu/aic-eb202-cp-xp1-e202cp03-1

[146] Jeffrey Steiner. 2024. Oracle database performance optimization and benchmarking. *NetApp*. Retrieved September 5, 2026 from https://docs.netapp.com/us-en/ontap-apps-dbs/oracle/oracle-notes-optimization-and-benchmarking.html

[147] Jared Still. 2021. How to Make Oracle Use the Correct Index. *Pythian*. Retrieved September 5, 2026 from https://www.pythian.com/blog/how-to-make-oracle-use-the-correct-index

[148] Daniel Suherman, Ashish Agrawal, Waleed Ahmed, Helen Altmar, Lance Ashdown, Pete Belknap, Supiti Buranawatanachoke, Romain Colle, Karl Dias, Kurt Engeleiter, Leonidas Galanis, Veeranjaneyulu Goli, Prabhaker Gongloor, Prakash Gupta, Shantanu Joshi, Prathiba Kalirengan, Karen McKeen, Mughees Minhas, Konstantinos Morfonios, Valarie Moore, Ravi Pattabhi, Bert Rich, Yujun Wang, Keith Wong, Qinyi Wu, Khaled Yagoub, Hailing Yu, Yury Berezin, and Sunil Surabhi. Using Workload Analysis. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/ratug/using-workload-analysis.html

[149] Ardit Sulce. 2025. Functions vs. Classes in Python: How to Know Which One to Use. *Daily Python Projects*. Retrieved March 16, 2026 from https://dailypythonprojects.substack.com/p/when-to-use-a-function-vs-a-class

[150] Michael Swengel. 2025. Linux IS Harder to Use. So Why Do People Like It? *Medium*. Retrieved September 10, 2026 from https://medium.com/@michaelswengel/linux-is-harder-to-use-so-why-do-people-like-it-14c86a6d4909

[151] Matt Tanner. 2025. The Ultimate Guide to the Best SQL Database Options in 2025. *SingleStore*. Retrieved September 4, 2026 from https://www.singlestore.com/blog/ultimate-guide-best-sql-database-options-2025/

[152] Sharoz Tanveer. 2024. Functions vs Classes: When to Use Which and Why? *DEV Community*. Retrieved March 16, 2026 from https://dev.to/sharoztanveer/functions-vs-classes-when-to-use-which-and-why-48d6

[153] EPM Information Development Team. Batch Processing. *Oracle Help Center*. Retrieved September 4, 2026 from https://docs.oracle.com/en/cloud/saas/enterprise-performance-management-common/erpia/batch_processing.html

[154] Morph Team. 2026. What Is AI Inference? How LLM Inference Turns a Prompt Into Tokens. *Morph*. Retrieved September 6, 2026 from https://www.morphllm.com/ai-inference

[155] Oracle Team and Franck Pachot. 2015. Does the block size matter? *dbi services*. Retrieved September 5, 2026 from https://www.dbi-services.com/blog/do-the-block-size-matter/

[156] The Postman Team. 2026. JSON Schema Data Types: A Complete Guide to Validation. *Postman Blog*. Retrieved February 11, 2026 from https://blog.postman.com/json-schema-data-types/

[157] Intelegain Technologies. 2019. What are web frameworks and why you need them? *Medium*. Retrieved September 4, 2026 from https://intelegain-technologies.medium.com/what-are-web-frameworks-and-why-you-need-them-c4e8806bd0fb

[158] T. Thomas, W. Weijermars, and E. Van Berkum. 2010. Predictions of Urban Volumes in Single Time Series. *IEEE Trans. Intell. Transp. Syst.* 11, 1 (March 2010), 71–80. https://doi.org/10.1109/tits.2009.2028149

[159] Tim. 2015. Multiple Block Sizes. *oracle-base*. Retrieved September 5, 2026 from https://oracle-base.com/articles/9i/multiple-block-sizes

[160] TQ. 2026. Vast AI Alternatives: Dedicated GPU for Enterprise AI. *Private AI Resource Center: Definitions, FAQs & Industry News*. Retrieved September 6, 2026 from https://www.onesourcecloud.net/cms/2026-vast-ai-alternatives.html

[161] Nicholas Tsim. 2020. Best practices for REST APIs: How to write endpoint URLs. *Nicholas Tsim*. Retrieved September 5, 2026 from https://ntsim.uk/posts/best-practices-for-rest-apis-endpoint-urls

[162] UserBenchmark.org. UserBenchmark - Real-time GPU benchmarking. *UserBenchmark*. Retrieved September 4, 2026 from https://userbenchmark.org/fr/

[163] Gyanaa Vaibhav. 2025. Scaling Systems — How to Handle More Users Without Breaking Everything. *Medium*. Retrieved September 4, 2026 from https://medium.com/@gynanrudr0/scaling-systems-how-to-handle-more-users-without-breaking-everything-39b873a67dc3

[164] Yury Velikanov. 2013. What Are the Best Oracle Database Load Testing Tools? *Pythian*. Retrieved September 5, 2026 from https://www.pythian.com/blog/oracle-database-load-testing-tools-overview

[165] Velorum. 2025. 7 FastAPI Images Under 100MB (Without Pain). *Medium*. Retrieved September 6, 2026 from https://medium.com/@1nick1patel1/7-fastapi-images-under-100mb-without-pain-0e455912a603

[166] John Vester. 2024. FastAPI Got Me an OpenAPI Spec Really... Fast. *DEV Community*. Retrieved February 23, 2026 from https://dev.to/johnjvester/fastapi-got-me-an-openapi-spec-really-fast-4ck0

[167] Vijay. 2025. How to Build a Lightning-Fast API with FastAPI and AsyncIO. *Medium*. Retrieved February 1, 2026 from https://python.plainenglish.io/how-to-build-a-lightning-fast-api-with-fastapi-and-asyncio-9f4e290d7ca5

[168] K Vinay Kumar. 2025. Optimising Linux Performance on IBM Cloud: Best Practices for RHEL & Ubuntu Servers. *Medium*. Retrieved September 15, 2026 from https://medium.com/@vinaykumar1998gangadkar/optimising-linux-performance-on-ibm-cloud-best-practices-for-rhel-ubuntu-servers-91c066ce5554

[169] Alice Vitard. 2026. “Nous ne pouvons pas mettre ces données dans des clouds hors de France ”: Faute d’offres jugées suffisamment souveraines, le ministère de la Justice développe son propre assistant d’IA générative. *L’Usine Digitale*. Retrieved September 5, 2026 from https://www.usine-digitale.fr/souverainete/nous-ne-pouvons-pas-mettre-ces-donnees-dans-des-clouds-hors-de-france-faute-doffres-jugees-suffisamment-souveraines-le-ministere-de-la-justice-developpe-son-propre-assistant-dia-generative.LCEKHYMEIRF3DDBYBWR3GM2LVU.html

[170] Jannik Wempe. 2021. Domain-Driven Design: Distinguish Entities, Value Objects. *Jannik Wempe*. Retrieved April 21, 2026 from https://wempe.dev/blog/domain-driven-design-entities-value-objects

[171] Indika Wimalasuriya. 2023. Amazon Prime Video’s 90% Cost Reduction throuh moving to Monolithic. *DEV Community*. Retrieved September 4, 2026 from https://dev.to/indika_wimalasuriya/amazon-prime-videos-90-cost-reduction-throuh-moving-to-monolithic-k4a

[172] WIPO. 2020. Annual financial report and financial statements 2020. Retrieved from HTTPs://www.wipo.int/edocs/pubdocs/en/wipo_pub_rn2021_18e.pdf

[173] Scott Wlaschin. Functional Programming Design Patterns. *F# for fun and profit*. Retrieved March 6, 2026 from https://fsharpforfunandprofit.com/fppatterns/

[174] Michał Wojciechowski. 2025. Clean Architecture and DDD in Practice 2025 - Complete Guide. *wojciechowski.app*. Retrieved April 20, 2026 from https://wojciechowski.app/en/articles/clean-architecture-domain-driven-design-2025

[175] Paul Wolf. 2021. Class Design — Python Guidelines 0.1.0 documentation. *Python Guidelines*. Retrieved March 17, 2026 from https://python-coding-guidelines.readthedocs.io/en/latest/class_design.html

[176] Wondelai. clean-architecture. *Skills*. Retrieved September 5, 2026 from https://www.skills.sh/wondelai/skills/clean-architecture

[177] Rayan Yassminh. 2024. Choosing the Right Cloud Service Model: A Guide from Simple to Advanced. *Medium*. Retrieved September 5, 2026 from https://medium.com/@ryassminh/choosing-the-right-cloud-service-model-a-guide-from-simple-to-advanced-68ac9d876d01

[178] Sławomir Zakrzewski. 2025. SQL Injection in the Age of ORM: Risks, Mitigations, and Best Practices. *Afine*. Retrieved September 4, 2026 from https://afine.com/sql-injection-in-the-age-of-orm-risks-mitigations-and-best-practices

[179] zsolt.ero. 2023. What is a request in load balancing pricing? - Application Performance / Load Balancing. *Cloudflare Community*. Retrieved September 5, 2026 from https://community.cloudflare.com/t/what-is-a-request-in-load-balancing-pricing/588265

[180] 2006. DBWR Multiblock Writes? Yes, Indeed! *Kevin Closson's Blog: Platforms, Databases and Storage*. Retrieved September 5, 2026 from https://kevinclosson.net/2006/12/01/dbwr-multiblock-writes-yes-indeed/

[181] 2013. *Swingbench Tutorial for Oracle 12c Load Testing and Benchmarking*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=bovbie3kGGw

[182] 2014. SLOB Deployment – A Picture Tutorial. *Kevin Closson's Blog: Platforms, Databases and Storage*. Retrieved September 5, 2026 from https://kevinclosson.net/2014/08/04/slob-deployment-a-picture-tutorial/

[183] 2015. *What Impact Do Indexes Have on Inserts?: Finding All the Red Sweets Part 0*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=f3U9F_wbo1I

[184] 2017. *Redo Logs in an Oracle Database - DBArch Video 11*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=-oPdDx9JYp4

[185] 2018. *What should be the Database block size? - DBPerf Video 15*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=xGMaThyqvEs

[186] 2019. *048 Oracle DBA Complete Tutorial - Temporary Tablespace Management.* Retrieved September 5, 2026 from https://www.youtube.com/watch?v=9WR3-22KNQ0

[187] 2020. Exxact TensorEX Servers Now Accelerating HPC and AI Workloads with NVIDIA V100S Tensor Core GPUs |. *Exxact Corp Blog*. Retrieved September 6, 2026 from https://www.exxactcorp.com/blog/HPC/exxact-tensorex-servers-now-accelerating-hpc-and-ai-workloads-with-nvidia-v100s-tensor-core-gpus

[188] 2021. *Oracle Database Performance with NetApp*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=_YuSmeJmsco

[189] 2021. *The Strategy Pattern: Write BETTER PYTHON CODE Part 3*. Retrieved March 2, 2026 from https://www.youtube.com/watch?v=WQ8bNdxREHU

[190] 2021. Best practices for REST API security: Authentication and authorization - Stack Overflow. Retrieved February 21, 2026 from https://stackoverflow.blog/2021/10/06/best-practices-for-authentication-and-authorization-for-rest-apis/

[191] 2022. *Stored Procedures are Better than Raw SQL*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=hTLW5fKbMCo

[192] 2022. TechEmpower Web Framework Performance Comparison. *TechEmpower*. Retrieved September 4, 2026 from https://www.techempower.com/benchmarks/#section=data-r21&test=db

[193] 2022. Difference Between Relational and Non Relational Database. *NebulaGraph*. Retrieved September 4, 2026 from https://nebula-graph.io/posts/difference-between-relational-and-non-relational-database

[194] 2023. Database Capacity Planning: From Metrics to Growth Projections. *JusDB*. Retrieved September 5, 2026 from https://www.jusdb.com/blog/database-capacity-planning-metrics-growth

[195] 2023. *What is Coupling and How Can You Avoid It?* Retrieved February 26, 2026 from https://www.youtube.com/shorts/XJGe6fP7Bco

[196] 2023. *Raw SQL, SQL Query Builder, or ORM?* Retrieved February 19, 2026 from https://www.youtube.com/watch?v=x1fCJ7sUXCM

[197] 2023. What to consider when deciding which database block size to use. *Progress Community*. Retrieved September 5, 2026 from https://community.progress.com/s/article/18293

[198] 2023. Federal Risk and Authorization Management Program (FedRAMP). *CMS CyberGeek*. Retrieved September 5, 2026 from https://security.cms.gov/learn/fedramp

[199] 2023. *Why Use Design Patterns When Python Has Functions?* Retrieved March 2, 2026 from https://www.youtube.com/watch?v=vzTrLpxPF54

[200] 2023. TechEmpower Web Framework Performance Comparison - round 22 - single query. *www.techempower.com*. Retrieved September 6, 2026 from https://www.techempower.com/benchmarks/#section=data-r22&test=db

[201] 2024. Choosing a Database for Your Project: Step by Step. *NebulaGraph*. Retrieved September 4, 2026 from https://nebula-graph.io/posts/choosing-a-database-for-your-project-step-by-step

[202] 2024. GitHub - kgrzybek/modular-monolith-with-ddd: Full Modular Monolith application with Domain-Driven Design approach. · GitHub. *GitHub*. Retrieved September 4, 2026 from https://github.com/kgrzybek/modular-monolith-with-ddd

[203] 2024. Concurrency and Parallelism: Understanding I/O. *RisingStack Engineering*. Retrieved February 6, 2026 from https://blog.risingstack.com/concurrency-and-parallelism-understanding-i-o/

[204] 2024. United States - Corporate - Taxes on corporate income. *PwC*. Retrieved September 16, 2026 from https://taxsummaries.pwc.com/united-states/corporate/taxes-on-corporate-income

[205] 2024. *SQLModel + FastAPI: Say Goodbye to Repetitive Database Code*. Retrieved February 17, 2026 from https://www.youtube.com/watch?v=GONyd0CUrPc

[206] 2024. *Why PostGreSQL is better than MS SQL*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=wRRTZZH54Iw

[207] 2024. comparing Domain-Driven Design (DDD) and Clean Architecture. *DEV Community*. Retrieved March 6, 2026 from https://dev.to/sallbro/comparing-domain-driven-design-ddd-and-clean-architecture-8d

[208] 2024. *This Tool Replaces pip, Poetry, pyenv, and More (It's Fast)*. Retrieved February 6, 2026 from https://www.youtube.com/watch?v=qh98qOND6MI

[209] 2024. What's the difference between cloud and virtualization? *RedHat*. Retrieved September 5, 2026 from https://www.redhat.com/en/topics/cloud-computing/cloud-vs-virtualization

[210] 2025. *Why Pydantic Is a Must for Data Validation*. Retrieved from https://youtube.com/shorts/H6SeIkCybts?si=zw4akAI6BfBtBfSl

[211] 2025. TechEmpower Web Framework Performance Comparison. *TechEmpower*. Retrieved September 4, 2026 from https://www.techempower.com/benchmarks/#section=data-r23&test=db&s=1&l=zijzen-pa7&w=zik0zj-zik0zj-zik0zj-hra0hr&a=2&o=b

[212] 2025. SUPERMICRO RACK SCALE LIQUID COOLING SOLUTIONS. Retrieved from https://www.supermicro.com/solutions/Solution-Brief_Supermicro_Liquid_Cooling_Solution_Guide.pdf

[213] 2025. Domain Driven Design. *DEV Community*. Retrieved March 2, 2026 from https://dev.to/lovestaco/domain-driven-design-3i2j

[214] 2025. RabbitMQ 4.1 Performance Improvements | RabbitMQ. *RabbitMQ*. Retrieved September 12, 2026 from https://www.rabbitmq.com/blog/2025/04/08/4.1-performance-improvements

[215] 2025. Coupling and Cohesion - Software Engineering. *GeeksforGeeks*. Retrieved February 26, 2026 from https://www.geeksforgeeks.org/software-engineering/software-engineering-coupling-and-cohesion/

[216] 2025. *Oracle Linux: The Ideal Choice for Running Oracle Database*. Oracle. Retrieved from https://www.oracle.com/cn/a/ocom/docs/linux/oracle-linux-the-ideal-choice-for-running-oracle-database.pdf

[217] 2025. Here's Why HDDs are Still the Preferred Option Over SSDs! *Silicon Power*. Retrieved September 5, 2026 from https://www.silicon-power.com/knowledge-detail/ssd-vs-hdd/

[218] 2025. Faster Python: Concurrency in async/await and threading | The PyCharm Blog. *The JetBrains Blog*. Retrieved February 2, 2026 from https://blog.jetbrains.com/pycharm/2025/06/concurrency-in-async-await-and-threading/

[219] 2025. Oracle License Definitions and Rules Booklet. Retrieved from https://www.oracle.com/contracts/docs/lic_definitions_and_rules_v061525.pdf

[220] 2025. Server-side rendering (SSR) - Glossary | MDN. *MDN Web Docs*. Retrieved September 16, 2026 from https://developer.mozilla.org/en-US/docs/Glossary/SSR

[221] 2025. Domain-Driven Design (DDD). *GeeksforGeeks*. Retrieved March 5, 2026 from https://www.geeksforgeeks.org/system-design/domain-driven-design-ddd/

[222] 2025. Asyncio Vs Threading In Python. *GeeksforGeeks*. Retrieved February 2, 2026 from https://www.geeksforgeeks.org/python/asyncio-vs-threading-in-python/

[223] 2025. Server vs. Serverless Architecture. *GeeksforGeeks*. Retrieved September 5, 2026 from https://www.geeksforgeeks.org/system-design/server-vs-serverless-architecture/

[224] 2025. Run multiple models - General. *vLLM Forums*. Retrieved September 5, 2026 from https://discuss.vllm.ai/t/run-multiple-models/1181

[225] 2025. Oracle vs. PostgreSQL: Key Differences and Best Use Cases. *EDB*. Retrieved September 4, 2026 from https://enterprisedb.com/blog/oracle-vs-postgresql-comparison

[226] 2025. *UV Just Got A Serious Upgrade*. Retrieved February 6, 2026 from https://www.youtube.com/watch?v=CJJzrtHeLGI

[227] 2025. Understanding Temporary Tablespaces: Monitoring and Troubleshooting in Oracle. *Oracle DBA*. Retrieved September 5, 2026 from https://odba.in/understanding-temporary-tablespaces-monitoring-and-troubleshooting-in-Oracle/

[228] 2025. Oracle 23ai Databases: Free or Enterprise Guide. Retrieved September 9, 2026 from https://www.ncs-london.com/blog/oracle-23ai-free-vs-enterprise-which-version-is-right-for-your-business/

[229] 2025. IBM Cloud for Financial Services. *IBM*. Retrieved September 10, 2026 from https://www.ibm.com/products/cloud/financial-services

[230] 2025. Connect to Oracle using Python. *Devart*. Retrieved February 5, 2026 from https://docs.devart.com/odbc-driver-for-oracle/using-from-programming-languages/use-the-driver-from-python.html

[231] 2025. Combining asyncio with Multi-Worker ProcessPoolExecutor: How to Run Blocking Functions Concurrently and Optimize Execution Time. *pythontutorials.net*. Retrieved February 6, 2026 from https://www.pythontutorials.net/blog/combining-asyncio-with-a-multi-worker-processpoolexecutor/

[232] 2025. Categorizing Web APIs | Overview | Gravitee Documentation. Retrieved February 19, 2026 from https://documentation.gravitee.io/platform-overview/before-you-begin/api-fundamentals/categorizing-web-apis

[233] 2025. *Hexagonal vs Clean vs Onion Architecture... It Doesn't Matter*. Retrieved February 26, 2026 from https://www.youtube.com/watch?v=Thzz5bvUI40

[234] 2025. *How Bare Metal is different from Cloud Computing and On Premise Servers*. Retrieved September 6, 2026 from https://www.youtube.com/shorts/5rlwc-CaStk

[235] 2025. Qwen/Qwen3-4B. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/Qwen/Qwen3-4B

[236] 2026. aio-libs/aioodbc. Retrieved February 4, 2026 from https://github.com/aio-libs/aioodbc

[237] 2026. Differences between static and dynamic SQL. *IBM*. Retrieved September 5, 2026 from https://www.ibm.com/docs/en/db2-for-zos/12.0.0?topic=program-differences-between-static-dynamic-sql

[238] 2026. Secret Management Service. Retrieved September 4, 2026 from https://docs.oracle.com/en-us/iaas/Content/secret-management/home.htm

[239] 2026. Oracle Processor Core Factor Table. Retrieved from https://www.oracle.com/contracts/docs/processor-core-factor-table-070634.pdf?download=false

[240] 2026. BAAI/bge-m3. *Hugging Face*. Retrieved September 4, 2026 from https://huggingface.co/BAAI/bge-m3

[241] 2026. *Bare Minimum Setup You Need to Run AI Agents Locally*. Retrieved September 5, 2026 from https://www.youtube.com/watch?v=77FU71SzaUM

[242] 2026. Introduction to API (Application Programming Interface) - GeeksforGeeks. *GeeksforGeeks*. Retrieved September 4, 2026 from https://www.geeksforgeeks.org/software-testing/what-is-an-api/

[243] 2026. Top Document Extraction Software: From Legacy OCR to Agentic AI. *llamaIndex*. Retrieved September 4, 2026 from https://www.llamaindex.ai/insights/top-document-extractaction-software

[244] 2026. Securely deliver applications with Cloudflare. *Cloudflare Docs*. Retrieved September 4, 2026 from https://developers.cloudflare.com/reference-architecture/design-guides/secure-application-delivery/

[245] 2026. SOLID Principles with Real Life Examples - GeeksforGeeks. *GeeksforGeeks*. Retrieved February 26, 2026 from https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/

[246] 2026. Quickstart. *Cloudflare Docs*. Retrieved September 5, 2026 from https://developers.cloudflare.com/load-balancing/get-started/quickstart/

[247] 2026. Geo. *Cloudflare Docs*. Retrieved September 4, 2026 from https://developers.cloudflare.com/load-balancing/understand-basics/traffic-steering/steering-policies/geo-steering/

[248] 2026. What Is UptimeRobot? *UptimeRobot Knowledge Hub*. Retrieved September 4, 2026 from https://uptimerobot.com/knowledge-hub/monitoring/what-is-uptimerobot/

[249] 2026. Rate limiting parameters. *Cloudflare Docs*. Retrieved September 4, 2026 from https://developers.cloudflare.com/waf/rate-limiting-rules/parameters/

[250] 2026. *The Best Local Agentic Coding Workflow (Complete Guide)*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=UngVdAsQEiU

[251] 2026. GPU Pricing 2026: Wholesale Rates vs Hyperscaler List Prices. Retrieved September 6, 2026 from https://gpuaas.com/blog/gpu-pricing

[252] 2026. *How API Rate Limiting Actually Works and How to Build Your Own*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=s3wIoIN_-u4

[253] 2026. SLOB – The Silly Little Oracle Benchmark. *flashdba*. Retrieved September 5, 2026 from https://flashdba.com/slob/

[254] 2026. *MCP vs API: Why traditional APIs are failing AI agents*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=185XGEMefgc

[255] 2026. Oracle Technology Global Price List. Retrieved from https://www.oracle.com/europe/a/ocom/docs/corporate/pricing/technology-price-list-070617.pdf

[256] 2026. HTTP caching - HTTP. *MDN Web Docs*. Retrieved September 5, 2026 from https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching

[257] 2026. *Why Composition Over Inheritance Isn't Always Better*. Retrieved September 4, 2026 from https://www.youtube.com/watch?v=JsWQ0jFM-jc

[258] OLTP vs. OLAP: Differences and Applications. *Snowflake*. Retrieved September 4, 2026 from https://www.snowflake.com/en/fundamentals/olap-vs-oltp-the-differences/

[259] OLTP vs OLAP - Difference Between Data Processing Systems - AWS. *Amazon Web Services, Inc.* Retrieved September 4, 2026 from https://aws.amazon.com/compare/the-difference-between-olap-and-oltp/

[260] What is SSO? | How single sign-on works. *Cloudflare*. Retrieved September 16, 2026 from https://www.cloudflare.com/learning/access-management/what-is-sso/

[261] Architecture Decision Records (ADRs): The 2026 Guide | Catio. Retrieved September 4, 2026 from https://www.catio.tech/blog/architecture-decision-record

[262] Geekbench 7 - Cross-Platform Benchmark. Retrieved September 4, 2026 from https://www.geekbench.com/

[263] Cloud Email Sending Service - Amazon Simple Email Service. *Amazon Web Services, Inc.* Retrieved September 12, 2026 from https://aws.amazon.com/ses/

[264] Cross Site Scripting Prevention. *OWASP Cheat Sheet Series*. Retrieved September 4, 2026 from https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[265] Rate Limiting. *Redis*. Retrieved September 4, 2026 from https://redis.io/glossary/rate-limiting/

[266] Security. *Academy*. Retrieved September 4, 2026 from https://academy.optersoft.com/python/fastapi/security

[267] API — Jinja Documentation (3.1.x). *Jinja*. Retrieved September 4, 2026 from https://jinja.palletsprojects.com/en/stable/api/

[268] SlowApi Documentation. *Documentation*. Retrieved September 4, 2026 from https://slowapi.readthedocs.io/en/stable/

[269] Cloudflare Bot Management - Stop Bad Bots. *Cloudflare*. Retrieved September 4, 2026 from https://www.cloudflare.com/products/bot-mitigation/

[270] Web Security Platform - WAF, DDoS Protection, Bot Management. *Cloudflare*. Retrieved September 4, 2026 from https://www.cloudflare.com/solutions/security/

[271] Cloudflare Rate Limiting - API Abuse Prevention. *Cloudflare*. Retrieved September 4, 2026 from https://www.cloudflare.com/products/rate-limiting/

[272] What is Database Scalability? Definition & FAQs | ScyllaDB. *ScyllaDB*. Retrieved September 4, 2026 from https://www.scylladb.com/glossary/database-scalability/

[273] LM-Kit.NET VLM OCR - Vision Model Text Extraction from Images and PDFs in C# .NET. *LM-Kit.NET*. Retrieved September 4, 2026 from https://docs.lm-kit.com/lm-kit-net/guides/samples-overview/vlm-ocr-demo.html

[274] Web APIs - introduction. *W3 Schools*. Retrieved September 4, 2026 from https://www.w3schools.com/js/js_api_intro.asp

[275] REST APIs Explained: How They Work and Why They’re Essential. Retrieved February 21, 2026 from https://dnsmadeeasy.com/resources/rest-apis-explained-how-they-work-and-why-theyre-essential

[276] Web API vs. REST API: Comparing RESTful and Non-RESTful Web APIs. Retrieved February 19, 2026 from https://www.catchpoint.com/api-monitoring-tools/web-api-vs-rest-api

[277] JWT vs Session-Based Auth: Pros and Cons - System Design Interview Guide. *bugfree.ai*. Retrieved February 21, 2026 from https://www.bugfree.ai/knowledge-hub/jwt-vs-session-based-auth-pros-and-cons

[278] Review: Stop Using JWT for Sessions. Retrieved February 21, 2026 from https://www.ducktypelabs.com/review-stop-using-jwt-for-sessions/

[279] JWTs vs. sessions: which authentication approach is right for you? Retrieved February 21, 2026 from https://stytch.com/blog/jwts-vs-sessions-which-is-right-for-you/

[280] Benchmarks - FastAPI. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/benchmarks/#benchmarks-and-speed

[281] Concurrency and async / await. *FastAPI*. Retrieved February 1, 2026 from https://fastapi.tiangolo.com/async/

[282] Python Types Intro. *FastAPI*. Retrieved February 18, 2026 from https://fastapi.tiangolo.com/python-types/

[283] Frontend - FastAPI. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/tutorial/frontend/

[284] Templating - Jinja2Templates - FastAPI. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/reference/templating/

[285] What is Domain-Driven Design? Benefits, Challenges & Implementation. Retrieved March 6, 2026 from https://www.port.io/glossary/domain-driven-design

[286] Database Caching Strategies – Database Guide. *Solarwinds*. Retrieved September 4, 2026 from https://www.solarwinds.com/database-optimization/database-caching

[287] Template Designer Documentation. *Jinja Documentation (3.1.x)*. Retrieved September 4, 2026 from https://jinja.palletsprojects.com/en/stable/templates/

[288] hx-swap Attribute. *HTMX*. Retrieved September 4, 2026 from https://htmx.org/attributes/hx-swap/

[289] htmx ~ Documentation. *HTMX*. Retrieved September 4, 2026 from https://htmx.org/docs/#requests

[290] What is Embedding? - Embeddings in Machine Learning Explained - AWS. *Amazon Web Services, Inc.* Retrieved September 4, 2026 from https://aws.amazon.com/what-is/embeddings-in-machine-learning/

[291] Which type of concurrency is best for CPU-bound programs? *Design Gurus*. Retrieved February 1, 2026 from https://www.designgurus.io/answers/detail/which-type-of-concurrency-is-best-for-cpu-bound-programs

[292] Sync vs. Async. *Litestar*. Retrieved February 1, 2026 from https://docs.litestar.dev/main/topics/sync-vs-async.html

[293] Developing with asyncio. *Python documentation*. Retrieved September 4, 2026 from https://docs.python.org/3/library/asyncio-dev.html

[294] Asynchronous I/O (asyncio) — SQLAlchemy 2.0 Documentation. *SQLAlchemy*. Retrieved September 7, 2026 from https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html

[295] Concurrent Programming with asyncio. *python-oracledb 2.3.0 documentation*. Retrieved February 5, 2026 from https://python-oracledb.readthedocs.io/en/v2.3.0/user_guide/asyncio.html

[296] Configuration and defaults. *Celery 5.6.2 Documentation*. Retrieved January 30, 2026 from https://docs.celeryq.dev/en/main/userguide/configuration.html

[297] Oracle Instant Client - Free tools and libraries for connecting to Oracle Database. *Oracle*. Retrieved February 5, 2026 from https://www.oracle.com/database/technologies/instant-client.html

[298] What is an ODBC compliant database? *Portable Technology Solutions*. Retrieved February 4, 2026 from https://www.ptsmobile.com/what-is-odbc.htm

[299] pyodbc: DB API module for ODBC. Retrieved February 5, 2026 from https://github.com/mkleehammer/pyodbc

[300] aio-libs/aiohttp: Asynchronous HTTP client/server framework for asyncio and Python. *GitHub*. Retrieved February 4, 2026 from https://github.com/aio-libs/aiohttp

[301] dataclasses — Data Classes. *Python documentation*. Retrieved March 16, 2026 from https://docs.python.org/3/library/dataclasses.html

[302] Coroutines and Tasks. *Python documentation*. Retrieved February 23, 2026 from https://docs.python.org/3/library/asyncio-task.html

[303] APIRouter class - FastAPI. *FastAPi*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/reference/apirouter/

[304] Response Model - Return Type. *FastAPI*. Retrieved September 4, 2026 from https://fastapi.tiangolo.com/tutorial/response-model/

[305] API keys. *Stripe docs*. Retrieved September 4, 2026 from https://docs.stripe.com/keys

[306] Develop a KNIME Extension with Python. *Knime*. Retrieved September 5, 2026 from https://docs.knime.com/developers/latest/create_a_node_with_python/

[307] Using uv with FastAPI. *Astral*. Retrieved September 5, 2026 from https://docs.astral.sh/uv/guides/integration/fastapi/

[308] Working on projects. *Astral*. Retrieved September 5, 2026 from https://docs.astral.sh/uv/guides/projects/

[309] SQLModel. *SQLModel*. Retrieved September 5, 2026 from https://sqlmodel.tiangolo.com/#editor-support-everywhere

[310] SQL (Relational) Databases - FastAPI. *fastAPI*. Retrieved September 5, 2026 from https://fastapi.tiangolo.com/tutorial/sql-databases/

[311] 8. Using Bind Variables. *Python-oracledb 26.0.0b1 Documentation*. Retrieved September 7, 2026 from https://python-oracledb.readthedocs.io/en/latest/user_guide/bind.html

[312] 7. API: Cursor Objects. *Python-oracledb 26.0.0b1 Documentation*. Retrieved September 5, 2026 from https://python-oracledb.readthedocs.io/en/latest/api_manual/cursor.html

[313] 30. Appendix B: Differences between python-oracledb Thin and Thick Modes. *python-oracledb 26.0.0b1 documentation*. Retrieved September 5, 2026 from https://python-oracledb.readthedocs.io/en/latest/user_guide/appendix_b.html

[314] Offline Inference - vLLM. *vLLM*. Retrieved September 5, 2026 from https://docs.vllm.ai/en/latest/serving/offline_inference/

[315] Use AI Models Locally. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/docs/hub/en/local-apps

[316] Agentic AI Document Processing & Extraction. *Unstract*. Retrieved September 5, 2026 from https://unstract.com/

[317] What Happens in a TLS Handshake? | SSL Handshake. *Cloudflare*. Retrieved September 5, 2026 from https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/

[318] Behind a Proxy. *FastAPI*. Retrieved September 5, 2026 from https://fastapi.tiangolo.com/advanced/behind-a-proxy/

[319] <input type="password"> HTML attribute value - HTML. *MDN Web Docs*. Retrieved September 5, 2026 from https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/password

[320] Client-side form validation - Learn web development. *MDN Web Docs*. Retrieved September 5, 2026 from https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

[321] Card verification checks. *Stripe docs*. Retrieved September 5, 2026 from https://docs.stripe.com/disputes/prevention/verification

[322] Request Body. *FastAPI*. Retrieved February 4, 2026 from https://fastapi.tiangolo.com/tutorial/body/

[323] Validators - Pydantic Validation. *Pydantic Docs*. Retrieved February 11, 2026 from https://docs.pydantic.dev/latest/concepts/validators/

[324] Models - Pydantic Validation. *Pydantic Docs*. Retrieved February 11, 2026 from https://docs.pydantic.dev/latest/concepts/models/

[325] Fields - Pydantic Validation. *Pydantic Docs*. Retrieved February 11, 2026 from https://docs.pydantic.dev/latest/concepts/fields/

[326] General Purpose VM pricing. *Google Cloud*. Retrieved September 5, 2026 from https://cloud.google.com/products/compute/pricing/general-purpose

[327] About Oracle Data Guard. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/en/cloud/paas/base-database/data-guard/index.html

[328] How Elastic Load Balancing works - Elastic Load Balancing. *AWS Documentation*. Retrieved September 5, 2026 from https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/how-elastic-load-balancing-works.html

[329] Resource constraints. *Docker Documentation*. Retrieved September 5, 2026 from https://docs.docker.com/engine/containers/resource_constraints/

[330] What is DNS? *Cloudflare*. Retrieved September 5, 2026 from https://www.cloudflare.com/learning/dns/what-is-dns/

[331] What is Amazon Route 53? *AWS Documentation*. Retrieved September 5, 2026 from https://docs.aws.amazon.com/en_en/Route53/latest/DeveloperGuide/Welcome.html

[332] Third-Party Integrations. *FastAPI Cloud Docs*. Retrieved September 5, 2026 from https://fastapicloud.com/docs/integrations/third-party-integrations/

[333] ISO/IEC 27001:2022. *ISO*. Retrieved September 5, 2026 from https://www.iso.org/standard/27001

[334] ISO/IEC 42001:2023. *ISO*. Retrieved September 5, 2026 from https://www.iso.org/standard/42001

[335] Uptime Robot - Crunchbase Company Profile & Funding. *Crunchbase*. Retrieved September 5, 2026 from https://www.crunchbase.com/organization/uptime-robot

[336] Run Inference on servers. *Hugging Face*. Retrieved September 5, 2026 from https://huggingface.co/docs/huggingface_hub/en/guides/inference

[337] Bare Metal Dedicated Servers. *OVHcloud*. Retrieved September 5, 2026 from https://us.ovhcloud.com/bare-metal/

[338] Bare Metal Dedicated Servers. *OpenMetal IaaS*. Retrieved September 5, 2026 from https://openmetal.io/products/bare-metal/

[339] Kubernetes vs Docker - Difference Between Container Technologies. *Amazon Web Services, Inc.* Retrieved September 5, 2026 from https://aws.amazon.com/compare/the-difference-between-kubernetes-and-docker/

[340] Configure Liveness, Readiness and Startup Probes. *Kubernetes*. Retrieved September 5, 2026 from https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[341] Oracle AI Database 26ai Technical Architecture - Log Writer Process (LGWR). *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/dbiad/db_LGWR.html

[342] Database Concepts - 3: Indexes and Index-Organized Tables. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/E11882_01/server.112/e40540/indexiot.htm#CNCPT721

[343] Choosing the Best Applicable Index. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/other-databases/nosql-database/24.4/sqlreferencefornosql/choosing-best-applicable-index.html

[344] DB_BLOCK_SIZE. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/B13789_01/server.101/b10755/initparams039.htm

[345] Oracle Database Online Documentation 11g, Release 2 (11.2), Database Performance Tuning Guide - 8.2.6 Choosing Data Block Size. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/E11882_01/server.112/e41573/iodesign.htm#PFGRF94404

[346] Transactions. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/21/cncpt/transactions.html

[347] Data Blocks, Extents, and Segments. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/A57673_01/DOC/server/doc/SCN73/ch3.htm#data%20bl%20format

[348] How Oracle Uses Dirty Buffers | LRU List to Manage Cache Space. *RelationalDBDesign*. Retrieved September 5, 2026 from https://www.relationaldbdesign.com/database-architecture/module4/writing-dirty-buffers.php

[349] Oracle 23c Database Writer (DBWn): Dirty Buffer Management, Checkpoints, Performance. *RelationalDBDesign*. Retrieved September 5, 2026 from https://www.relationaldbdesign.com/database-architecture/module4/dbw0-oracle-database-writer.php

[350] Oracle Database Online Documentation 12c Release 1 - Database Administrator's Guide - 12: Managing Archived Redo Log Files. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/database/121/ADMIN/archredo.htm

[351] Oracle® Database Administrator's Guide 11g Release 2 (11.2) - Planning the Redo Log. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/html/E25494_01/onlineredo002.htm

[352] Oracle Database - Release 18 - Database Concepts - 15. Process Architecture. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/18/cncpt/process-architecture.html

[353] Documentation, 12c Release 5 - Enterprise Manager ASM Config Assistant Context-Sensitive Online Help. *Oracle Help Center*. Retrieved September 8, 2026 from https://docs.oracle.com/cd/E24628_01/doc.121/e59369/usmca034.htm#USMCA186

[354] Oracle7 Tuning, release 7.3.3 - Tuning I/O. *Oracle*. Retrieved September 5, 2026 from https://docs.oracle.com/cd/A57673_01/DOC/server/doc/A48506/io.htm

[355] SQL Language Reference. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/file_specification.html

[356] Oracle Database - Release 18 - Database Performance Tuning Guide - 4 Configuring a Database for Performance. *Oracle Help Center*. Retrieved September 5, 2026 from https://docs.oracle.com/en/database/oracle/oracle-database/18/tgdba/configuring-database-for-performance.html

[357] AMD EPYC™ 9006 Server CPUs for AI-First Data Centers. *AMD*. Retrieved September 5, 2026 from https://www.amd.com/en/products/processors/server/epyc/9006-series.html

[358] AMD EPYC™ 9256. *AMD*. Retrieved September 5, 2026 from https://www.amd.com/en/products/processors/server/epyc/9006-series/amd-epyc-9256.html

[359] JSON dummy data. *Demos*. Retrieved September 5, 2026 from https://microsoftedge.github.io/Demos/json-dummy-data/

[360] BGE-M3 — BGE documentation. *BGE*. Retrieved September 5, 2026 from https://bge-model.com/bge/bge_m3.html

[361] FAQ - What are the KV cache sizes for popular models? And why is LMCache important? *LMCache*. Retrieved September 6, 2026 from https://docs.lmcache.ai/getting_started/faq.html

[362] Deployments Concepts - FastAPI. *FastAPI*. Retrieved September 6, 2026 from https://fastapi.tiangolo.com/deployment/concepts/

[363] Parallel Requests. *LM Studio Docs*. Retrieved September 6, 2026 from https://lmstudio.ai/docs/app/advanced/parallel-requests

[364] AI inference vs. training: What is AI inference? *Cloudflare*. Retrieved September 6, 2026 from https://www.cloudflare.com/learning/ai/inference-vs-training/

[365] NVIDIA Data Center Products: Next Wave of AI Innovation. *NVIDIA Data Center Products*. Retrieved September 6, 2026 from https://www.nvidia.com/en-gb/data-center/products/

[366] NVIDIA-Certified Systems Configuration Guide. *NVIDIA Certification Programs Documentation*. Retrieved September 6, 2026 from https://docs.nvidia.com/certification-programs/latest/nvidia-certified-configuration-guide.html

[367] NVIDIA Grace CPU Superchip Datasheet. *NVIDIA*. Retrieved September 6, 2026 from https://resources.nvidia.com/en-us-grace-cpu/data-center-datasheet

[368] GPUDirect Storage. *Nvidia*. Retrieved September 6, 2026 from https://docs.nvidia.com/gpudirect-storage/

[369] FastAPI - FastAPI. *FastAPI*. Retrieved September 6, 2026 from https://fastapi.tiangolo.com/

[370] Server Behavior. *Uvicorn*. Retrieved September 6, 2026 from https://uvicorn.dev/server-behavior/#timeouts

[371] Intel® Xeon® Gold 6330 Processor (42M Cache, 2.00 GHz) - Product Specifications. *Intel*. Retrieved September 6, 2026 from

https://www.intel.com/content/www/us/en/products/sku/212458/intel-xeon-gold-6330-processor-42m-cache-2-00-ghz/specifications.html

[372] Intel® Xeon® 6980E+ processor (528M Cache, 2.10 GHz) - Product Specifications. *Intel*. Retrieved September 6, 2026 from https://www.intel.com/content/www/us/en/products/sku/246075/intel-xeon-6980e-processor-528m-cache-2-10-ghz/specifications.html

[373] Akash Network - Decentralized Compute Marketplace. *Akash*. Retrieved September 6, 2026 from https://akash.network/

[374] Public Cloud vs Private Cloud - Difference Between Computing Environments - AWS. *Amazon Web Services, Inc.* Retrieved September 6, 2026 from https://aws.amazon.com/compare/the-difference-between-public-cloud-and-private-cloud/

[375] Network bandwidth | Compute Engine. *Google Cloud Documentation*. Retrieved September 6, 2026 from https://docs.cloud.google.com/compute/docs/network-bandwidth

[376] Amazon EC2 instance network bandwidth - Amazon Elastic Compute Cloud. *Amazon Web Services, Inc.* Retrieved September 6, 2026 from https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html

[377] Configurable Limits. *RabbitMQ*. Retrieved September 11, 2026 from https://www.rabbitmq.com/docs/limits

[378] Containers explained: What they are and why you should care. *RedHat*. Retrieved September 6, 2026 from https://www.redhat.com/en/topics/containers

[379] Use containerized databases. *Docker Documentation*. Retrieved September 6, 2026 from https://docs.docker.com/guides/databases/

[380] Pricing & Fees. *Stripe*. Retrieved September 9, 2026 from https://stripe.com/en-fr/pricing

[381] Dell 7.68TB SSD PCIe NVMe 2.5" Read Intensive 2533N New Pull. *ETB*. Retrieved September 6, 2026 from https://www.etb-tech.com/dell-7-68tb-ssd-pcie-nvme-2-5-read-intensive-2533n-new-pull-hd0818.html

[382] Dell 7.68 NVME Price - Dell Price List 2022. *IT Price*. Retrieved September 6, 2026 from https://itprice.com/dell-price-list/7.68%20nvme.html

[383] Gigabyte 1U 4Bays Single Processor Rackmount Server - R163-S30. *YMI RackmountPro*. Retrieved September 6, 2026 from https://www.rackmountpro.com/product/4604/R163-S30.html

[384] Server Colocation from $99.99/month. *KEEKLO*. Retrieved September 6, 2026 from https://keeklo.com/colocation.html

[385] 1U Colocation Pricing | $125/mo | No Contracts. *ColoByTheU*. Retrieved September 6, 2026 from https://colobytheu.us/pricing/

[386] Dedicated server prices - Explore our full server range. *OVHcloud*. Retrieved September 6, 2026 from https://www.ovhcloud.com/en/bare-metal/prices/

[387] Amazon EC2 High Memory U7i instances now available in AWS Europe (Paris) region - AWS. *Amazon Web Services, Inc.* Retrieved September 6, 2026 from https://aws.amazon.com/about-aws/whats-new/2026/05/amazon-ec2-u7i-aws-europe-paris/

[388] Seagate 24TB Exos X24 7200 rpm SATA III 6 Gb/s 3.5" Internal HDD (OEM). *B&H*. Retrieved September 6, 2026 from https://www.bhphotovideo.com/c/product/1806048-REG/seagate_st24000nm002h_24tb_exos_x24_3_5.html

[389] GODO 3.5 inch Dual Bay Hard Drive Enclosure,USB3.0 to 3.5" SATA HDDs External Enclosure,RAID 0/1/JBOD/PM Mode,Max 40TB RAID Hard Drive Docking Station,Aluminum: Amazon.co.uk: Computers & Accessories. *Amazon*. Retrieved September 6, 2026 from https://www.amazon.co.uk/GODO-3-5-Enclosure-SATA-External-Black/dp/B0CRH8TSJ7?th=1

[390] Seagate 24TB Exos X24 SAS 12Gb s 3.5" Internal Hard Drive (ST24000NM007H). *Connection*. Retrieved September 6, 2026 from https://www.connection.com/product/seagate-24tb-exos-x24-sas-12gb-s-3.5-internal-hard-drive/st24000nm007h/41738002

[391] QNAP TL-R2400PES-RP PCIe Interface 24-Bay JBOD Enclosure (TL-R2400PES-RP-US). *Connection*. Retrieved September 6, 2026 from https://www.connection.com/product/qnap-tl-r2400pes-rp-pcie-interface-24-bay-jbod-enclosure/tl-r2400pes-rp-us/41732624

[392] Storage pricing. *Google Cloud*. Retrieved September 6, 2026 from https://cloud.google.com/storage/pricing

[393] 1U 2 Node NVIDIA Grace/Grace CPU Superchip System. *Avantek Computer Limited*. Retrieved September 6, 2026 from https://store.avantek.co.uk/2u2n-nvidia-grace-grace-cpu-superchip-system.html

[394] Supermicro GPU SuperServer SYS-421GE-TNHR2-LCC, DP Intel 4U Liquid-Cooled System with NVIDIA HGX H100/H200 8-GPU, Complete System Only. *Newegg*. Retrieved September 6, 2026 from https://www.newegg.com/p/N82E16816139581

[395] Gigabyte H263-V60-AAW1 NVIDIA Superchip 2U 4-Node 16-Bay Rack-Mountable Server. *Saitech Inc*. Retrieved September 6, 2026 from https://esaitech.com/products/gigabyte-h263-v60-aaw1-nvidia-grace-cpu-superchip-2u-4-node-gen5-nvme-rackmount-server

[396] Electricity Cost Calculator: Any kWh (500=$92, 1,000=$184). *PriceOfElectricity.com*. Retrieved September 6, 2026 from https://www.priceofelectricity.com/electricity-cost-calculator

[397] The global colocation pricing & capacity index. *Coloprice*. Retrieved September 6, 2026 from https://coloprice.com/

[398] Nvidia GB200 vs GH200: Specs & Cloud Pricing (2026). *GetDeploying*. Retrieved September 6, 2026 from https://getdeploying.com/gpus/nvidia-gb200-vs-nvidia-gh200

[399] CoreWeave Cloud Pricing. *CoreWeave*. Retrieved September 6, 2026 from https://www.coreweave.com/pricing

[400] Comparing Amazon EBS volume types gp2 and gp3 - Amazon EMR. *Amazon Web Services, Inc.* Retrieved September 6, 2026 from https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-plan-storage-compare-volume-types.html

[401] Intel® Xeon® 6980E+ processor (528M Cache, 2.10 GHz) Ordering and Compliance Information. *Intel*. Retrieved September 6, 2026 from https://www.intel.com/content/www/us/en/products/sku/246075/intel-xeon-6980e-processor-528m-cache-2-10-ghz/ordering.html

[402] Base Database Features. *Oracle*. Retrieved September 9, 2026 from https://www.oracle.com/database/base-database-service/features/

[403] Base Database Service Pricing. *Oracle*. Retrieved September 9, 2026 from https://www.oracle.com/database/base-database-service/pricing/

[404] AMD EPYC™ 9756. *AMD*. Retrieved September 9, 2026 from https://www.amd.com/en/products/processors/server/epyc/9006-series/amd-epyc-9756.html

[405] AMD EPYC™ 9755. *AMD*. Retrieved September 9, 2026 from https://www.amd.com/en/products/processors/server/epyc/9005-series/amd-epyc-9755.html

[406] Create a new endpoint. *Runpod*. Retrieved September 10, 2026 from https://docs.runpod.io/api-reference/endpoints/POST/endpoints

[407] AI Infrastructure Security & Compliance. *Runpod*. Retrieved September 10, 2026 from https://www.runpod.io/legal/compliance

[408] The AI Developer Cloud. *Runpod*. Retrieved September 10, 2026 from https://www.runpod.io/

[409] Welcome to Runpod. *Runpod*. Retrieved September 10, 2026 from https://docs.runpod.io/overview

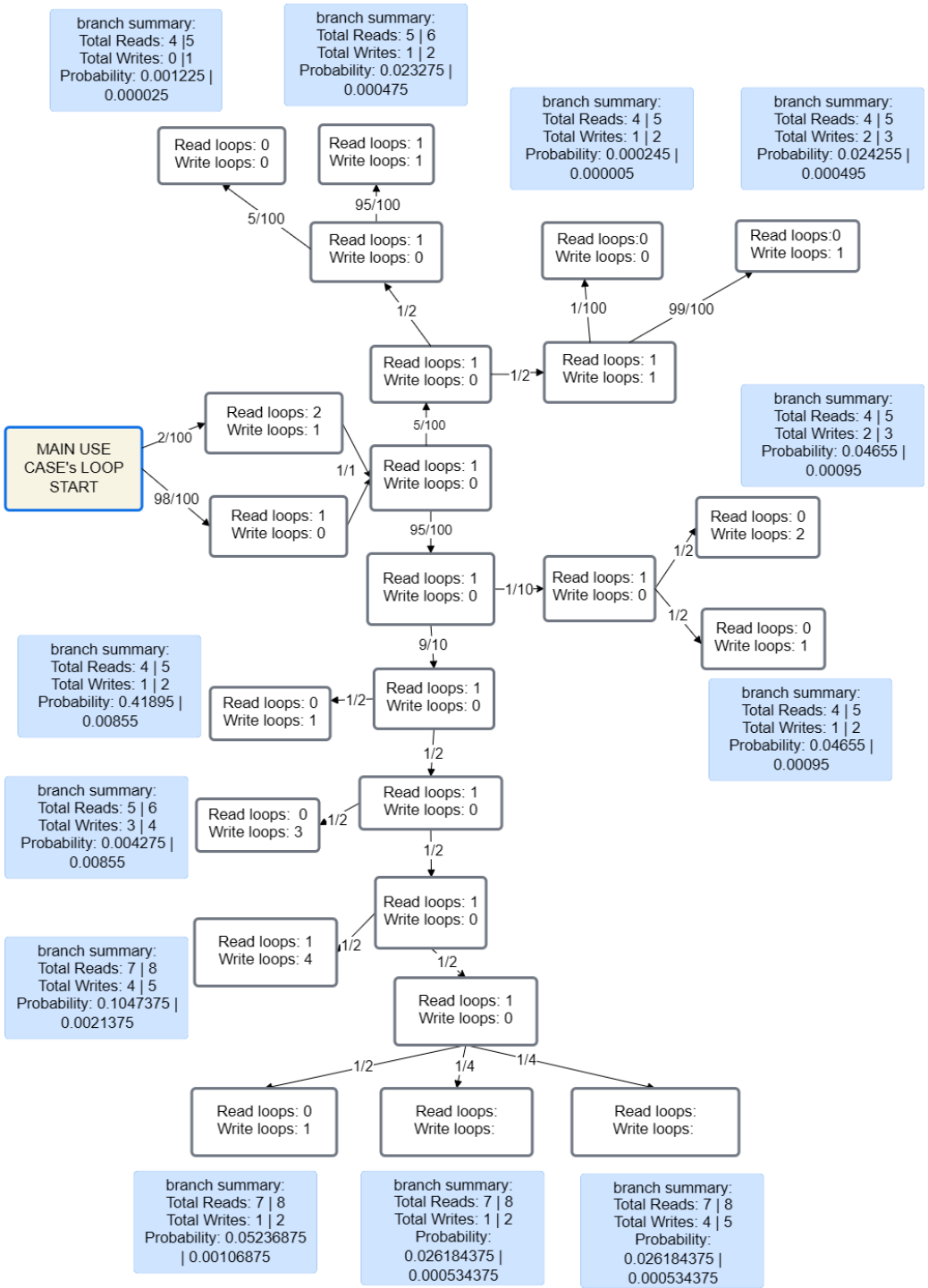
[410] IBM Cloud FedRAMP compliance. *IBM*. Retrieved September 15, 2026 from https://www.ibm.com/products/cloud/compliance/fedramp

[411] NVIDIA GPUs on IBM Cloud. *IBM*. Retrieved September 10, 2026 from https://www.ibm.com/products/gpu-ai-accelerator/nvidia

[412] Beyond Trends: A Practical Guide to Choosing the Right Message Broker. *InfoQ*. Retrieved September 11, 2026 from https://www.infoq.com/articles/choosing-message-broker/

Appendix 1: probabilistic tree of write and read loops happening during 1 loop of the main use case

The blue blocks sum up the read and write loops when the use case loop takes that orchestration path. After the first division in the execution of the use case's loop, I visually merge routes again because both logical tree sections from that point onwards are similar. Thus, each terminal branch displayed in the diagram represents two terminal branches, and the "|" separates data of these two branches.



Appendix 2: samples of the AWR Report of a DB of my Oracle contact:

For reminder, as said in the text the report shows two sessions (see left column of the first table under the “I#” cell, you can see session 1 and session 2) and both count in the analysis.

System Statistics

I#	Logical Reads	Physical Reads	Physical Writes	Redo Size (k)	Block Changes	User Calls	Execs	Parses	Logons	Txns
1	2,925,369	4,113	8,346	48,364	324,318	2,854	121,079	67,371	396	94
2	1,235,811	60	2,172	7,441	28,740	3,433	7,014	5,595	433	26
Sum	4,161,180	4,173	10,518	55,805	353,058	6,287	128,093	72,966	829	120
Avg	2,080,590	2,087	5,259	27,903	176,529	3,144	64,047	36,483	415	60
Std	1,194,698	2,866	4,366	28,937	209,005	409	80,656	43,682	26	48

Back to Global Activity Load Profile
Back to Top

System Statistics - Per Second

I#	Logical Reads/s	Physical Reads/s	Physical Writes/s	Redo Size (k)/s	Block Changes/s	User Calls/s	Execs/s	Parses/s	Logons/s	Txns/s
1	812.07	1.14	2.32	13.43	90.03	0.79	33.61	18.70	0.11	0.03
2	343.06	0.02	0.60	2.07	7.98	0.95	1.95	1.55	0.12	0.01
Sum	1,155.13	1.16	2.92	15.49	98.01	1.75	35.56	20.26	0.23	0.03
Avg	577.57	0.58	1.46	7.75	49.00	0.87	17.78	10.13	0.12	0.02
Std	331.64	0.80	1.21	8.03	58.02	0.11	22.39	12.13	0.01	0.01

Back to Global Activity Load Profile
Back to Top

System Statistics - Per Transaction

I#	Logical Reads/tx	Physical Reads/tx	Physical Writes/tx	Redo Size (k)/tx	Block Changes/tx	User Calls/tx	Execs/tx	Parses/tx	Logons/tx
1	31,120.95	43.76	88.79	514.51	3,450.19	30.36	1,288.07	716.71	4.21
2	47,531.19	2.31	83.54	286.21	1,105.38	132.04	269.77	215.19	16.65
Avg	39,326.07	23.03	86.16	400.36	2,277.79	81.20	778.92	465.95	10.43

				per Second			
Statistic	Total	per Second	per Trans	Average	Std Dev	Min	Max

physical read IO requests	2,780	0.77	23.17	0.39	0.52	0.02	0.76
physical read bytes	34,185,216	9,489.73	284,876.80	4,744.86	6,517.29	136.44	9,353.28
physical read total IO requests	34,409	9.55	286.74	4.78	0.59	4.36	5.19
physical read total bytes	651,311,616	180,802.39	5,427,596.80	90,401.20	27,118.06	71,225.83	109,576.56
physical read total multi block requests	156	0.04	1.30	0.04		0.04	0.04
physical reads	4,173	1.16	34.78	0.58	0.80	0.02	1.14
physical reads cache	4,173	1.16	34.78	0.58	0.80	0.02	1.14
physical reads cache prefetch	1,800	0.50	15.00	0.50		0.50	0.50
physical write IO requests	4,895	1.36	40.79	0.68	0.44	0.37	0.99
physical write bytes	86,163,456	23,918.75	718,028.80	11,959.37	9,927.89	4,939.30	18,979.45
physical write total IO requests	29,055	8.07	242.13	4.03	0.69	3.54	4.52
physical write total bytes	300,548,096	83,431.36	2,504,567.47	41,715.68	28,596.54	21,494.87	61,936.49
physical write total multi block requests	319	0.09	2.66	0.04	0.05	0.01	0.08
physical writes	10,518	2.92	87.65	1.46	1.21	0.60	2.32
physical writes direct	49	0.01	0.41	0.01	0.00	0.01	0.01
physical writes direct (lob)	4	0.00	0.03	0.00	0.00	0.00	0.00
physical writes direct temporary tablespace	45	0.01	0.38	0.01	0.00	0.01	0.01
physical writes from cache	10,469	2.91	87.24	1.45	1.21	0.60	2.31
physical writes non checkpoint	6,247	1.73	52.06	0.87	0.84	0.27	1.46
pinned cursors current	5	0.00	0.04	0.00	0.00	0.00	0.00
prefetched blocks aged out before use	2	0.00	0.02	0.00		0.00	0.00
process last non-idle time	7,199	2.00	59.99	1.00	0.00	1.00	1.00
queries parallelized	113	0.03	0.94	0.02	0.02	0.00	0.03
recursive calls	414,713	115.12	3,455.94	57.56	55.85	18.07	97.05
recursive cpu usage	11,333	3.15	94.44	1.57	0.68	1.09	2.05
redo blocks checksummed by FG (exclusive)	14,698	4.08	122.48	2.04	0.07	1.99	2.09
redo blocks written	121,979	33.86	1,016.49	16.93	16.68	5.13	28.73
redo blocks written (group 0)	120,808	33.54	1,006.73	16.77	16.61	5.02	28.52
redo blocks written (group 1)	1,171	0.33	9.76	0.16	0.07	0.11	0.21
redo entries	188,165	52.23	1,568.04	26.12	30.11	4.83	47.41
redo ordering marks	329	0.09	2.74	0.05	0.00	0.04	0.05
redo size	57,144,732	15,863.23	476,206.10	7,931.61	8,225.49	2,115.31	13,747.91
redo subscn max counts	306	0.08	2.55	0.04	0.02	0.03	0.06
redo synch time (usec)	32,201	8.94	268.34	4.47	5.85	0.33	8.61

				per Second			
Statistic	Total	per Second	per Trans	Average	Std Dev	Min	Max

redo blocks written	121,979	33.86	1,016.49	16.93	16.68	5.13	28.73
redo blocks written (group 0)	120,808	33.54	1,006.73	16.77	16.61	5.02	28.52
redo blocks written (group 1)	1,171	0.33	9.76	0.16	0.07	0.11	0.21
redo entries	188,165	52.23	1,568.04	26.12	30.11	4.83	47.41
redo ordering marks	329	0.09	2.74	0.05	0.00	0.04	0.05
redo size	57,144,732	15,863.23	476,206.10	7,931.61	8,225.49	2,115.31	13,747.91
redo subscn max counts	306	0.08	2.55	0.04	0.02	0.03	0.06
redo synch time (usec)	32,201	8.94	268.34	4.47	5.85	0.33	8.61
redo synch time overhead (usec)	2,570	0.71	21.42	0.36	0.45	0.04	0.68
redo synch time overhead count (2ms)	43	0.01	0.36	0.01	0.01	0.00	0.01
redo synch writes	318	0.09	2.65	0.04	0.00	0.04	0.04
redo wastage	3,260,508	905.11	27,170.90	452.55	8.19	446.77	458.34
redo write active strands	9,280	2.58	77.33	1.29	0.06	1.24	1.33
redo write broadcast ack count	173	0.05	1.44	0.02	0.01	0.02	0.03
redo write broadcast ack time	23,507	6.53	195.89	3.26	0.47	2.93	3.60
redo write finish time	8,671,097	2,407.07	72,259.14	1,203.54	57.35	1,162.98	1,244.09
redo write gather time	1,074,126	298.17	8,951.05	149.09	9.66	142.25	155.92
redo write info find	43	0.01	0.36	0.01	0.01	0.00	0.01
redo write issue time	1,274,434	353.78	10,620.28	176.89	9.19	170.39	183.39
redo write schedule time	1,266,898	351.69	10,557.48	175.84	9.19	169.34	182.34
redo write size count (4KB)	8,790	2.44	73.25	1.22	0.05	1.18	1.26
redo write size count (8KB)	125	0.03	1.04	0.02	0.00	0.02	0.02
redo write size count (16KB)	48	0.01	0.40	0.01	0.00	0.01	0.01
redo write size count (32KB)	25	0.01	0.21	0.00	0.00	0.00	0.00
redo write size count (128KB)	42	0.01	0.35	0.01	0.00	0.01	0.01
redo write size count (256KB)	13	0.00	0.11	0.00	0.00	0.00	0.00
redo write size count (512KB)	8	0.00	0.07	0.00	0.00	0.00	0.00
redo write size count (1024KB)	9	0.00	0.08	0.00	0.00	0.00	0.00
redo write size count (inf)	13	0.00	0.11	0.00		0.00	0.00
redo write time	880	0.24	7.33	0.12	0.01	0.12	0.13
redo write time (usec)	8,805,422	2,444.36	73,378.52	1,222.18	56.17	1,182.47	1,261.90
redo write total time	8,998,357	2,497.92	74,986.31	1,248.96	55.60	1,209.65	1,288.27
redo write worker delay (usec)	791,153	219.62	6,592.94	109.81	3.22	107.53	112.09
redo write worker delay count	9,121	2.53	76.01	1.27	0.06	1.23	1.31
redo writes	9,073	2.52	75.61	1.26	0.06	1.22	1.30
user commits	120	0.03	1.00	0.02	0.01	0.01	0.03
DBWR fusion writes	279	0.08	2.33	0.04	0.00	0.04	0.04
DBWR object drop buffers written	8	0.00	0.07	0.00		0.00	0.00
DBWR transaction table writes	388	0.11	3.23	0.05	0.01	0.05	0.06
DBWR undo block writes	4,089	1.14	34.08	0.57	0.47	0.23	0.90

Top SQL with Top Events

- Top SQL statements by DB Time along with the top events by DB Time for those SQLs.
- % Activity is the percentage of DB Time due to the SQL.
- % Event is the percentage of DB Time due to the event that the SQL is waiting on.
- % Row Source is the percentage of DB Time due to the row source for the SQL waiting on the event.
- Executions is the number of executions of the SQL that were sampled in ASH.

SQL ID	Plan Hash	Executions	% Activity	Event	% Event	Top Row Source	% Row Source	SQL Text
bgxtkrz2p3k08	4250213283	16	23.19	CPU + Wait for CPU	23.19	FIXED TABLE - FULL	20.29	SELECT VALUE FROM SYS.V_$PARAM...
gw788nxk7vbwg	4250213283	4	5.80	CPU + Wait for CPU	5.80	FIXED TABLE - FULL	4.35	SELECT VALUE FROM SYS.V_$PARAM...
cz8wbmy7k5bxn		3	4.35	CPU + Wait for CPU	4.35	** Row Source Not Available **	4.35	begin sys.dbms_aq_inv.internal...
69wffm3uch21b	426073554	2	2.90	CPU + Wait for CPU	2.90	FIXED TABLE - FULL	1.45	WITH events AS (SELECT instan...
c1kyf72yncjpw	3329590876	2	2.90	CPU + Wait for CPU	2.90	FIXED TABLE - FULL	2.90	WITH ash_rows AS (SELECT inst...

Back to Active Session History(ASH) Report
Back to Top

Top SQL with Top Row Sources

- Top SQL statements by DB Time along with the top row sources by DB Time for those SQLs.
- % Activity is the percentage of DB Time due to the SQL.
- % Row Source is the percentage of DB Time spent on the row source by that SQL.
- % Event is the percentage of DB Time spent on the event by the SQL executing the row source.
- Executions is the number of executions of the SQL that were sampled in ASH.

SQL ID	Plan Hash	Executions	% Activity	Row Source	% Row Source	Top Event	% Event	SQL Text
bgxtkrz2p3k08	4250213283	16	23.19	FIXED TABLE - FULL	20.29	CPU + Wait for CPU	20.29	SELECT VALUE FROM SYS.V_$PARAM...
				HASH JOIN	2.90	CPU + Wait for CPU	2.90	
gw788nxk7vbwg	4250213283	4	5.80	FIXED TABLE - FULL	4.35	CPU + Wait for CPU	4.35	SELECT VALUE FROM SYS.V_$PARAM...
				HASH JOIN	1.45	CPU + Wait for CPU	1.45	
cz8wbmy7k5bxn		3	4.35	** Row Source Not Available **	4.35	CPU + Wait for CPU	4.35	begin sys.dbms_aq_inv.internal...
69wffm3uch21b	426073554	2	2.90	FIXED TABLE - FULL	1.45	CPU + Wait for CPU	1.45	WITH events AS (SELECT instan...
				FIXED TABLE - FIXED INDEX	1.45	CPU + Wait for CPU	1.45	
c1kyf72yncjpw	3329590876	2	2.90	FIXED TABLE - FULL	2.90	CPU + Wait for CPU	2.90	WITH ash_rows AS (SELECT inst...

	Total								Per Execution							Percentage of Total							
SQL Id	Cluster (s)	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Execs	Cluster (s)	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Cluster	DB time	DB CPU	Gets	IO Wait	Reads	Execs	SQL Text
05s9358mm6vrr	0.59	39.67	35.70	0.94	1,192,000	3,572	1	1	0.59	39.67	35.70	0.94	1,192,000.00	3,572.00	1.00	13.06	63.47	59.64	63.16	28.65	85.60	0.00	begin dbms_feature_usage_inter...
fy4t9uzppnqbw	0.39	1.09	0.82	0.01	102,068	0	1	1	0.39	1.09	0.82	0.01	102,068.00	0.00	1.00	8.57	1.74	1.37	0.40	2.45	0.00	0.00	SELECT TO_CHAR(current_timesta...
3mkdy3ns38nxf	0.30	1.45	1.18	0.02	46,650	42	6,018	1	0.30	1.45	1.18	0.02	46,650.00	42.00	6,018.00	6.58	2.32	1.97	1.58	1.12	1.01	0.00	delete /* QOSH:PURGE_OLD_STS *...
1j6tnz8fcm4c3	0.23	6.57	6.17	0.17	377,005	312	40,240	5	0.05	1.31	1.23	0.03	75,401.00	62.40	8,048.00	5.00	10.51	10.30	11.45	9.06	7.48	0.00	delete /* QOSH:PURGE_OLD_STS *...
f7xnvp44nukp7	0.08	0.18	0.10	0.00	1,674	1	253	253	0.00	0.00	0.00	0.00	6.62	0.00	1.00	1.82	0.29	0.17	0.04	0.04	0.02	0.20	insert into wrm$_snapshot_deta...
43w0r9122v7jm	0.06	1.02	0.81	0.10	24,203	207	1	1	0.06	1.02	0.81	0.10	24,203.00	207.00	1.00	1.27	1.63	1.35	7.02	0.58	4.96	0.00	select max(bytes) from dba_seg...
03guhbfpak0w7	0.04	0.15	0.09	0.08	1,721	158	430	274	0.00	0.00	0.00	0.00	6.28	0.58	1.57	0.89	0.25	0.15	5.55	0.04	3.79	0.21	select /*+ index(idl_ub1$ i_id...
c179sut1vgpc8	0.04	2.60	2.55	0.00	150	0	69	24	0.00	0.11	0.11	0.00	6.25	0.00	2.88	0.81	4.16	4.26	0.00	0.00	0.00	0.02	INSERT /*+ LEADING(@"SEL$F5BB7...

	Total								Per Execution							Percentage of Total							
SQL Id	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Cluster (s)	Execs	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Rows	Cluster (s)	DB time	DB CPU	IO Wait	Gets	Reads	Cluster	Execs	SQL Text
0gwc9vhhygvbb	208.13	203.51	0.00	155,894	0	3,389	0.00	3,389	0.06	0.06	0.00	46.00	0.00	1.00	0.00	332.99	339.98	0.00	3.75	0.00	0.00	2.65	DECLARE po dbms_aqadm.aq$_purg...
3kqrku32p6sfn	0.00	5.63	5.53	38	0	0	0.00	4	0.00	1.41	1.38	9.50	0.00	0.00	0.00	0.00	9.00	9.23	0.00	0.00	0.00	0.00	MERGE /*+ OPT_PARAM('_pa

Some of the SQL Texts of my contact's db:

c179sut1vgpc8 INSERT /*+ LEADING(@"SEL$F5BB74E1" "H"@"SEL$2" "A"@"SEL$1") USE_NL(@"SEL$F5BB74E1" "A"@"SEL$1") */ INTO WRH$_ACTIVE_SESSION_HISTORY (snap_id, dbid, instance_number , sample_id, sample_time, sample_time_utc, usecs_per_row , session_id, session_serial#, session_type , flags , user_id , sql_id, sql_child_number, sql_opcode, force_matching_signature , top_level_sql_id, top_level_sql_opcode , sql_plan_hash_value, sql_full_plan_hash_value , sql_adaptive_plan_resolved , sql_plan_line_id , sql_plan_operation#, sql_plan_options# , sql_exec_id, sql_exec_start , plsql_entry_object_id, plsql_entry_subprogram_id , plsql_object_id, plsql_subprogram_id , qc_instance_id, qc_session_id, qc_session_serial#, px_flags , event_id, seq#, p1, p2 , p3 , wait_time, time_waited , blocking_session, blocking_session_serial#, blocking_inst_id , current_obj#, current_file#, current_block#, current_row# , top_level_call#, consumer_group_id, xid, remote_instance#, time_model , service_hash, program, module, action, client_id, machine, port, ecid , dbreplay_file_id, dbreplay_call_counter , con_dbid, per_pdb , tm_delta_time, tm_delta_cpu_time, tm_delta_db_time, delta_time, delta_read_io_requests, delta_read_io_bytes, delta_write_io_bytes, delta_interconnect_io_bytes, pga_allocated, temp_space_allocated , dbop_name, dbop_exec_id) (SELECT /*+ PARAM('_module_action_old_length', 0) */:snap_id, :dbid, :instance_number , a.sample_id, a.sample_time, a.sample_time_utc , a.usecs_per_row * :disk_filter_ratio , a.session_id, a.session_serial#, a.session_type , decode(a.flags, 0, to_number(NULL), a.flags) , a.user_id , a.sql_id, a.sql_child_number, a.sql_opcode, a .force_matching_signature , a.top_level_sql_id, a.top_level_sql_opcode , a.sql_plan_hash_value, a.sql_full_plan_hash_value , a.sql_adaptive_plan_resolved , a.sql_plan_line_id , a.sql_plan_operation#, a.sql_plan_options# , a.sql_exec_id, a.sql_exec_start , a.plsql_entry_object_id, a.plsql_entry_subprogram_id , a.plsql_object_id, a.plsql_subprogram_id , a.qc_instance_id, a.qc_session_id, a.qc_session_serial#, a.px_flags , a.event_id, a.seq#, a.p1, a.p2, a.p3 , a.wait_time, a.time_waited , a.blocking_session, a.blocking_session_serial#, a.blocking_inst_id , a.current_obj#, a.current_file#, a.current_block#, a.current_row# , a.top_level_call#, a.consumer_group_id, a.xid, a.remote_instance# , a.time_model , a.service_hash , substrb(a.program, 1, 64) , a.module, a.action, a.client_id, a.machine, a.port, a.ecid , a.dbreplay_file_id, a.dbreplay_call_counter , a.con_dbid, a.con_id , decode(a.tm_delta_time, 0, to_number(null), a.tm_delt ta_time), decode(a.tm_delta_time, 0, to_number(null), a.tm_delta_cpu_time), decode(a.tm_delta_time, 0, to_number(null), a.tm_delta_db_time), decode(a.delta_time, 0, to_number(null), a.delta_time), decode(a.delta_time, 0, to_number(null), decode(a.delta_read_io_requests, 0, to_number(null), a.delta_read_io_requests)), decode(a.delta_time, 0, to_number(null), decode(a.delta_write_io_requests, 0, to_number(null), a.delta_write_io_requests)), decode(a.delta_time, 0, to_number(null), decode(a.delta_read_io_bytes, 0, to_number(null), a.delta_read_io_bytes)), decode(a.delta_time, 0, to_number(null), decode(a.delta_write_io_bytes, 0, to_number(null), a.delta_write_io_bytes)), decode(a.delta_time, 0, to_number(null), decode(a.delta_interconnect_io_bytes, 0, to_number(null), a.delta_interconnect_io_bytes)), decode(a. pga_allocated, 0, to_number(null), a.pga_allocated), decode(a.pga_allocated, 0, to_number(null), decode(a.temp_space_allocated, 0, to_number(null), a.temp_space_allocated)) , a.dbop_name, a.dbop_exec_id FROM x$ash a, (SELECT h.sample_addr, h.sample_id FROM x$kewash h WHERE ((h.sample_id >= :begin_flushing) and (h.sample_id < :latest_sample_id)) and (nlssort(h.is_awr_sample, 'nls_sort=BINARY') = nlssort('Y', 'nls_sort=BINARY'))) shdr WHERE (1 = 1) and shdr.sample_addr = a.sample_addr and shdr.sample_id = a.sample_id and nlssort(a.need_awr_sample, 'nls_sort=BINARY') = nlssort('Y', 'nls_sort=BINARY'))

f7xnvp44nukp7 insert into wrm$_snapshot_details (snap_id, dbid, instance_number, table_id, begin_time, end_time, step_id) values (:snap_id, :dbid, :instance_number, :bind1, :bind2, :bind3, :step_id)

3mkdy3ns38nxf delete /* QOSH:PURGE_OLD_STS *//*+ dynamic_sampling(4) */ from sys.wri$_optstat_histhead_history where savtime < :1 and savtime > timestamp '1900-01-01 00:00:00 -0:0' and rownum <= :2

1j6tnz8fcm4c3 delete /* QOSH:PURGE_OLD_STS *//*+ dynamic_sampling(4) */ from sys.wri$_optstat_histgrm_history where savtime < :1 and savtime > timestamp '1900-01-01 00:00:00 -0:0' and rownum <= :2

3kqrku32p6sfn	MERGE /*+ OPT_PARAM('_parallel_syspls_obey_force' 'false') */ INTO OPTSTAT_USER_PREFS$ D USING (SELECT * FROM (SELECT O.OBJ#, SYSTIMESTAMP CHGTIME, ROUND(MAX(S.DELTA_READ_IO_BYTES/ S.DELTA_TIME), 3) SCANRATE FROM GV$ACTIVE_SESSION_HISTORY S, GV$SQL_PLAN P, OBJ$ O, USER$ U WHERE S.INST_ID = P.INST_ID AND S.SQL_ID = P.SQL_ID AND S.SQL_PLAN_HASH_VALUE = P.PLAN_HASH_VALUE AND S.SQL_PLAN_LINE_ID = P.ID AND P.OBJECT_NAME = O.NAME AND (:B4 IS NULL OR O.OBJ# = :B4) AND O.OWNER# = U.USER# AND P.OBJECT_OWNER = U.NAME AND O.TYPE# = 2 AND S.CON_ID = SYS_CONTEXT('userenv', 'con_id') AND P.CON_ID = SYS_CONTEXT('userenv', 'con_id') AND S.SQL_PLAN_OPERATION = 'TABLE ACCESS' AND P.OPERATION = 'TABLE ACCESS' AND S.DELTA_TIME > 0 AND S.DELTA_READ_IO_BYTES > 0 AND FROM_TZ(S.SAMPLE_TIME, :B3) > :B2 AND (:B1 = 'ON' OR (:B1 = 'HADOOP_ONLY' AND EXISTS (SELEC T NULL FROM SYS.EXTERNAL_TAB$ ET WHERE ET.OBJ# = O.OBJ# AND TYPE$ IN ('ORACLE_HIVE', 'ORACLE_HDFS', 'ORACLE_BIGDATA')))) GROUP BY O.OBJ#) C WHERE NOT EXISTS (SELECT NULL FROM OPTSTAT_USER_PREFS$ PR WHERE PNAME = 'SCAN_RATE' AND C.OBJ# = PR.OBJ# AND (PR.SPARE1 IS NULL OR C.SCANRATE <= TO_NUMBER(PR.VALCHAR))) SRC ON (D.OBJ#=SRC.OBJ# AND D.PNAME = 'SCAN_RATE') WHEN MATCHED THEN UPDATE SET D.VALCHAR= TO_CHAR(SRC.SCANRATE), D.CHGTIME=SRC.CHGTIME, D.SPARE1=1 WHEN NOT MATCHED THEN INSERT (OBJ#, PNAME, VALCHAR, CHGTIME, SPARE1) VALUES(SRC.OBJ#, 'SCAN_RATE', TO_CHAR(SRC.SCANRATE), SRC.CHGTIME, 1)

Top Timed Background Events

- % of bg time: % of background elapsed time
- Instance '*' - cluster wide summary
- '*' Waits, %Timeouts, Wait Time Total(s) : Cluster-wide total for the wait event
- '*' 'Wait Time Avg' : Cluster-wide average computed as (Wait Time Total / Event Waits)
- '*' Summary 'Avg Wait Time' : Per-instance 'Wait Time Avg' used to compute the following statistics
- '*' [Avg/Min/Max/Std Dev] : average/minimum/maximum/standard deviation of per-instance 'Wait Time
- '*' Cnt : count of instances with wait times for the event

	Wait		Event		Wait Time		
I#	Class	Event	Waits	%Timeouts	Total(s)	Avg Wait	%DB time
*		background cpu time			770.32		121.87
	Other	latch free	10,172	0.00	15.71	1.54ms	2.48
	Other	oracle thread bootstrap	249	0.00	13.64	54.78ms	2.16
	System I/O	control file sequential read	29,532	0.00	11.76	398.23us	1.86
	System I/O	log file parallel write	9,135	0.00	7.26	794.63us	1.15
	Other	LGWR all worker groups	7,229	0.00	7.23	1.00ms	1.14
	Other	IMR slave acknowledgement msg	21,606	0.00	6.21	287.57us	0.98
	Other	PX Deq: reap credit	254,182	100.00	3.31	13.03us	0.52
	System I/O	control file parallel write	2,758	0.00	2.23	806.90us	0.35
	System I/O	db file parallel write	4,790	0.00	2.15	448.43us	0.34
1		background cpu time			512.67		113.49
	Other	latch free	5,111	0.00	8.01	1.57ms	1.77
	Other	oracle thread bootstrap	127	0.00	7.14	56.18ms	1.58
	System I/O	control file sequential read	15,416	0.00	6.24	404.55us	1.38
	System I/O	log file parallel write	4,715	0.00	3.74	793.27us	0.83
	Other	LGWR all worker groups	3,592	0.00	3.63	1.01ms	0.80
	Other	IMR slave acknowledgement msg	10,803	0.00	3.17	293.71us	0.70
	Other	PX Deq: reap credit	129,529	100.00	1.67	12.87us	0.37
	Cluster	gc cr block 2-way	3,824	0.00	1.56	408.69us	0.35
	System I/O	db file parallel write	3,498	0.00	1.52	433.69us	0.34

Block Size
8192

Appendix 3: US revenue, profit, and pricing model

The pricing options of my service are to make the individual pay per user project, or pay per usage cycle, or to make the customers pay a monthly fee/yearly/weekly fee (based on activity or not), or indexing that fee on the number of requests or on the time spent on the user project or spent on the usage cycle, or indexing that fee on the number of tasks ran by the server for a user project.

First, due to the nature of the service, one customer can join and leave the pool of active users at any moment, and there is in fact no certitude that a given user will complete a user project at all times all year/month/week-round at the same frequency. Secondly, the usage cycle contains tasks aside of the user project (creating an account, designating billing methods, exporting results files, etc.). These side tasks would be priced per request, but that would likely deter users from accessing the website and considering creating user projects. Thirdly billing per user project makes each specific usage, which might vary in time and computing effort, be valued at a fixed priced. This means either using a middle-effort price, and top price, or a lowest-effort. Lowest effort price is simply unfair for myself. The app would often take way more effort than the lowest possible effort, and it would mean it usually work at a loss. Middle and highest-effort prices make no sense either, because users completing less complex project than the most complex imaginable (which is usually the case) would be unhappy overpaying. Finally, the app is programmed so that it makes close to no mistakes: there is an ai agent document info extractor in use, but I program it so that if it cannot extract a value being sure 100% it is the right value, it should not extract it and let user extract it. A 100% sure extraction implies the ai agent clearly reads the name of what I must find, alongside its value. The rest of the app makes no mistakes in theory within user projects, but if it did because of erroneous domain writing or other reasons, this would be on the provider (myself) to compensate users for that. Now, that makes each HTTP request a symbol of the complexity of the things discussed in the user project. And just as with a real person, the more time unavoidably taken to solve problems, the more you pay that person. Now, there is no useless HTTP requests in user projects, in the sense that each request in user projects provides expertise and works towards elivering the service. Therefore, opting for a per-HTTP request pricing within user project is very attractive in theory. But in practice, it is not the best option for two reasons. First, if users know they pay everytime they click on a button, they will be less satisfied with the company and the service than if they paid just once. Secondly, and even more so importantly, 1 customer spends 1 workday on a usage cycle on average, and if another human was providing the service for them, the customer would not have that 1 day of work required. Due to the low complexity of project filling, the customer pays around 200$ for someone or themselves to complete a user project (secretary-like salary plus side company charges)[12]. So they would be really reluctant of paying even $100 per user project, because that represents $300 of costs for them on average per usage project, and that’s often above what a low level equivalent service provider would ask for a day’s work (minimum $250). So I choose a pricing of $10, so customer charges for one usage cycle remain a bit under $250 at $210. the program has mechanisms to ensure that delivery of the expertise happens only once per project (you need to validate the last step of the project or receive the expertise, and then you cannot go back in the project to modify it. **Therefore, I choose pricing of 10 US dollars per user project, paid at project creation.**

---

12 HTTPs://www.indeed.com/cmp/Indeed/salaries/Secretary?period=DAILY

Knowing 1 usage cycle = 1 user project, and that in Scenario 1, and because the average number of new usage cycles a day per million inhabitants is 214.4, the profit before expenses is 214.14 ×350×10 dollars×21 workdays per month (because customers activity is calculated as if only on these days) = $15,739,290 per month in the US ($188,871,480 a year).